INTUTE ERP

Complete Technical Documentation

Backend API · Data Model · Web CRM · PDI Mobile App · Integrations · Deployment

Version 1.0 | 09 October 2026
Prepared for: Compage / Intute AI engineering team
Classification: Confidential - internal use only

Document Control

Item	Detail
Document	Intute ERP - Complete Technical Documentation
Version / date	1.0 - 09 October 2026
Systems covered	CRM_BACKEND (Node/Express API), CRM (React web app), pdi-erp-app (React Native mobile app)
Source of truth	Backend source at commit e353328d and the live PostgreSQL schema (information_schema) read on the date above
Audience	Software engineers, DevOps, QA and technical stakeholders maintaining or extending the ERP
Generation	Chapters come from docs/technical/chapters/*.md. Appendices A-C are regenerated from the route files and the database catalogue on every build (npm run docs:pdf).

Revision history

Version	Date	Author	Changes
1.0	09 Oct 2026	Rahul Srivastava	First complete technical documentation: backend, data model, web CRM, PDI mobile app, integrations, deployment, plus generated API, permission and schema appendices.

Contents

System Overview . . . 6
Purpose and scope . . . 6
Applications . . . 6
Technology stack . . . 6
Architecture . . . 8
Logical architecture . . . 8
Backend layering . . . 8
Request pipeline . . . 8
Startup sequence . . . 9
Real-time messaging . . . 9
Caching strategy . . . 9
Security and Access Control . . . 10
Authentication . . . 10
Authorisation model . . . 10
Roles . . . 10
Other protections . . . 11
Core, Sales, Invoicing and Dispatch Modules . . . 12
0. Cross-cutting plumbing that these modules rely on . . . 12
1. Auth (routes/auth.js, controllers/auth.controller.js, models/core/user.js) . . . 13
2. Core . . . 13
3. Sales . . . 18
4. Invoicing . . . 21
5. Dispatch . . . 22
6. Summary tables . . . 23
Manufacturing, Service, HR, Operations and Automation . . . 25
1. Manufacturing . . . 25
2. Service: Service Repair: /api/service-repair . . . 30
3. HR . . . 31
4. Operations . . . 34
5. Chatbot (chatbot/) . . . 35
6. Background jobs (jobs/), email, Drive, logger . . . 36
7. Tests (tests/) . . . 37
8. Migration scripts (scripts/migrations/) . . . 38
9. Consolidated known gaps / risks (from code) . . . 38
Pre-Dispatch Inspection (PDI) Subsystem . . . 40
1. Overview . . . 40

2. Template engine architecture . . . 41
3. Built-in template specifics . . . 43
4. Tolerance evaluation and rounding (models/operations/pdi/tolerance.js) . . . 44
5. Report lifecycle . . . 45
6. Photos: limits and optimisation . . . 47
7. Lots / batches (AutoNXT) . . . 48
8. Access control . . . 49
9. Database tables and key columns . . . 50
10. API endpoint reference . . . 51
11. Error codes . . . 52
12. Mobile-app compatibility notes . . . 53
13. Test coverage (tests/pdi_.test.js, Jest) . . . 53
14. Web frontend map (CRM) . . . 54
15. Notable gotchas and caveats . . . 55

Client Applications: Web CRM and PDI Mobile App . . . 56

1. Web CRM (React SPA) . . . 56
2. PDI Mobile App (pdi-erp-app) . . . 63
3. Observations useful for the documentation . . . 66

Deployment and Operations . . . 67

Environments and configuration . . . 67
Backend build and release . . . 67
Database migrations . . . 67
Logging and monitoring . . . 67
Development workflow . . . 68

Appendix A - REST API Endpoint Catalogue . . . 69

A.1 Authentication (5) . . . 69
A.2 Core (39) . . . 69
A.3 Sales (33) . . . 70
A.4 Manufacturing (51) . . . 71
A.5 Invoicing (9) . . . 72
A.6 Dispatch (10) . . . 72
A.7 HR (14) . . . 72
A.8 Operations (incl. PDI) (39) . . . 73
A.9 Service & Repair (12) . . . 74
A.10 Chatbot (6) . . . 74

Appendix B - Role Permission Matrix (live data) . . . 75

Appendix C - Database Schema Reference (live) . . . 77

C.1 Table summary . . . 77

C.2 Table definitions 78

Appendix D - Glossary 94

System Overview

Purpose and scope

The Intute ERP is a role-based enterprise resource planning and CRM platform. It runs the whole business cycle for two companies that share one installation:

- **Compage**: the manufacturing business. It covers the sales pipeline, design, manufacturing and work orders, stores, dispatch, accounts, service and repair, and HR and attendance for Compage staff.
- **Intute AI (IA)**: a separate people-and-orders space with its own roles (`ia_employee`, `ia_hr`). It covers IA activities, IA orders, IA invoices and payslips.

This document describes the system as implemented in source code and in the live database schema on the date shown on the cover. It covers the backend API, the PostgreSQL data model, the web CRM, the PDI mobile app, the integrations, the background jobs, security, and deployment.

Applications

Application	Repository folder	Technology	Role
ERP Backend API	`CRM_BACKEND`	Node.js 22, Express 4, PostgreSQL, Redis, Socket.IO	The REST API and real-time hub used by all clients
Web CRM	`CRM`	React 19, Vite 6, Redux Toolkit, React Router 7, Tailwind CSS	Browser front end for all 14 roles
PDI Mobile App	`pdi-erp-app`	React Native 0.87, React Navigation, AsyncStorage	On-site Pre-Dispatch Inspection capture for production technicians

Technology stack

Layer	Technology	Notes
Runtime	Node.js 22.17 (Docker `node:22.17.0-alpine`)	Pinned because nodemailer 10 and sanitize-html need Node 20 or later
Web framework	Express 4.19	`helmet`, `cors`, `cookie-parser`, 40 MB JSON body limit
Database	PostgreSQL on AWS RDS	Accessed through `pg` Pool (max 20 connections, TLS, 10 s connect timeout)
Cache	Redis 4 client	`REDIS_URL`; custom `delPattern()` helper for wildcard invalidation
Real-time	Socket.IO 4.8	Path `/socket.io`; CORS restricted to `FRONTEND_URL`
Authentication	JWT (`jsonwebtoken`) + `bcryptjs`	1 h token for the web, 12 h for the mobile app
Validation	`express-validator`, `sanitize-html`	Per-route validators
Documents	PDFKit 0.17, `svg-to-pdfkit`, `sharp`	Roboto fonts in `assets/fonts`
Email	Nodemailer 10 (Gmail SMTP)	HTML templates in `utils/emailTemplates.js`
File storage	Google Drive API (service account)	Photos, invoices, attachments
Scheduling	`node-cron` 4	All jobs run on the Asia/Kolkata time zone
Logging	Winston 3	`logs/error.log`, `logs/combined.log`, console
Uploads	`multer` 2	In-memory storage, then sent to Drive
Testing	Jest 29 + Supertest 7	`tests/` (about 28 suites, mostly PDI)
Front end	React 19, Redux Toolkit 2, Axios, socket.io-client, Tailwind 3, Bootstrap 5, lucide-react, react-select, xlsx, qrcode	Built with Vite

Layer	Technology	Notes
Mobile	React Native 0.87, @react-navigation/native-stack, AsyncStorage, react-native-image-crop-picker, react-native-blob-util	Android-first field app

Architecture

Logical architecture

All three clients talk to one Express process. That process owns all business rules and is the only component that reaches the database.



Backend layering

The code follows a strict three-layer layout, and each business domain gets the same folder name in every layer:

- **routes/<domain>/** declares the endpoints and wires up middleware: `authenticateToken`, `checkPermission(module, action)`, validators, `multer`, and role guards.
- **controllers/<domain>/** handles HTTP. It parses requests, applies business rules, emits Socket.IO events, and maps errors to status codes.
- **models/<domain>/** holds the SQL access through the shared `pg` pool. Multi-step writes run inside `BEGIN/COMMIT` transactions on a single client.

The domain folders are `core`, `sales`, `manufacturing`, `invoicing`, `dispatch`, `hr`, `operations` and `service`. A separate `chatbot/` tree has its own routes, services, models, and intent and formatter utilities.

Request pipeline

These steps run for every request, in order (see `server.js`):

1. `app.set('trust proxy', 1)` makes `req.ip` the real client address behind nginx, so the IP-keyed rate limiter works.
2. `helmet()` sets the security headers.
3. `cors({ origin: FRONTEND_URL, credentials: true })`.
4. `cookieParser()` and `express.json({ limit: '40mb' })`. The large limit exists because the PDI generator sends base64 photos.
5. The global rate limiter (`middleware/rateLimit.js`) allows 5,000 requests per 10 minutes, keyed by user id when known and by IP otherwise. When exceeded it returns `429` with `RATE_LIMIT_EXCEEDED`. The inventory router adds its own tighter limit of 100 requests per 15 minutes.
6. `req.io` is attached so controllers can emit real-time events.
7. The domain routers run.
8. The global `errorHandler` returns `{ error, code }` with `err.status` or 500.

Startup sequence

`initializeServer()` does the following:

1. Runs `SELECT 1` to check the database connection.
2. Recovers PDI lots stuck in the `Finalizing` state (left there if a previous process died mid-render) by setting them back to `In Progress`.
3. Connects Redis.
4. Starts listening on `PORT` (default 8000).

The cron jobs are loaded only when `server.js` is the main module. Tests can `require('../server')` for Supertest without binding a port or starting timers. On `SIGTERM` the HTTP server closes gracefully.

Real-time messaging

A single Socket.IO server is created in `server.js` and exposed as `app.get('io')` and as `req.io`. Controllers broadcast mutation events, such as stock updates after an order dispatches and record changes for enquiries, queries and orders, so open dashboards refresh live. The web client connects with `auth: { token }`, uses WebSocket with polling fallback, and retries up to 5 times with a 2 s delay. Its connection status is kept in Redux (`socketStatus`).

Caching strategy

Redis caches read-heavy, expensive list queries, such as part drawings, inventory, queries and enquiry templates. Keys are built from the query parameters (limit, offset, search, user and role). Entries normally expire after 300 s, and every write invalidates them through `redisClient.delPattern()`. A Redis outage is logged but does not stop startup.

Security and Access Control

Authentication

- `POST /api/auth/login` checks the email and bcrypt password hash. It returns `{ role, token, name, user_id }`. The role string comes from `ROLE_MAP` in `controllers/auth.controller.js`, keyed by `role_id`.
- The JWT payload carries `user_id`. Tokens last **1 hour** by default. Requests that send an `X-App-Version` header (the PDI mobile app does) get a **12-hour** token, so a long on-site inspection is not interrupted.
- `authenticateToken` (`middleware/auth.js`) reads `Authorization: Bearer <token>`, verifies it, and **re-reads the user and role from the database on every request**. A role change therefore takes effect immediately.
- Error contract: `401 AUTH_NO_TOKEN`; `403 AUTH_INVALID_TOKEN`, `AUTH_INVALID_USER` or `AUTH_INVALID_TOKEN` for a bad payload. If the database lookup fails, the response is `503 AUTH_UNAVAILABLE`. The clients do not log the user out on a 503.

Authorisation model

The backend enforces access in three ways:

1. **Permission matrix (RBAC).** `checkPermission(module, action)` looks up the `permissions` table (`role_id`, `module`, `can_read`, `can_write`, `can_delete`); `can_create` maps to `can_write`. A denied request gets `403 PERM_DENIED`. `hasPermission()` offers the same check to model code, for example the rule for editing a finalized PDI report.
2. **Role allow-lists.** `requirePdiAccess` (`middleware/pdiAccess.js`) admits only `admin` and `production`. It accepts either `roles.role_name` or the `ROLE_MAP` name, and is deliberately independent of the permissions table. The PDI template administration routes are admin-only (`role_id === 1`). Activities are limited to role ids 9, 10, 11 and 12 (Compage and IA people roles).
3. **Authenticated-only routes.** Some document generators and utility routes need only a valid token. These are listed as "Any authenticated" in the API catalogue.

The web CRM adds a client-side gate on top: `routeConfig.jsx` gives every route an `allowedRoles` list, and `App.jsx` redirects users to their role dashboard when they open a path their role does not allow. This gate only improves navigation. The API remains the source of truth.

Roles

role_id	DB role_name	Client role key	Company	Landing route
1	Admin	admin	Compage	/admin-dashboard
2	Customer	customer	External	/customer-dashboard
3	Sales	sales	Compage	/sales-dashboard
4	Design	design	Compage	/design-dashboard
5	Production	production	Compage	/production-dashboard
6	store	store	Compage	/store-dashboard
7	Dispatch	dispatch	Compage	/dispatch-dashboard
8	Accounts	accounts	Compage	/accounts-dashboard
9	employee	employee	Compage	/employee-dashboard
10	hr	hr	Compage	/hr-dashboard
11	IA_employee	ia_employee	Intute AI	/ia-employee-dashboard
12	IA_HR	ia_hr	Intute AI	/ia-hr-dashboard
13	service_repair	service_repair	Compage	/service-repair-dashboard
15	representative	representative	Compage (field sales)	/representative-dashboard

Other protections

- `helmet` sets the security headers. CORS allows only `FRONTEND_URL` (credentials enabled).
- All SQL uses parameterised queries (`$1..$n`).
- Rich text is cleaned with `sanitize-html`.
- Secrets are read from `.env` and `secrets/` (the Google service account). Both are kept out of git, but the Dockerfile copies `.env` into the image.
- Rate limiting is global and per router.
- The logs keep error stacks, while client responses carry only a message and a code.

Core, Sales, Invoicing and Dispatch Modules

This chapter covers the commercial side of the ERP: authentication, customers, orders, inventory and stock, dashboards and reports, the sales pipeline (enquiries, requirements, quotations, proforma invoices, purchase orders, price list), invoicing, and dispatch. That last group includes delivery challans and Intute AI orders.

All paths are relative to `CRM_BACKEND/`. Layout: `routes/<domain>/*.js` -> `controllers/<domain>/*.controller.js` -> `models/<domain>/*.js`. Mounts live in `server.js`.

0. Cross-cutting plumbing that these modules rely on

- **Socket.IO**: `server.js` creates one `Server` (path `/socket.io`, CORS = `FRONTEND_URL`), stores it with `app.set('io')` / `app.set('socketio')`, and a global middleware sets `req.io = io` on every request. The connection handler only logs connect/disconnect. **No code joins sockets to rooms**, so the `io.to('user_<id>').emit('notification', …)` calls in the enquiry model reach no one unless a room join exists elsewhere (none found in `server.js` or `config/socket.js`). `config/socket.js` (`initSocket`) is a second, unused factory.
- **Routers that override `req.io`**: `routes/sales/quotation.js`, `proforma.js`, `purchaseOrder.js`, `routes/dispatch/deliveryChallan.js` and `iaOrders.js` reset `req.io = req.app.get('io')` and then set `req.io.user = req.user`. Because `io` is a process-wide singleton, `io.user` is shared state. Concurrent requests can overwrite it, and the IA Orders model reads `io.user.user_id` as `created_by` (race condition).
- **Auth middleware** (`middleware/auth.js`):
  - `authenticateToken` reads only `Authorization: Bearer <jwt>`. The README says cookies are also accepted, but the code does not do that.
  - It verifies with `JWT_SECRET`, then re-loads the user and `role_name` from `users`/`roles` on every request.
  - Error codes: `AUTH_NO_TOKEN` (401), `AUTH_INVALID_TOKEN` (403), `AUTH_INVALID_USER` (403), and `AUTH_UNAVAILABLE` (503) when the DB lookup fails. A DB error is deliberately not treated as a bad token, so clients do not log the user out.
  - `checkPermission(module, action)` runs `SELECT <action> FROM permissions WHERE role_id AND module`, maps `can_create` to `can_write`, and returns `PERM_DENIED` (403) or `PERM_CHECK_FAILED` (500).
- **Redis** (`config/redis.js`): node-redis v4 client, `REDIS_URL` (default `redis://localhost:6379`). It exposes a `delPattern` helper, but most controllers do `redis.keys(pattern)` + `del` inline. Several controllers call `redis.del('<prefix>_*')` with a literal wildcard string. `DEL` does not expand globs, so those calls invalidate nothing (details per module below).
- **Google Drive** (`services/googleDrive.js`): service account key at `GOOGLE_SERVICE_ACCOUNT_PATH` (default `secrets/drive-sa.json`), scope `drive`, target folder `GOOGLE_DRIVE_FOLDER_ID`.
  - `uploadBufferToDrive` creates the file (`supportsAllDrives`), grants `anyone:reader`, and returns `directUrl = https://drive.google.com/uc?export=view&id=<id>` plus view and download links.
  - `uploadBufferToDrivePrivate` (no public permission), `streamFileToDrive` (proxies a file to the response with `Cache-Control: public, max-age=86400`), and `deleteDriveFile` also exist. `deleteDriveFile` *trashes* the file rather than deleting it, because the service account lacks `canDelete` on the Shared Drive.
- **Body limit**: `express.json({ limit: '40mb' })`, sized for the PDI generator. `trust proxy = 1` (single nginx hop) is set so the global IP rate limiter can see real client IPs.

1. Auth (`routes/auth.js`, `controllers/auth.controller.js`, `models/core/user.js`)

Purpose: credential login and JWT issuance for all 14 role IDs.

Endpoint	Behaviour
`POST /api/auth/login`	Requires `email` + `password` (else 400 `AUTH_MISSING_FIELDS`). Looks up `users JOIN roles` by email. If the user is not found or bcrypt compare fails, returns 401 `AUTH_INVALID_CREDENTIALS` (the same message for both). If `password_hash` is null, returns 500 `AUTH_NO_PASSWORD`. Returns `{ role, token, name, user_id }`.
`GET /api/auth/verify-token`	Echoes `req.user` (populated by the middleware).
`GET /api/auth/user`	Returns `{ name }`, or 404 `USER_NOT_FOUND`.
`POST /api/auth/logout`	Stateless: logs and returns a message. There is no token blacklist.
`PUT /api/auth/update-password`	`User.updatePassword`: old password must match (401 `AUTH_INVALID_OLD_PASSWORD`), new password must be at least 6 characters (400 `AUTH_PASSWORD_TOO_SHORT`), bcrypt cost 10.

- **JWT payload:** `{ user_id, role_id, name }`, signed with `JWT_SECRET`.
- **Token TTL:** `1h` by default, `12h` if the request carries an `X-App-Version` header. The PDI mobile app sends this header so tokens do not expire mid-inspection. The comment notes the header is self-declared and only widens expiry for someone who already has valid credentials.
- **Role map** (`ROLE_MAP` in the controller):

ID	Role	ID	Role
1	admin	8	accounts
2	customer	9	employee
3	sales	10	hr
4	design	11	ia_employee
5	production	12	ia_hr
6	store	13	service_repair
7	dispatch	15	representative

There is no role 14. Any unmapped role ID returns `'unknown'`.

- **`User.create`** (used by customer creation):
  - Runs inside its own client transaction.
  - First runs `setval('users_user_id_seq', GREATEST(MAX(user_id)+1, nextval(...)))` to resync the sequence.
  - Inserts the user with default `role_id = 2`. If no password is given, it uses a hard-coded default bcrypt hash.
  - Maps `23505` on `users_email_key` to 400 `DUPLICATE_EMAIL`, and on `users_pkey` to 500 `USER_ID_CONFLICT`.
  - Also signs a 1h JWT that the customer-create flow does not use.

2. Core

2.1 Customers (`routes/core/customers.js`, `controllers/core/customers.controller.js`, `models/core/customer.js`)

Purpose: customer master. Each customer is a `users` row (role 2) plus a `customers` row (contact, GST, addresses).

- **Endpoints:**
  - `GET /api/customers/search?q=` (auth only, no permission check): `ILIKE` on `users.name`, limit 20.
  - `GET /api/customers` (`Customers.can_read`): paginated, with `limit` capped at 100 and a non-negative `offset`.
  - `POST /api/customers` (`Customers.can_create`).
- **Validation** (route middleware `validateCustomerInput`):
  - name at least 3 characters; contact_person at least 3 characters; city required.
  - phone matches `^[0-9]{10}$`; email matches a basic regex.

  - optional GST matches `^[0-9A-Z]{15}$`; shipping and billing addresses required.
  - All failures return 400 `VALIDATION_ERROR`.
- **Create flow:** `Customer.create` opens a transaction on its own client, but calls `User.create`, which uses a *separate* pooled connection and transaction. The user row therefore commits independently. If the `customers` insert then fails, an orphan user is left behind.
- **List query:** joins `customers`, `users`, `orders` and `queries` (LEFT JOINs) and returns `COUNT(DISTINCT order_id)` and `COUNT(DISTINCT query_id)` per customer.
- **Caching:** key `customers_<limit>_<offset>`, TTL 3600 s. On create, all keys matching `customers_*_*` are deleted and the `customerUpdate` socket event is emitted.
- **Unused method:** `Customer.update` exists in the model but no route uses it. It runs the customer and user updates in parallel with no transaction.

2.2 Users (`routes/core/users.js`, `controllers/core/users.controller.js`)

- `GET /api/users/customers` (`Customers.can_read`): all users with `role_id = 2`, as `user_id, name`.
- `GET /api/users/employees-hr`: returns team members within the caller's organisation.
  - IA roles [11, 12] see users with roles 11 and 12.
  - Compage roles [9, 10] see users with roles 9 and 10.
  - Any other role gets 403.

2.3 Orders (`routes/core/orders.js`, `controllers/core/orders.controller.js`, `models/core/order.js`)

Purpose: customer sales orders for finished goods (`inventory` products), with inventory reservation (holds) and stock consumption at dispatch.

Endpoints

Method / path	Permission	Notes
`GET /api/orders`	`Orders.can_read`	Keyset pagination on `created_at` (`cursor`, `limit` up to 100). Admin (1), production (5) and sales (3) see all orders; other roles see only their own (`user_id`).
`GET /api/orders/export`	`Orders.can_read`	CSV export (declared before `/:id`). No cache; fetches up to 100000 rows under the same visibility rules. Columns: Order ID, Customer Name, Items (`name (Qty: n)` joined with `; `), Total Amount (INR), Status, Status Reason, Target Delivery, Payment Status, Created At. Every field is double-quote escaped. Sets the `X-Total-Exported` header and the filename `orders_export_YYYY-MM-DD.csv`.
`POST /api/orders`	`Orders.can_write`	Admin and sales may pass `user_id` to order on a customer's behalf; other roles always order for themselves. Requires a non-empty `items`.
`PUT /api/orders/:id/update`	`Orders.can_write` + controller role check {1, 5, 3}	Rejects `status: 'Cancelled'`. `payment_status` must be `Pending` or `Paid`. An empty `items` array is rejected.
`POST /api/orders/:id/cancel`	`Orders.can_write`	Body `{ goods_returned }`.

Status state machine (`STATUS_RANK` in `models/core/order.js`)

Status	Rank
Pending	0 (initial)
Processing	1
Testing	2
Ready for Shipment	3
Shipped	4 (dispatch threshold)
Partially Delivered	5 (requires `status_reason`)

Status	Rank
Delivered	6
Cancelled	not ranked; terminal; only via `cancelOrder()`

- Both the old and new status must be ranked, otherwise the update fails with `Invalid status transition`.
- **Downgrades** are blocked only once the old rank is at least Shipped. Before shipping, moving backwards (for example Testing to Processing) is allowed.
- **Line items** become immutable once the order is at Shipped or later. The comparison sorts the incoming items by product and compares product, quantity and price against the stored rows.
- **Errors:** the controller maps all update and cancel errors to HTTP 400 (create uses `error.status || 400`).

Inventory holds (reservation)

- **On create** (single transaction): `checkInventoryAvailability` computes `stock_quantity - SUM(active holds)` per product, but only logs warnings. **Negative stock is allowed.** The flow then:
  1. inserts the order with status `Pending` and payment status `Pending`;
  2. bulk-inserts `order_items` via `unnest`;
  3. inserts one `inventory_holds` row per item with `reason='Order fulfillment'`, `reference_type='ORDER'`, `reference_value=<order_id>`, `status='ACTIVE'`.
- **On item change** (before shipping): releases the order's active holds (`status='RELEASED'`, `released_at=NOW()`), deletes and re-inserts the items, and creates new holds.
- **Dispatch hook:** fires the first time the rank crosses from below 4 to 4 or above, including direct jumps such as Testing to Delivered. It:
  - runs `UPDATE inventory SET stock_quantity = stock_quantity - qty` for every item (may go negative; negatives are logged as a warning);
  - releases the holds;
  - emits `stockUpdate { product_id, stock_quantity }` per product.
- **Cancel rules:**
  - Only admin or the order owner may cancel (otherwise `Unauthorized`).
  - An already-cancelled order is rejected.
  - A Delivered order needs `goods_returned=true`.
  - Shipped or Delivered with goods returned: stock is added back and `stockUpdate` is emitted.
  - Not shipped: holds are released.
  - Shipped or Delivered without goods returned: nothing is restocked; the code comment says to use the inventory `accept-return` endpoint later.
  - **Gap:** `wasShipped` only checks `Shipped`/`Delivered`, so cancelling a `Partially Delivered` order takes the "never shipped" branch. It releases (already released) holds and does not restore stock.
- **Invoice event:** create, update and cancel each look up `customer_invoices WHERE order_id` and emit `invoiceUpdate` if a row exists.

Response shape and caching

- Responses compute `totalAmount = Σ price × quantity` from `order_items`, include `timezone: 'Asia/Kolkata'`, and look up the customer name with one query per order (N+1).
- `GET` cache key: `orders_<limit>_<cursor|null>_<user_id>`, TTL 300 s. `force_refresh=true` bypasses it.
- After every write, `setImmediate` deletes `orders_*` and `inventory_*` keys.
- `Order.getAll` contains a leftover `console.log('Orders from DB:', …)` debug line.

Status sync with dispatch: the README (`README.md` around line 353) says `orders` and `dispatch_tracking_details` are kept in sync by bidirectional PostgreSQL triggers (`sync_orders_status` and `sync_dispatch_status`, with a loop guard). The trigger SQL is not in this repo; it lives only in the database.

Tables: `orders` (order_id, user_id, status, status_reason, target_delivery_date, payment_status, created_at), `order_items` (order_item_id, order_id, product_id, quantity, price), `inventory_holds`, `inventory`, `customer_invoices`.

2.4 Inventory: finished goods (`routes/core/inventory.js`, `controllers/core/inventory.controller.js`, `models/core/inventory.js`)

Purpose: finished-product master (`inventory` table) with stock, returnable quantity, manual holds and returns.

- **Rate limit:** the router applies `express-rate-limit` at 100 requests per 15 minutes per IP to all inventory routes.
- **Endpoints:**
  - `POST /` and `PUT /:id` (`Inventory.can_write`).
  - `GET /available` and `GET /check-part-number` (auth only).
  - `GET /` (`Inventory.can_read`).
  - `DELETE /:id`.
  - `POST /:id/hold` and `POST /hold/:hold_id/release`.
  - `GET /:id/holds`.
  - `POST /:id/accept-return`.
- **Numbering scheme:**
  - `product_code` must be exactly 11 characters. Its first 4 characters are the **Part Number** (`part_number` column, unique constraint `inventory_part_number_unique`; `product_code` is unique via `unique_product_code`).
  - `check-part-number` requires a 4-digit `part_number` and accepts an optional `exclude_id`.
  - With `part_number_auto`, after insert the controller rewrites the code prefix to `String(product_id).padStart(4,'0')`. If the product ID is above 9999, or the rewrite hits a unique violation, the response includes a `partNumberWarning` instead of failing.
- **Validation and sanitising:** product_name is required; stock and returnable_qty may not be negative; price must be 0 or more. Name, description and code are passed through `sanitize-html` with no tags allowed.
- **Update:**
  - `Inventory.update(..., isAdmin=true)` is always called. It overwrites every column directly, including `stock_quantity` and `returnable_qty`.
  - If the client omits those fields they are bound as `undefined`, which becomes NULL (or a constraint error).
  - When `price` is supplied, `syncPriceWithPriceList` updates `price_list.price` for that product.
  - Emits `stockUpdate` when stock or returnable quantity changed.
- **Delete guard:** refused when any `ACTIVE` hold exists or when `stock_quantity` or `returnable_qty` is non-zero. Deletes `RELEASED` holds first to avoid an FK violation, then emits `stockUpdate` with zeros.
- **Holds:**
  - `createHold` requires `quantity > 0` and a `reason`, and records `reference_type/value` and `created_by`.
  - `releaseHold` only releases `ACTIVE` holds; otherwise it returns 404 "Hold not found or already released".
  - Availability = `stock_quantity - SUM(ACTIVE holds)` (`getInventoryWithAvailability`). `/available` adds `available: available_quantity > 0`.
- **Accept return:** `qty` must be greater than 0 and no more than `returnable_qty`. Runs `stock_quantity += qty, returnable_qty -= qty` as a single UPDATE and emits `stockUpdate`. `returnable_qty` is raised by returnable Delivery Challans (see 5.2).
- **Caching:**
  - `inventory_<limit>_<offset>` and `inventory_availability_<limit>_<offset>`, both TTL 3600 s.
  - Writes delete `inventory_*`; hold changes delete only `inventory_availability_*`.
  - `redis.del('price_list_*')` is a literal-key delete and is ineffective.
- **Socket:** `stockUpdate { product_id, stock_quantity, returnable_qty }`.

2.5 Stock: raw materials (`routes/core/stock.js`, `controllers/core/stock.controller.js`, `models/core/stock.js`)

Purpose: raw-material master (`raw_materials` table) with location, required quantity, photo, and an audited adjustment log (`stock_adjustments`).

- **Validation:**
  - `validateStockInput`: productName (string); productCode is a string of exactly 11 characters; price is a number of 0 or more; stockQuantity and qtyRequired are numbers of 0 or more if present; location and imageUrl are strings. Failures return 400 `INVALID_INPUT`.
  - `validateAdjustInput`: `quantity` must be a number and may be negative; `reason` must be a string.
- **Part numbers:** same 4-digit prefix scheme as inventory, against `raw_materials.part_number` (constraints `raw_materials_part_number_unique` and `raw_materials_product_code_key`). Error codes `PART_NUMBER_TAKEN` and `DUPLICATE_CODE`.
- **Update:** `COALESCE` keeps the existing stock, image and returnable values when they are omitted, but `qty_required` defaults to 0 and `location` to NULL when omitted.
- **Photo upload:** `POST /:productId/photo`.
  - multer memory storage, **8 MB** limit, field `photo`.
  - Allowed types: JPEG, PNG, WebP, GIF.
  - Filename `raw_<id>_<timestamp>.<ext>`, uploaded with `uploadBufferToDrive` (public).
  - `image_url` is set to the Drive `directUrl` and `stockUpdate` is emitted.
- **Adjust:** `POST /:productId/adjust` runs `stock_quantity += quantity` and inserts into `stock_adjustments (product_id, quantity, reason, created_by, created_at)`.
  - **Bug:** `BEGIN`/`COMMIT`/`ROLLBACK` are issued with `pool.query`, so each statement may run on a different pooled connection. The adjustment and its log entry are therefore **not actually atomic**.
- **Delete:** `DELETE /:productId` requires `Stock.can_delete`, has no guards, and returns 204.
- **Caching:** none. **Socket:** `stockUpdate` (with `location` and `image_url`, or `status: 'deleted'`).

2.6 Reports (`routes/core/reports.js`, `controllers/core/reports.controller.js`)

- `GET /api/reports/order-summary` (`Reports.can_read`): per order, `item_count` and `total_amount = SUM(quantity*price)`. Uses an inner join to `order_items`, so orders without items are excluded. Paginated.
- `GET /api/reports/inventory-status`: inventory rows with `stock_quantity < 10`, ascending.

2.7 Dashboard (`routes/core/dashboard.js`, `controllers/core/dashboard.controller.js`)

- Eight role-specific endpoints (`/sales-dashboard`, `/admin-dashboard`, …) each check `dashboard.can_read` and then require an exact `role_id` match (403 `PERM_DENIED` otherwise). Each only returns `{ dashboard: '<name>' }`, so they act as access probes.
- `GET /api/dashboard`: checks `permissions(module='dashboard').can_read` and maps roles 1 to 8 to a dashboard name. Other roles get 403 `INVALID_ROLE`.
- `GET /api/dashboard/admin-stats` (admin only) runs four counts in parallel:
  - open orders: `COALESCE(status,'Pending') NOT IN ('Delivered','Cancelled')`;
  - pending queries: `COALESCE(query_status,'Open') <> 'Closed'`;
  - low stock: `COALESCE(stock_quantity,0) < 10`;
  - dispatches today: `dispatch_tracking_details.dispatch_date::date` equals today in **IST** (`NOW() AT TIME ZONE 'Asia/Kolkata'`).

3. Sales

3.1 Enquiry, the CRM lead pipeline (`routes/sales/enquiry.js`, `controllers/sales/enquiry.controller.js`, `models/sales/enquiry.js`)

Purpose: capture and track sales leads, assign them between Sales, Design and field Representatives, keep a comment/activity thread, and track pipeline stage.

Endpoints (all require `Enquiries.*` permissions)

Endpoint	Purpose
`GET /templates`	Global comment templates (`comment_templates WHERE is_global`).
`GET /representatives`	Users whose role name is `representative`.
`POST /refresh`	Deletes `enquiry_*` and `enquiry_list_*` cache keys.
`POST /`, `GET /`, `GET /:id`, `PUT /:id`, `DELETE /:id`	CRUD.
`POST /:id/assign`	Reassign (`assigned_to` required).
`POST /:id/mark-done`	Design hands the enquiry back to Sales.
`POST /:id/comment`	Add a comment with mentions.
`PATCH /:id/stage`	Pipeline stage change.
`POST /:id/follow`, `DELETE /:id/follow`	Watchers (`enquiry_watchers`, `ON CONFLICT DO NOTHING`).
`POST /:enquiryId/activity/:activityId/read`	Read receipt (`enquiry_read_receipts`, upsert `read_at`).
`POST /:id/photo`, `DELETE /:id/photo`	Photo array management.

ID scheme

- If no `enquiry_id` is supplied, it comes from the DB function `SELECT get_next_enquiry_id()` (not defined in this repo).
- If that call fails, the fallback is `ENQ<year><random 4 digits>`.
- A duplicate ID raises `23505`, returned as 400 `DUPLICATE_ENQUIRY_ID`.

Status, lead and stage

- **`status`** is one of `Pending`, `In Progress`, `Closed`, `Cancelled`. Invalid input is normalised to `In Progress` when assigned, `Pending` otherwise.
- **`lead`** (exposed as `priority`) is one of `hotlead`, `followup`, `lead`, `not_interested`, `closed`, lower-cased; the default is `lead`.
- **`stage`:**
  - Initial value is `new`, or `in_discussion` if the enquiry is created already assigned.
  - Valid values for `changeStage` and the status each one sets:

Stage	Resulting status
closed_won	Closed
closed_lost	Closed
regret	Cancelled
in_discussion	In Progress
design_review	In Progress
admin_review	In Progress

  - A stage change also adds a comment (the note, or "Stage changed to …") and emits `enquiryUpdate` with `type: 'stage_change'`. No transition-order rules are enforced.

Ownership and visibility

- Roles whose name contains `design` or `representative` only see and mutate enquiries where `assigned_to` is their own ID.

- This applies to `getAll`, `getById` and every mutating method via `assertMutationAllowed`. A failure throws `Forbidden`, returned as 403 `FORBIDDEN`.
- A representative creating an enquiry without an assignee gets it auto-assigned to themselves.

Assignment and hand-off

- **`assign`** (transaction):
  - sets `assigned_to`, `assigned_by`, `assigned_at=NOW()`, `due_date`, `stage='in_discussion'`, `status='In Progress'`;
  - inserts an `enquiry_activities` row of type `assignment` with mentions `[assigned_to]`;
  - emits `enquiryUpdate` (`type: 'assigned'`) and, to room `user_<id>`, `notification`.
- **`mark-done`:**
  - Only roles whose name contains `design`, and only the current assignee.
  - In a transaction with `SELECT … FOR UPDATE` row locking, it reassigns the enquiry to the default Sales user `DEFAULT_SALES_USER_ID` (environment variable, fallback **7**) and logs an assignment activity ("Design completed work and returned enquiry to Sales").
  - `enquiryUpdate` is emitted twice: once by the model and once by the controller.

Comments and other behaviour

- **Comments:** `enquiry_activities(activity_type='comment', message, mentions int[], expected_by, is_internal)`. Emits `enquiryActivity { enquiryId, activity }` plus a `notification` (`type: 'mention'`) to each mentioned user's room.
- **Overdue check:** `checkOverdue(io)` posts an "overdue by N hours" system comment if the due date has passed, the stage is not closed, and no overdue message was posted in the last hour. **It is never called anywhere in the repo** (dead code).
- **Delete:** a transaction deletes `enquiry_activities`, then `enquiries`, and emits `enquiryUpdate { type: 'deleted' }`.
- **Search:** `LOWER(...) LIKE` across id, company, contact, mail, phone, items, status and application. Ordered by `created_at DESC`, with a `LIMIT/OFFSET` page plus an optional `created_at` cursor.
- **Photos:**
  - multer memory storage, **10 MB**, JPEG/PNG/WebP/GIF.
  - Filename `enquiry_<id>_<ts>.<ext>`, uploaded with public `uploadBufferToDrive`, then `photos = array_append(...)`.
  - Removal runs `array_remove` on the URL only; the Drive file is not deleted.

Caching: list key `enquiry_list_<user>_<limit>_<offset>_<search|all>`, single-record key `enquiry_<user>_<id>`, both TTL 300 s. Mutations delete `enquiry_*_<id>` and `enquiry_list_*`. Create only clears the list keys.

Tables: `enquiries` (enquiry_id, company_name, contact_person, mail_id, phone_no, items_required, source [default 'Website'], application, lead, tags[], stage, status, assigned_to/by/at, due_date, last_discussion, next_interaction, created_by, city, photos[], timestamps), `enquiry_activities`, `enquiry_watchers`, `enquiry_read_receipts`, `comment_templates`.

3.2 Enquiry Requirements (`routes/sales/enquiryRequirements.js`, `controllers/sales/enquiryRequirements.controller.js`, `models/sales/enquiryRequirements.js`)

Purpose: structured technical requirements attached to an enquiry, including motor specifications.

- **Tables:**
  - `enquiry_requirements` (id, enquiry_id, title, description, requirement_type, priority [default `medium`], status [default `open`], assignee_id, due_date, attachments, metadata, created_by, updated_by, timestamps).
  - `enquiry_requirement_motors` (requirement_id, application, power_rating, motor_voltage, motor_rpm, motor_max_rpm, motor_type, em_brake bool, em_brake_voltage, shaft_type, gvw, controller, controller_text).
- **Validation:** route middleware `validateMotorType`: `motors` must be an array and `motor_type` must be `BLDC` or `PMSM` (or null).

- **Endpoints:** `GET /` with filters `enquiryId`, `status`, `priority`, `assigneeId`, plus CRUD. `GET /:id?includeMotors=true` also returns the motors.
- **Create/update with motors:** both run in a transaction. Update *replaces* all motors when a `motors` array is supplied: it deletes them and runs a multi-row parameterised INSERT (13 parameters per motor).
- **Single-motor endpoints:** `/:requirementId/motors` (list, add) and `/motors/:motorId` (update, delete). `updateMotor` builds a dynamic SET from the provided fields and throws 'Nothing to update for motor' when the patch is empty.
- **Socket events:** `enquiryRequirement:created`, `:updated`, `:deleted`, `:motors:added`, `:motors:updated`, `:motors:deleted`. No caching.

3.3 Price List (`routes/sales/priceList.js`, `controllers/sales/priceList.controller.js`, `models/sales/priceList.js`)

Purpose: sales price per finished product, kept in sync with `inventory.price`.

- **Read:** `GET` (`PriceList.can_read`) joins `price_list` to `inventory` (inner join, so rows without a product are hidden), with an optional `search` (`ILIKE` on price_id or description). Ordered by `created_at DESC`.
- **Create/update** (transaction): write `price_list`, then `UPDATE inventory SET price` for the product. Update merges the supplied fields over the current row. The reverse sync from inventory edits lives in `Inventory.syncPriceWithPriceList`.
- **Delete:** removes only the price row; the inventory price is left as is.
- **Caching:** key `price_list_<limit>_<offset>_<search>`, TTL 300 s. Writes delete `price_list_*` and `inventory_*`.
- **Socket:** `priceListUpdate { type: CREATE|UPDATE|DELETE, item|itemId, timestamp }`.
- **Errors:** all write errors return 400 `INVALID_INPUT`, including not-found.

3.4 Quotation, Proforma Invoice, Purchase Order: stateless PDF generators

All three are `POST /api/<doc>/generate` behind `authenticateToken` only (**no `checkPermission`). **They take the full document data in the JSON body,** persist nothing**, and stream a PDFKit A4 PDF back as an attachment. Fonts are Roboto Regular/Bold (Quotation also Medium) from `assets/fonts/`, falling back to Helvetica. Images come from `assets/` (`compage_header_left/right.png`, `compage_signature.png`; Quotation also falls back to `/mnt/data/`). None of them upload to Drive.

	Quotation (`models/sales/quotation.js`)	Proforma (`models/sales/proforma.js`)	Purchase Order (`models/sales/purchaseOrder.js`)
Required field	`quotation_no`	`proforma_no`	`po_no`
File name	`QUOTATION_<safe>.pdf`	`PROFORMA_<safe>.pdf`	`PO_<safe>.pdf`
Margin	36	18 (outer bordered box)	36 (single outer box)
Item columns	S.No, Description, HSN Code, Qty, Unit Price (₹), Total Price (₹)	S.No, Description, Qty, Rate, Total	S.No, Description, Qty (KG), Unit price, Total Price, Discount, Net Price
Line total	`total_price` ?? `unit_price × qty` (qty defaults to 1)	`total_price` ?? `qty × rate`	`total_price` ?? `qty × unit_price`; `net = net_price ?? total × (1 − discount%/100)`
Tax	`gst_percent` (default **18**): Subtotal, `GST @ x%`, Grand Total = subtotal × (1 + x/100)	`gst_percent` (default 18): Taxable Amount, `Tax @ x%`, Total Payable	No tax computation; the default terms say "Taxes : GST Extra". Shows only the grand net total.
Amount in words	Yes, Indian system (thousand/lakh/crore): "Rupees … and … paise only"	Yes, title case: "Rs. … and … Paise Only"	No
Terms	`data.terms` or 4 standard terms (10-day validity, Ex-Works Faridabad, 100% advance, 4–6 weeks delivery); heading marks Default or Custom (`allowOverrideTerms`)	Not shown	`data.terms` / `data.notes` or defaults (Ex-Faridabad, GST extra, payment after delivery, immediate delivery)

	Quotation (models/sales/quotation.js)	Proforma (models/sales/proforma.js)	Purchase Order (models/sales/purchaseOrder.js)
Fixed blocks	Company header, recipient GST, Kind Attn / Subject, sign-off, signature, hard-coded Axis Bank account details, two-line footer on every page	Full header on first page and a reduced header on continuation pages; "To" plus a 4-row meta box (Proforma No, Date, Order No, Date 2); RTGS detail block; receiver/company signatures; green/magenta footer	Company registration numbers (ST/CST, TIN, ECC, GST, RC), To / Ref No. / dates, signature, "Authorized Signatory", footer
Pagination	Repeats the table header on new pages; reserves 90 pt for the footer; page breaks inside terms, sign-off and bank blocks	Adds pages when rows exceed the limit (reserves 340 pt) and keeps the trailing blocks (about 380 pt) together	Single page layout (no addPage calls)

- **Shared details:**
  - Currency is formatted with `toLocaleString('en-IN')` to 2 decimals.
  - Company GSTIN `06AAACC3923F1Z4` is embedded in all three.
  - Number-to-words floors rupees and rounds paise.
- **Proforma stream handling:** the controller attaches `pdfStream.on('error')` and destroys the stream when the response closes. Quotation and PO lack these handlers.

4. Invoicing

4.1 Customer Invoices (routes/invoicing/customerInvoices.js, controllers/invoicing/customerInvoices.controller.js, models/invoicing/customerInvoice.js)

Purpose: register of sales invoices: number, date, value and PDF link, optionally linked to a customer and an order. **No PDF is generated here**; `link_pdf` is a stored URL.

- **Endpoints:** CRUD with `CustomerInvoices.can_*`. Create requires `invoice_number`. The model also has a fallback of `INV-<Date.now()>` and defaults `issue_date` to now and `total_value` to 0.00.
- **List:** keyset by `issue_date` (cursor), ordered `issue_date DESC, invoice_id DESC`. Joins customers, users (`customer_name`) and orders (`order_status`). Returns a `sr_no` column.
- **Update:** overwrites customer_id, order_id, invoice_number, issue_date and link_pdf, setting missing values to NULL. **`total_value` cannot be changed after create.**
- **Errors:** FK violation `23503` returns 400 `FOREIGN_KEY_VIOLATION` ("Invalid customer_id or order_id"); not-found returns 404.
- **Caching:** `invoice_list_<limit>[_<cursor>]` and `invoice_<id>`, TTL 300 s. Writes delete `invoice_*`.
- **Socket:** `invoiceUpdate { invoice_id, invoice_number, total_value, issue_date }` (or `{status:'Deleted'}`). The same event is also emitted from the Orders model.

4.2 Purchase Invoices (routes/invoicing/purchaseInvoices.js, controllers/invoicing/purchaseInvoices.controller.js, models/invoicing/purchaseInvoice.js)

Purpose: supplier invoices for raw materials (one line per invoice, tied to a `raw_materials.product_id`).

- **Access:** routes use `authenticateToken` only, with **no permission check**.
- **Validation:** `validateInvoiceInput` requires supplierCode, supplierName, invoiceNumber and description as strings; `issueDate` must parse as a date; unitPrice and quantity must be numbers of 0 or more (falsy 0 is rejected because of `!unitPrice`); `productId` must be an integer of 1 or more; `linkPdf` is an optional string.
- **Table:** `purchase_invoices` (invoice_id, sr_no, supplier_code, supplier_name, invoice_number, issue_date, description, unit_price, quantity, link_pdf, product_id, created_at, updated_at).
- **Stock side effect:** after create, update and delete, the controller reads back `raw_materials.stock_quantity` and logs it. No application code changes stock here, so any automatic stock-in would have to come from a DB trigger that is not in this repo. **The code does not show this; verify in the DB.**

- **Caching:** key `purchase_invoices_<limit>_<offset>`, TTL 300 s. Writes call `redis.del('purchase_invoices_*')`, a literal key, so **cached pages are not invalidated** until the TTL expires.
- **Socket:** `invoiceUpdate` (the same event name as customer invoices, with a different payload shape).

5. Dispatch

5.1 Dispatch Tracking (`routes/dispatch/dispatchTracking.js`, `controllers/dispatch/dispatchTracking.controller.js`, `models/dispatch/dispatchTracking.js`)

Purpose: shipment and docket tracking per order (`dispatch_tracking_details`: tracking_id, sr_no, order_id, docket_number, dispatch_date, delivery_date, status, timestamps).

- **Endpoints:** `GET /` (paginated, ordered by `sr_no ASC`), `PUT /:trackingId`, `DELETE /:trackingId` (`DispatchTracking.can_*`). There is **no create endpoint**; rows are expected to come from the DB status-sync trigger described in the README (`sync_orders_status` / `sync_dispatch_status`).
- **Update:** uses `COALESCE`, so only the provided fields change.
  - **Bug:** the controller passes `order_id`, but the model destructures `tracking_id`. `order_id` is silently ignored and `tracking_id` is never changed.
  - A `status` change here is expected to propagate to `orders` through the trigger. No application-level validation of status values happens here.
- **Caching:** `dispatch_tracking_<limit>_<offset>`, TTL 300 s. Invalidation uses the literal `redis.del('dispatch_tracking_*')`, which is **ineffective**.
- **Socket:** `dispatchUpdate` (no payload).

5.2 Delivery Challan (`routes/dispatch/deliveryChallan.js`, `controllers/dispatch/deliveryChallan.controller.js`, `models/dispatch/deliveryChallan.js`)

Purpose: generate a Delivery Challan PDF for goods leaving the factory, and record **returnable** goods (for example items sent for repair or trial) so they can later be accepted back into stock.

- **Endpoint:** `POST /api/delivery-challan/generate` (auth only, no permission check).
- **Validation:** `challan_no` is required. `items` must be non-empty, and each item needs `productId`, `qty > 0`, and `source` of `'raw'` or `'inventory'`.
- **Flow:**
  1. Render the PDF into a buffer (`generateBuffer`).
  2. In a transaction, for each item with `returnable` true, run `UPDATE <inventory|raw_materials> SET returnable_qty = COALESCE(returnable_qty,0) + qty`. Inventory quantities are rounded to integers. A missing product rolls back with a 500.
  3. Commit, emit `stockUpdate { product_id, returnable_qty, status:'updated' }`, and send the PDF with `Content-Length`.
- **Stock effect:** the challan **does not deduct `stock_quantity`**; only `returnable_qty` increases. Returns are accepted via `POST /api/inventory/:id/accept-return`. There is no raw-material equivalent in these modules.
- **PDF layout:** PDFKit A4, margin 24, Roboto.
  - "DELIVERY CHALLAN" title; To / M/s. block with optional party GST.
  - Meta rows: Challan No / Date, Order No / Order Date, Vehicle No.
  - Table columns: No., Description (**only `productName`**, by design), Qty, Remarks.
  - Fixed text: a discrepancy clause (report within 2 days; guarantee void unless signed copies are returned), receiver and company signature lines, and a factory footer.
  - It has no prices or tax.

5.3 IA Orders: Intute AI dispatch register (`routes/dispatch/iaOrders.js`, `controllers/dispatch/iaOrders.controller.js`, `models/dispatch/iaOrders.js`)

Purpose: record Intute AI dispatches by device serial. Each order lists VCU units (serial, make, model) and optional paired HMI units (IMEI, make, model).

- **Endpoints** (`ia_orders.can_*`): `GET /` (cursor pagination, limit 1–100, default 20), `GET /check-serials?serials=a,b,c` (returns the VCU serials already used), `GET /:id`, `POST /`, `PUT /:id`, `DELETE /:id`.
- **Validation:**
  - Order needs customer_name, invoice_number, dispatch_date and at least one item.
  - Each item needs a VCU serial, make and model.
  - If any HMI field is supplied, all three HMI fields are required.
- **Uniqueness:** unique constraints on `ia_order_items.vcu_serial` and `hmi_imei`. A `23505` violation becomes a readable message ("VCU serial "X" already exists in another order"), returned as 400.
- **Transactions:**
  - Create inserts `ia_orders` and its items in one transaction.
  - Update patches only the supplied header fields. If `items` is supplied, it deletes all items and re-inserts them.
  - Delete removes `ia_orders` only, so item cleanup relies on an FK cascade in the DB.
- **`created_by`** is taken from `io.user.user_id`, the shared singleton field described in section 0 (race-prone).
- **List query:** aggregates items with `JSON_AGG`. The cursor is formatted as `"<order_id>:<created_at>"` and parsed with `split(':')`. Because timestamps contain colons, the parsed timestamp is truncated, which is a likely pagination bug.
- **Caching:** prefix `ia_orders_`, keys `ia_orders_list_<limit>_<cursor|initial>`, **TTL 120 s**, written asynchronously via `setImmediate`. Invalidation is the only place in these modules that uses `SCAN` (`scanIterator MATCH ia_orders_*`, batches of 500) rather than `KEYS`.
- **Socket:** `ia_orders:created`, `ia_orders:updated`, `ia_orders:deleted`.

6. Summary tables

Socket.IO events emitted by these modules

Event	Emitted by
`customerUpdate`	customers create
`stockUpdate`	orders (dispatch, cancel), inventory (create, update, delete, accept-return), stock (create, update, photo, delete, adjust), delivery challan
`invoiceUpdate`	orders (create, update, cancel), customer invoices, purchase invoices
`enquiryUpdate` (type created / updated / assigned / stage_change / deleted), `enquiryActivity`, `notification` (room `user_<id>`)	enquiry
`enquiryRequirement:created/updated/deleted`, `enquiryRequirement:motors:added/updated/deleted`	enquiry requirements
`priceListUpdate`	price list
`dispatchUpdate`	dispatch tracking
`ia_orders:created/updated/deleted`	IA orders

Redis keys

Key pattern	TTL	Invalidation works?
`customers_<limit>_<offset>`	3600 s	Yes (KEYS `customers_*_*`)
`orders_<limit>_<cursor>_<user>`	300 s	Yes (KEYS `orders_*`)
`inventory_<l>_<o>`, `inventory_availability_<l>_<o>`	3600 s	Yes

Key pattern	TTL	Invalidation works?
`enquiry_list_<user>_<l>_<o>_<search>`, `enquiry_<user>_<id>`	300 s	Yes
`price_list_<l>_<o>_<search>`	300 s	Yes from price-list writes; no from inventory writes (literal `del`)
`invoice_list_<l>[_<cursor>]`, `invoice_<id>`	300 s	Yes
`purchase_invoices_<l>_<o>`	300 s	**No** (literal `del`)
`dispatch_tracking_<l>_<o>`	300 s	**No** (literal `del`)
`ia_orders_list_<l>_<cursor>`	120 s	Yes (SCAN)

File-upload limits

Upload	Limit	Types	Destination
Raw-material photo (`/api/stock/:id/photo`)	8 MB	JPEG/PNG/WebP/GIF	Google Drive, public
Enquiry photo (`/api/enquiry/:id/photo`)	10 MB	JPEG/PNG/WebP/GIF	Google Drive, public

Known gaps and risks seen in code

1. `Stock.adjustStock` uses `pool.query('BEGIN')`, so the adjustment and its log entry are not atomic.
2. Customer creation is not atomic across `users` and `customers`, because `User.create` uses its own connection.
3. `PUT /api/inventory/:id` always runs the admin path and can null out stock if the fields are omitted.
4. Cancelling a `Partially Delivered` order neither restocks nor takes the post-shipment path.
5. `io.user` is a shared singleton; IA Orders `created_by` is race-prone.
6. Literal-wildcard `redis.del` calls (purchase invoices, dispatch tracking, inventory→price_list) do not invalidate anything.
7. Dispatch tracking update ignores `order_id` (parameter name mismatch).
8. IA Orders cursor parsing splits on `:` inside an ISO timestamp.
9. `Enquiry.checkOverdue` is never scheduled, and no socket room joins exist for `user_<id>` notifications.
10. PDF generator routes and purchase-invoice routes have no `checkPermission` RBAC; any authenticated user can call them.
11. The README says JWTs are accepted from a cookie, but `authenticateToken` only reads the `Authorization` header.
12. DB-side objects referenced but not defined in this repo: `get_next_enquiry_id()`, the order/dispatch status-sync triggers, and any stock trigger on `purchase_invoices`.

Manufacturing, Service, HR, Operations and Automation

This chapter covers engineering and shop-floor modules (BOM, parts, part drawings, motor recipes, processes and work orders, IPT kits), service and repair, HR (attendance, employee details, payslips, invoices), operations (activities, queries, problems), the chatbot, scheduled jobs, email and Google Drive integrations, tests and migrations.

All paths are relative to `CRM_BACKEND/`. Route mounts come from `server.js` (lines 168-213). Unless stated otherwise, every route runs `authenticateToken`. `checkPermission(module, action)` (`middleware/auth.js`) looks up the `permissions` table for `(role_id, module)` and checks `can_read`, `can_write` or `can_delete`. It returns 403 `PERM_DENIED`, or 500 `PERM_CHECK_FAILED` if the lookup fails.

Cross-cutting facts:

- **Socket.IO**: `server.js` creates one `Server` and stores it with `app.set('io')` and `app.set('socketio')`. Global middleware also sets `req.io = io` on every request. Some routers (iptKits, payslip, invoice, invoiceRecords, activities) re-assign `req.io = req.app.get('io')` and attach `req.io.user = req.user`. Their models use `io.user` to fill `created_by`. **No code anywhere calls `socket.join(...)`**, so every emit is a global broadcast, and room-targeted emits (`io.to('user:<id>')`) reach nobody.
- **Redis**: `config/redis.js` is a node-redis v4 client (`REDIS_URL`, default `redis://localhost:6379`). It adds a custom `delPattern(pattern)` method that runs `KEYS` then `DEL`. Most modules invalidate with `redis.keys(pattern)`, which blocks Redis (`KEYS` is O(N)). Only activities uses the non-blocking `scanIterator`.
- **Global error handler** (`middleware/error.js`): returns `err.status || 500` and `{ error: err.message, code: err.code || 'INTERNAL_ERROR' }`.
- **JSON body limit** is 40 MB (`express.json({ limit: '40mb' })`). The limit was raised for PDI photo uploads.

1. Manufacturing

1.1 Bill of Materials (BOM): `/api/bom`

Files: `routes/manufacturing/bom.js`, `controllers/manufacturing/bom.controller.js`, `models/manufacturing/bom.js`

Purpose: defines which raw materials, and how much of each, make up one unit of a finished product.

Endpoints. These use express-validator but have **no `checkPermission`**, so any authenticated user can call them.

Method	Path	Notes
GET	`/`	`limit` (1-100, default 10), `offset`, `force_refresh`; response is `{ data, total }`
GET	`/:id`	single BOM
POST	`/`	body `{ productId, materials:[{materialId, quantityPerUnit>0}] }` (min 1 material)
PUT	`/:id`	same body; replaces the material list
DELETE	`/:id`	

Tables:

- `bill_of_materials`: `bom_id`, `product_id`, `created_at`, `updated_at`.
- `bom_materials`: `bom_material_id`, `bom_id`, `material_id`, `quantity_per_unit`, `unit_price`, `total_value`.
- Joins `inventory` (finished product: `product_name`, `description`, `product_code`) and `raw_materials` (material name).

Behaviour:

- **Create** runs in a transaction on a dedicated `client` (BEGIN/COMMIT/ROLLBACK). It checks that the product exists in `inventory` and every material exists in `raw_materials`. It also accepts lookup by `productName` or `materialName`. It then inserts the header row and the material rows.

- The insert supplies only `(bom_id, material_id, quantity_per_unit)`. `unit_price` and `total_value` come back via `RETURNING`, so they are evidently filled by a DB trigger or default; the JS never computes them. The controller maps any error containing `"No price found"` or `"foreign key constraint"` to 400 `Invalid productId or materialId`. This suggests the price-lookup trigger raises "No price found".
- **Update** runs in one transaction. It updates the header, runs `DELETE FROM bom_materials WHERE bom_id`, then re-inserts all materials.
- **Delete** deletes the header row. Material-row cleanup relies on the FK.

Redis: list cache key `bom_list_<limit>_<offset>`, TTL 300 s. Every write invalidates `bom_list_*`. `force_refresh=true` also invalidates.

Socket.IO: `bom:created`, `bom:updated` (payload includes `triggeredBy`), `bom:deleted {bomId, triggeredBy}`.

Gaps:

- `BOM.getById` *throws* when the BOM is missing instead of returning null, so GET `/:id` returns 500 (not 404) for a missing BOM. PUT and DELETE behave the same way.

1.2 Parts master and part-code generation: `/api/parts`

Files: `routes/manufacturing/parts.js`, `controllers/manufacturing/parts.controller.js`, `models/manufacturing/parts.js`

Purpose: a catalogue of engineering parts with auto-generated part codes per part type.

Endpoints (permission module `Parts`):

Method	Path	Perm	Notes
GET	`/`	read	`{data,total}`, ordered by id ASC
GET	`/types`	read	`part_types` (`id`, `typeName`, `prefix`)
GET	`/next-code?partTypeId=`	**write**	preview only, consumes nothing
POST	`/`	write	body `partTypeId` (number), `name`, `description`, `drawingNo` required; `customerPartNo` and `supplierPartNo` optional strings
PUT	`/:id`	write	`part_code` and `part_type_id` are immutable
DELETE	`/:id`	delete	hard delete, returns 204

Numbering scheme:

- Format is `part_code = <part_types.prefix> + zero-padded serial (7 digits)`, for example `<prefix>0000001`.
- The per-type counter lives in `part_serials(part_type_id, last_serial)`.
- Create does `SELECT ... FROM part_types ... FOR UPDATE`, then `SELECT last_serial ... FOR UPDATE`. If no counter row exists it inserts one with value 0. It then increments, updates `part_serials`, and inserts into `parts`.

Tables:

- `parts`: `id`, `part_code`, `part_type_id`, `name`, `description`, `drawing_no`, `customer_part_no`, `supplier_part_no`, timestamps.
- `part_types`: `id`, `type_name`, `prefix`.
- `part_serials`.

Error codes: `INVALID_INPUT` (400), `INVALID_PART_TYPE` (400), `NOT_FOUND` (404), `SERVER_ERROR` (500).

Gap (significant):

- `Parts.create` runs `BEGIN`, `FOR UPDATE` and `COMMIT` through **`pool.query`, not a dedicated client**. Each statement can land on a different pooled connection, so the transaction and row locks do not actually protect the serial counter. Concurrent creates could produce duplicate codes, and ROLLBACK may hit the wrong connection.

1.3 Part drawings: `/api/part-drawings` and `/api/part-drawings-raw`

Files: `routes|controllers|models/manufacturing/partDrawings*.js`. The two modules are textual clones.

Purpose: store a drawing link (URL) per finished product (`part_drawings` → `inventory`) or per raw material (`part_drawings_raw` → `raw_materials`).

Endpoints: `GET /` (`limit`, `offset`, `search`, `force_refresh`), `POST / {drawing_link, product_id}`, `PUT /:srNo`, `DELETE /:srNo`. Only `authenticateToken` applies, with **no permission check**.

Columns: `sr_no` (PK), `drawing_id`, `item_name`, `drawing_link`, `product_id`, `updated_at`. Insert sets only `product_id` and `drawing_link`, so `drawing_id` and `item_name` must be filled by DB defaults or triggers.

Behaviour:

- Search is an ILIKE across sr_no, drawing_id, item_name, product_name and product_id. The response shape is `{ drawings, total }`.
- Create, update and delete each run in a client transaction. Update merges supplied fields with the current row.

Redis:

- Key `part_drawings_<limit>_<offset>_<search>` (or `part_drawings_raw_...`), TTL 300 s.
- Writes delete `part_drawings_*` plus `inventory_*`, or `part_drawings_raw_*` plus `raw_materials_*`.
- Both controllers also flush these patterns **once at module load** (an IIFE).
- The `part_drawings_*` pattern also matches the raw keys.

Socket.IO: `partDrawingsUpdate` / `partDrawingsRawUpdate` with `{type: 'CREATE'|'UPDATE'|'DELETE', item|itemId, timestamp}`.

Errors: every model error, including "not found", returns **400**.

1.4 Motor recipes: `/api/motor-recipes`

Files: `routes/manufacturing/motorRecipes.js`, controller, `models/manufacturing/motorRecipe.js`

Purpose: a customer-specific winding recipe for a motor product: number of turns and number of coils.

Endpoints (module `MotorRecipes`):

- `GET /` (`limit` default 50, max 200).
- `GET /dropdown-data`: customers joined to users plus the inventory product list.
- `GET /customer/:customerId`
- `GET /:customerId/:productId`
- `POST /` (upsert)
- `DELETE /:customerId/:productId`

Validation: `customer_id` and `product_id` are required. `num_turns` and `num_coils` must be positive integers (`VALIDATION_ERROR`).

Upsert: `INSERT ... ON CONFLICT ON CONSTRAINT uq_motor_recipes_customer_product DO UPDATE` (turns, coils, notes, `updated_at=now()`). This gives exactly one recipe per (customer, product).

Table: `motor_recipes` (`customer_id`, `product_id`, `num_turns`, `num_coils`, `notes`, `updated_at`, ...).

Errors: delete-not-found returns 404 `NOT_FOUND`. Other errors go to the global handler. No cache and no sockets.

1.5 IPT kits: `/api/ipt-kits`

Files: `routes/manufacturing/iptKits.js`, controller, `models/manufacturing/iptKits.js`. Test: `tests/iptKits.test.js`.

Purpose: records the assembly of an "IPT kit". It ties together the serial numbers of the seven components shipped as one kit, and supports reverse lookup ("which kit holds motor X?").

Endpoints (module `ipt_kits`): `GET /` (read), `GET /next-serial` (read), `GET /:id`, `POST /` (write), `PUT /next-serial` (write), `PUT /:id` (write), `DELETE /:id` (delete).

Component fields: `motor_serial`, `controller_serial`, `gearbox_serial`, `harness_serial`, `cluster_serial`, `vcu_serial`, `dcdc_serial`.

- All seven are required.
- Each is trimmed and upper-cased.
- A unique violation (`23505`) is mapped via the constraint name to a field-specific message: `"<component> serial "X" is already used in another kit"`, with `field`.

Kit serial numbering:

- `kit_serial = 'IPT' || lpad(nextval('ipt_kit_serial_seq'), 3, '0')`, for example `IPT001`. It is generated in SQL on insert.
- `GET /next-serial` previews the next value from the sequence's `last_value` and `is_called` without consuming it.
- `PUT /next-serial {next}` is the admin re-point. `next` must be a positive integer. It is rejected if an existing kit already uses that `IPTnnn`. Otherwise it runs `setval('ipt_kit_serial_seq', n, false)`. Numbers freed by deletes can be reused.

Table: `ipt_kits` (`kit_id`, `kit_serial`, the seven serials, `created_by`, `updated_by`, `created_at`, `updated_at`). `created_by` and `updated_by` come from `io.user.user_id`.

List pagination:

- Keyset cursor `"<kit_id>:<created_at text>"`, limit 1-100 (default 20).
- `created_at` is compared as `to_char(... 'YYYY-MM-DD HH24:MI:SS.US')` text, which avoids timezone shifting.
- Fetches `limit+1` rows to compute `hasMore`.
- Search is an ILIKE across all seven serials plus `kit_serial`.
- Response is `{ data, total, cursor }`.

Socket.IO: `ipt_kits:created`, `ipt_kits:updated`, `ipt_kits:deleted` (emits are wrapped in try/catch).

Errors: 400 `{error, field}` for validation or duplicates. 404 `'Kit not found'`.

1.6 Production process (work orders): `/api/process`

Files: `routes/manufacturing/process.js`, `controllers/manufacturing/process.controller.js` (242 lines), `models/manufacturing/process.js` (1028 lines). Permission module: `Processes`.

Purpose: tracks manufacturing of a sales order. The hierarchy is: order → optional instance groups → work orders → components with quantities → per-component process steps (status and quantities) plus raw-material allocations. Each work order also has milestone stages and testing results.

Data model:

Table	Key columns / constraints
`instance_groups`	`instance_group_id`, `order_id`, `instance_name`, `instance_type` in {`Motor`,`Non-Motor`}; constraint `unique_instance_name_per_order`
`components`	`component_id`, `component_name`, `product_type` in {`Motor`,`Non-Motor`}, `is_fixed`; `unique_component_name`
`component_processes`	`process_id`, `component_id`, `process_name`, `sequence` (int ≥0), `responsible_person`, `description`; `unique_process_per_component`
`work_orders`	`work_order_id`, `order_id`, `instance_group_id`, `target_date`, `status` (`Pending`/`In Progress`/`Completed`), `created_at`
`work_order_components`	`work_order_component_id`, `work_order_id`, `component_id`, `quantity`; unique (`work_order_id`,`component_id`)
`process_status`	`status_id`, `work_order_component_id`, `process_id`, `status`, `completed_quantity`, `in_use_quantity`, `allowed_quantity`, `completion_date`, `responsible_person`; unique (woc, process)
`work_order_materials`	`work_order_material_id`, `work_order_component_id`, `work_order_id`, `component_id`, `raw_material_id`, `quantity`; unique (woc, raw_material)
`process_material_usage`	(woc, process, raw_material) unique, `used_quantity`
`work_order_stages`	`work_order_stage_id`, `work_order_id`, `stage_name`, `stage_date`
`work_order_testing_results`	(`work_order_id`,`testing_type`) unique, `qty`, `test_date`, `controller_type`, `updated_at`

Endpoints:

Method	Path	Purpose
GET	`/orders`	all orders plus customer name (for the order picker)
GET	`/components`	component master with its process template (JSON-aggregated, ordered by sequence)
POST	`/components`	`{component_name, product_type}`
POST	`/components/:componentId/processes`	`{process_name, sequence, responsible_person?, description?}`
POST	`/work-orders/:workOrderId/components`	attach a component `{component_id, quantity>0}`
GET/PUT	`/components/:wocId/processes`, `/process-status`	view and update per-process progress
GET/POST/PUT/DELETE	`/components/:wocId/materials[/:materialId]`	material allocation
GET/PUT	`/work-orders/:id/stages`	milestone dates
GET/PUT	`/work-orders/:id/testing`	Primary/Final testing results
POST	`/:orderId/work-orders`	`{instance_group_id?, target_date?}`; status starts `Pending`
GET/POST	`/:orderId/instance-groups`	
GET	`/:orderId`	paginated full work-order tree

Business rules:

- **Attaching a component (transaction).** Duplicate attachment fails. If the component's `product_type === 'Motor'`, one `process_status` row (`Pending`, all quantities 0) is seeded for every `component_processes` step. **Non-Motor components get no process tracking.** Process status and material endpoints reject non-Motor components (`Processes are only applicable to Motor components` and `Materials can only be assigned to Motor components`).
- **Process status update (transaction, `updateProcessStatus`).** The documented rule (in-code comment):
  - `total_material = SUM(work_order_materials.quantity)` for the component.
  - **In-use is global.** The sum of `in_use_quantity` across all processes of the component must be ≤ total material. Otherwise the error is "Global in-use capacity exceeded…".
  - **Completed is local.** Each process's `completed_quantity` must be ≤ total material.
  - **Auto status per process:** `Completed` if completed ≥ total material, else `In Progress` if in_use > 0, else `Pending`.
  - **Work-order roll-up:** `Completed` if every process_status row in the work order is Completed. Otherwise `In Progress` if any row is In Progress or Completed. Otherwise `Pending`. The result is written to `work_orders.status`.
  - `allowed_quantity` is set to total material on all rows of that component (informational).
  - Controller validation: non-negative integers, and `completion_date` must be `YYYY-MM-DD`.
  - Edge case: when total material is 0, any update sets the process to `Completed` (0 ≥ 0).
- **Material allocation (POST, transaction).**
  - Upserts on (woc, raw_material) and recomputes the total.
  - If the current total in-use now exceeds the new total, it scales every process's in_use **proportionally**: `FLOOR(in_use * new/old)`. It re-derives status and logs a warning.
  - It updates `allowed_quantity`.
  - PUT and DELETE of a material run in the **controller** without a transaction. They recompute `allowed_quantity` but do not re-scale in-use.
- **Stages.** Valid names are `Assembly`, `Testing`, `PDI`, `Packing`, `Dispatch`, always ordered in that sequence. PUT only *updates* `stage_date` on an existing row. If the row is missing the error is "Stage X not found", so stage rows must be pre-created elsewhere (presumably a DB trigger on `work_orders` insert, since JS never inserts them).

- **Testing results.** `testing_type` in {`Primary`,`Final`}. Upsert on (work_order_id, testing_type). GET always returns both types, with nulls when unset.
- **Instance groups.** Order must exist. Type must be Motor or Non-Motor. A duplicate name per order gives an error.
- `upsertProcessMaterialUsage` and `getProcessMaterialUsage` exist in the model but are **not exposed by any route**. The `materialsUsed` array in the tree is read-only. That model function also deletes a stale cache key (`processes:order:<id>`) that does not match the real key format.

Redis:

- `GET /:orderId` caches under `processes_<orderId>_<instance_group|all>_<limit>_<cursor>_<userId>` for 300 s. It reads `req.user?.userId`, which is likely undefined because the auth user uses `user_id`, so the key ends in `_anon`.
- Every mutation clears `processes_<orderId>_*` in a `setImmediate` (fire-and-forget).

Pagination: cursor = previous page's last `created_at` (`wo.created_at < cursor`). `nextCursor` is set only when a full page is returned.

Socket.IO events:

- `instanceGroupUpdate`
- `componentUpdate`
- `componentProcessUpdate`
- `workOrderCreated`
- `workOrderComponentAdded`
- `processUpdate` (from the controller; the model's `processStatusUpdated` is not fired because the controller does not pass `io`)
- `workOrderMaterialUpdate`
- `materialDeleted`
- `workOrderStageUpdate`
- `workOrderTestingUpdate`

Several model emits fire *before* COMMIT.

Responses carry `timezone: 'Asia/Kolkata'`. Errors are mostly 400 with the raw message (`error.status || 400`).

2. Service: Service Repair: `/api/service-repair`

Files: `routes/service/serviceRepair.js`, `controllers/service/serviceRepair.controller.js`, `models/service/serviceRepair.js`. Permission module: `ServiceRepair`.

Purpose: logs customer units returned for repair or service: what came in, the reported fault, the actual issue found, repair status, who is responsible, and challan / photo evidence.

Table: `service_repair_records`.

- Columns: `record_id`, `customer_name` (required), `dispatch_material_no`, `part_details`, `receiving_quantity`, `dispatch_quantity`, `fault_query`, `actual_issue`, `repair_status` (default **`pending`**), `responsibility_person`, `testing_responsibility`, `remarks`, `sent_date`, `service_type` (default **`repair`**), `created_by`, `created_at`.
- Single-URL photo columns: `chalan_photo_url`, `delivery_challan_photo_url`.
- TEXT[] photo arrays: `fault_photos_urls`, `actual_issue_photos_urls`, `chalan_photos_urls`, `delivery_challan_photos_urls`, `repaired_photos_urls`.
- Status values are **not validated server-side**. The backend only defaults `pending` and `repair`.

Endpoints: `GET /` (limit ≤100, offset), `GET /:id`, `POST /`, `PUT /:id` (full replace; `customer_name` required), `DELETE /:id` (delete perm), `DELETE /:id/photo` body `{field, url}` (uses `array_remove`). Photo uploads are all write perm, `multipart` field `photo`:

- `POST /:id/photo?field=chalan_photo_url|delivery_challan_photo_url` sets a single column.

- `POST /:id/chalan-photo`, `/delivery-challan-photo`, `/fault-photo`, `/actual-issue-photo`, `/repaired-photo` append to the matching array (`array_append(COALESCE(col,'{}'), url)`).

File upload handling:

- multer `memoryStorage`, 10 MB limit.
- Allowed MIME types: JPEG, PNG, WebP, GIF, PDF.
- Upload goes through `services/googleDrive.uploadBufferToDrive` with filename `service_repair_<id>_<kind>_<timestamp>.<ext>`. The file is made **public** (anyone-with-link reader). The stored value is the `https://drive.google.com/uc?export=view&id=…` URL.

Errors: `VALIDATION_ERROR` 400, `NOT_FOUND` 404, `SERVER_ERROR` 500.

Gaps:

- The Drive upload happens *before* the record-exists check, so a 404 leaves an orphan public Drive file.
- `removePhoto` and `DELETE` do not trash the Drive file.
- A `proxyImage` controller (streams a Drive file through the service account) exists but **no route is wired to it**.
- `PUT` overwrites `repaired_photos_urls` from the body but not the other arrays.
- No Socket.IO events and no cache.

3. HR

Role constants (in the attendance, employeeDetails and activities code): `COM_ROLES = [9, 10]` (Compage; 9 = employee, 10 = HR) and `IA_ROLES = [11, 12]` (Intute AI). `getOrgRoles(role_id)` scopes HR views to the caller's own organisation. Any other role gets 403.

3.1 Attendance: `/api/attendance`

Files: `routes/hr/attendance.js`, controller, `models/hr/attendance.js`; table `attendance`.

Endpoints:

- `GET /` (perm `attendance_history:can_read`): the caller's own records, cursor = `created_at`.
- `POST /` (`mark_attendance:can_write`): check in, check out, or mark absent.
- `GET /summary` (`attendance_summary:can_read`): HR view filtered to the caller's org roles. Supports `date`, `search` (name/email/employee_id) and cursor.

Rules (controller):

- `present_absent` must be `present` or `absent` (`ATTENDANCE_INVALID_STATUS`).
- `date` must equal **today in IST** (`ATTENDANCE_INVALID_DATE`), so no back-dating is possible.
- Absent: times and mode must be null (`ATTENDANCE_INVALID_ABSENT_FIELDS`).
- Present: `mode` in {`office`,`remote`} (`ATTENDANCE_INVALID_MODE`). Other failure codes:
  - `ATTENDANCE_ALREADY_CHECKED_IN`
  - `ATTENDANCE_NO_CHECK_IN` (check-out without check-in)
  - `ATTENDANCE_ALREADY_CHECKED_OUT`
  - `ATTENDANCE_INVALID_TIME` (check-out must be after check-in)

Model `createOrUpdate` (transaction): one row per (user, date). Existing rows are updated with `COALESCE` so a check-out does not erase the check-in. Note that it also resets `created_at = CURRENT_TIMESTAMP` on update, which affects cursor ordering.

Socket.IO: `attendanceMarked` (name, email, employee_id, check_in, check_out, status, mode).

Redis:

- Personal key `attendance_<limit>_<cursor>_<user_id>`, TTL **10 s**.
- HR summary key `hr_attendance_summary_...`, TTL 30 s, cached **only for past dates**. Today's view always hits the DB.

- Marking attendance invalidates both.

Error codes: `SERVER_ERROR`, `HR_ATTENDANCE_SUMMARY_ERROR`.

3.2 Employee details: `/api/employee-details`

- `GET /`, `GET /:employee_id` (`employee_details:can_read`), `PATCH /:employee_id` (`can_write`).
- **Table:** `employee_details` (`employee_id` text key, `user_id`, `phone_number`, `date_of_joining`, `address`, timestamps), joined to `users` and `roles`.
- PATCH updates only `phone_number`, `date_of_joining` and `address` (COALESCE, so empty input means no change). It first verifies the employee belongs to the caller's org and otherwise returns 404 `EMPLOYEE_NOT_FOUND`.
- **Redis:** `employees_<limit>_<cursor>_<search>_<employee_id>_roles_<roles>`, TTL 60 s. Updates clear `employees_*`.

3.3 Payslip PDF: `POST /api/payslip/generate`

Files: `controllers/hr/payslip.controller.js`, `models/hr/payslip.js`.

- **No permission check**; any authenticated user can call it.
- **Stateless.** Nothing is stored. The PDF is streamed back as `attachment; filename="PAYSLIP_<NAME>_<PERIOD>.pdf"`.
- **Input:** `{ employee:{name, id}, period, payDate?, earnings:[{label, amount}], deductions:[{label, amount}] }`. Employee name and period are required.
- **Calculation:**
  - `gross = Σ finite earnings.amount`
  - `totalDeductions = Σ finite deductions.amount`
  - `netPay = max(0, gross − totalDeductions)`
  - Amount in words: `Rupees <number-to-words(round(netPay))> Only`.
  - No statutory logic (PF, ESI, TDS) is applied server-side. All components are supplied by the client.
- **PDF (PDFKit, A4, single page):**
  - Roboto fonts from `assets/fonts`, with a Helvetica fallback.
  - Header gradient, logo `assets/image.png`.
  - Company "Intute AI Technologies (OPC) Pvt. Ltd., A-5, Sector 68, Noida", with a "PAYSLIP" pill.
  - Employee card, then EARNINGS and DEDUCTIONS tables.
  - Gross/Total bar, NET PAY card (₹, en-IN format), amount-in-words box.
  - Stamp `assets/stamp.png` and a "Generated on" footer. Content is pulled up to guarantee a single page.
- **Socket:** `payslip:generated {filename, generated_at, generated_by}`.

3.4 IA invoice PDF: `POST /api/invoice/generate`

File: `models/hr/invoice.js`. **No permission check.**

- **Seller is fixed:** INTUTE-AI TECHNOLOGIES (OPC) PVT. LTD., GST `09AAHCI7346B1ZK`, plus registered and office addresses, tagline "AI | EV | CLEANTECH". The RTGS block hardcodes the HDFC Bank account, branch, type and IFSC.
- **Input:**
  - Required: `invoiceNumber`, `billing.name`, and `items:[{description, qty, rate}]` (at least one item with a description).
  - Optional: `date`, `orderNo`, `orderDate`, `billing.{address, phone, email, gst}`, `hsn`, `vendorCode`, `gstPercent` (default **18**).
- **Totals:**
  - `line = qty × rate`

  - `totalAmount = Σ line`
  - `gstAmount = totalAmount × gstPercent/100` (a single GST line, no CGST/SGST/IGST split)
  - `grandTotal = round2(total + gst)`
  - Words: `INR <words(round(grandTotal))>`.
- **PDF layout (bordered grid, A4):**
  1. Logo and company header, then the INVOICE title bar.
  2. "To," billing block next to the meta rows (Invoice No., Date, Order No., Order Date). Cell heights are measured so long values wrap.
  3. HSN and Vendor code row.
  4. Item table: Sr, Description, Qty, Unit Rate, Total. Page breaks reserve space for the trailing totals, RTGS and signature blocks.
  5. Totals, GST, TOTAL rows, then amount in words.
  6. RTGS details.
  7. Signature block: the stamp is sized from the PNG's real IHDR dimensions, followed by "For {company}".
  8. Footer addresses and tagline.
- **Socket:** `invoice:generated`.

3.5 Invoice records: `/api/invoice-records`

File: `models/hr/invoiceRecords.js`. **No permission check** (authenticateToken only).

Purpose: the persisted register of IA invoices. It holds both system-generated invoices and uploaded legacy PDFs.

Endpoints: `GET /` (cursor, search on number or billing name), `POST /` (create generated), `POST /upload` (multipart `file`, PDF only, 10 MB), `GET /:id`, `GET /:id/pdf`, `DELETE /:id`.

Table `invoices`:

- `invoice_id`, `invoice_number` (unique), `invoice_date`, `order_no`, `order_date`.
- `billing_name`, `billing_address`, `billing_phone`, `billing_email`, `billing_gst`.
- `hsn`, `vendor_code`, `gst_percent`, `items` (JSONB).
- `total_amount`, `gst_amount`, `grand_total`.
- `created_by`, `source` (`generated` | `uploaded`), `drive_file_id`, timestamps.
- Dates are always read back via `to_char` to avoid IST/UTC shifting (documented in-code).

Generated flow:

- Validates the input (items need a description *and* qty > 0) and uses the same totals formula.
- Renders the PDF to a buffer and makes a **best-effort private** Drive backup (`uploadBufferToDrivePrivate`, no public permission). A Drive failure does not block saving.
- If the insert then fails (for example a duplicate invoice number, which returns "Invoice number "X" already exists"), the Drive file is trashed to avoid orphans.

Uploaded flow:

- Requires `invoiceNumber`, `date`, `customerName` and `grandTotal > 0`.
- Validates the `%PDF-` magic bytes.
- Drive upload is mandatory because Drive holds the only copy.
- Stores `gst_percent 0`, `items []`, and `total = grand_total`.

`GET /:id/pdf`: **generated invoices are re-rendered on the fly** from stored data. Uploaded ones are streamed from private Drive (`streamFileToDrive`).

Delete: removes the row and trashes the Drive file (best effort).

Socket: `invoice_records:created`, `invoice_records:deleted`.

Pagination: the cursor is `"<id>:<created_at text>"`, split on the **first** colon, with a text comparison.

4. Operations

4.1 Activities (task manager): `/api/activities`

Files: `routes/operations/activities.js`, controller, `models/operations/activities.js`.

Access: a router-level allow-list `ALLOWED_ROLES = [9, 10, 11, 12]`. Any other role gets 403 "Not authorized to access activities". No `checkPermission` is used.

Org model: each activity has `org` in {`com`, `ia`}, derived from the creator's role (9/10 → `com`, 11/12 → `ia`). List queries are scoped by org.

Role rules (Compage employee, role 9):

- Cannot create (403) or delete (403).
- The list shows only activities assigned to them.
- Update may change **only `status`**, which must be one of `todo`, `in_progress`, `done`.
- Roles 10, 11 and 12 see and manage all activities in their org.

Tables:

- `activities`: `id`, `summary`, `status` (default `todo`), `due_date`, `priority` (default `medium`; templates know `low`, `medium`, `high`, `urgent`), `comments`, `org`, `created_by`, timestamps.
- `activity_assignees`: `activity_id`, `user_id`, `assigned_at`.

Behaviour:

- Create and update are transactional.
- Update replaces the assignee set when `assignee_ids` is supplied.
- FK violation `23503` is mapped to "Assignee user not found".

Email: on create, **for the IA org only**, each assignee receives a "New Task Assigned to You" email (`generateNewTaskAssignmentHtml`, link `https://intute.biz/activities/<id>`), sent asynchronously in `setImmediate`.

Redis:

- List key `activities_list_<limit>_<cursor|initial>_user_<id>` (role 9) or `_role_<roleId>`.
- Item key `activities_<id>`. TTL 300 s.
- Every write invalidates `activities_*` via `SCAN`.

Socket.IO: `activities:created`, `activities:updated`, `activities:deleted`.

Gaps:

- `GET /:id`, `PUT /:id` and `DELETE /:id` do **not** check org or assignment. A Compage employee can read, or status-update, any activity by id, including other-org activities.
- The list cursor is built from `created_at` (an ISO string containing colons) and parsed with `cursor.split(':')`. The timestamp is truncated, so pages after the first are unreliable.
- The create path builds the assignee INSERT by string interpolation, with ids filtered through `Number.isFinite`.

4.2 Customer queries (support tickets): `/api/queries`

Files: `routes/operations/queries.js`, `controllers/operations/queries.controller.js`, **`services/queryService.js`** (the logic). `models/operations/query.js` is an older, unused model.

Endpoints (authenticateToken only):

- `GET /` (limit/offset)
- `POST / {description}`
- `PUT /:id/respond {response}`
- `PUT /:id/in-progress`

- `PUT /:id/close`

State machine: `Open` → `In Progress` (on any response, or explicit set-in-progress) → `Closed`.

- Responding to, or setting in-progress, a Closed query gives 400.
- Setting in-progress when it is already In Progress gives 400.
- Closing an already-closed query gives 400.
- Role 2 (customer, by inference) may close only its own query (403 otherwise).

Visibility: roles 1 and 5 (admin, and role 5) see all queries. Everyone else sees only their own.

Tables:

- `queries`: `query_id`, `user_id`, `query_text`, `query_status`, `date_of_query_raised`, `last_updated`.
- `query_responses`: `query_id`, `responded_by`, `response`, `response_date`.

Each mutation uses a transaction with `SELECT ... FOR UPDATE`. `closeQuery` runs its UPDATE on `pool` rather than the locked client, which is a minor inconsistency.

Errors: `QueryError(message, statusCode)`, handled by a router-local error middleware.

Redis: key `queries:<user>:<limit>:<offset>`, TTL 3600 s. However, `getQueries` **deletes the key before reading it**, so caching is effectively disabled. Writes run `delPattern('queries:*')`.

Socket: `newQuery` emitted to room `user:<id>` and `queryUpdate` to `user:<ownerId>`. Since no socket ever joins rooms, these are not delivered.

Other gap: `getQueries` logs the full result set at info level.

4.3 Problems / solutions knowledge base: `/api/problems`

Files: `routes/operations/problems.js`, controller, `models/operations/Problem.js`. Permission module `Problems`.

Purpose: records product problems and the solutions attempted (with a success flag). It works as a troubleshooting log per inventory product.

Endpoints:

- `GET /` (limit ≤100, offset): each problem with a JSON-aggregated `solutions` array ordered by `attempted_at`.
- `POST / {product_id, problem_description}`
- `POST /:id/solutions {solution_description, is_successful}`

Tables:

- `problems`: `id`, `product_id`, `problem_description`, `created_at`.
- `problem_solutions`: `solution_id`, `problem_id`, `solution_description`, `is_successful` (default false), `attempted_at`.

Redis: `problems_<limit>_<offset>_<user_id>`, TTL 300 s. Writes invalidate only the *acting user's* keys, so other users' caches stay stale for up to 5 minutes.

Socket: `problem:created`, `solution:created`.

5. Chatbot (`chatbot/`)

How it works: purely **rule-based, exact-phrase matching. No LLM or NLP** (no OpenAI or Anthropic dependency anywhere in `chatbot/`).

- **Pipeline:** route → `*ChatbotService.handle(message, user)` → `detectIntent` → model query → formatter.
- **`detectIntent`** (`chatbot/utils/*Intents.js`) trims and lower-cases the message, collapses whitespace, and looks it up in a fixed `INTENTS` dictionary. An unknown phrase returns `{success:false, response:"I didn't understand that. Try commands like …"}`.
- **Formatters** (`chatbot/utils/*Formatter.js`) build plain-text, newline-separated answers (e.g. `Order 12 - Customer: X - Pending`).
- **Response:** `{ success, response, data }`.

- All routes are `POST` with body `{message}` and use authenticateToken only. There is **no role or permission filtering**, so any logged-in user can read company-wide order, invoice and stock figures.
- Results are typically capped at 10 rows.

Endpoint	Supported phrases (intents)
`POST /api/orders/chatbot`	show all / recent / pending / processing / testing / ready for shipment / shipped / partially delivered / delivered / cancelled orders; count orders by status
`POST /api/customer-invoices/chatbot`	show all / recent invoices; count total invoices; show high value invoices (≥ ₹50,000); show invoices this month; total invoice value; show monthly invoice trend (last 6 months)
`POST /api/customers/chatbot`	show all / recent customers; count total customers; top customers by orders; customers with no orders; customers by city; customer order breakdown
`POST /api/inventory/chatbot`	show all products; low stock (0 < qty ≤ 10); out of stock; count; top by stock; products with holds (reserved qty); inventory value
`POST /api/stock/chatbot` (raw materials)	all; low (stock < qty_required); out of stock; count; by location; most needed (qty_required − stock); raw material value
`POST /api/bom/chatbot`	show all BOMs; count total BOMs; most complex (by material count); recently updated; high value (by total BOM value)

6. Background jobs (`jobs/`), email, Drive, logger

Jobs are loaded only when `server.js` is run directly (`require.main === module`), so tests do not start cron timers. All use `node-cron` with `timezone: 'Asia/Kolkata'`.

File	Schedule (IST)	What it does
`jobs/daily-task-summaries.js`	09:00 (morning), 18:30 (evening)	For a **hard-coded user list `user_id IN (45, 69, 70, 71, 72, 73)`**: emails each user their pending activities (and, in the evening, those completed today, i.e. `status='done'` with `updated_at` today). Sends the manager (`MANAGER_DAILY_REPORT_EMAIL`, greeted as "Akshay") a combined report.
`jobs/daily-due-reminders.js`	11:00 daily	Finds activities with `due_date = tomorrow` and status `todo`/`in_progress`, groups them by assignee email, and sends one reminder per person (`generateDueTomorrowReminderHtml`). Runs every day, including Sunday.
`jobs/attendanceSummary.js`	10:00 check-in summary, 19:30 check-out summary	Emails the manager today's `present` check-ins, then check-outs with hours worked (`Xh Ym`) and "still active" employees. **Excludes** user_ids 45, 69-73.

- The "weekend" skip (`isWeekendIST`) checks only `getDay() === 0`, so the summary jobs run **Monday-Saturday** and skip Sunday only. A log message says "weekdays".
- Manager name and user ids are hard-coded.

`services/email.js`:

- Nodemailer with `service: 'gmail'`, `EMAIL_USER` / `EMAIL_PASS` (app password). The transporter is verified at load.
- `sendEmail({to, subject, text, html})` validates its inputs. It sends from `"Task Manager" <EMAIL_USER>`. It **returns true or false and never throws**; failures are only logged.

`utils/emailTemplates.js` (858 lines): inline-styled HTML email builders.

- The five builders are:
  - `generateDueTomorrowReminderHtml`
  - `generateDailyTaskSummaryHtml`
  - `generateNewTaskAssignmentHtml`
  - `generateAttendanceCheckInSummaryHtml`
  - `generateAttendanceCheckOutSummaryHtml`

- A shared `getPriorityBadge` gives colour-coded badges: Low, Medium, High, Urgent.
- Links point to `https://intute.biz/...`.
- Values are interpolated **without HTML escaping**.

`services/googleDrive.js`:

- `googleapis` with service-account auth. The key comes from `GOOGLE_SERVICE_ACCOUNT_PATH` (default `secrets/drive-sa.json`). The scope is full `drive`. The target folder is `GOOGLE_DRIVE_FOLDER_ID`, on a Shared Drive (`supportsAllDrives`).
- Functions:
  - `uploadBufferToDrive`: uploads, then grants `anyone: reader`. Returns `directUrl` (`uc?export=view`), `webViewLink` and `webContentLink`.
  - `uploadBufferToDrivePrivate`: no public permission. Used for invoices.
  - `streamFileToDrive(fileId, res)`: despite the name, it **streams a file from Drive to the HTTP response**, setting Content-Type and `Cache-Control: public, max-age=86400`.
  - `deleteDriveFile`: **trashes** rather than deletes. The service account has `canTrash` but not `canDelete` on the Shared Drive. Trash is auto-purged after 30 days.

`utils/logger.js`: Winston at level `info`, JSON format with timestamps. Transports: `logs/error.log` (errors only), `logs/combined.log`, and the console. There is no rotation.

7. Tests (`tests/`)

Runner: Jest 29 (`npm test` → `jest`), with Babel transform-ignore exceptions for htmlparser2 and related packages. Per project memory, `*.test.js` is gitignored but the files are tracked, so new tests need `git add -f`.

There are 28 files and about 340 test cases, concentrated on the **PDI (pre-dispatch inspection) module and auth**:

Area	Files (approx. # tests)	Style
Auth	`auth.test.js` (5: signup, login, password change/validation), `auth_permission.test.js` (8: `hasPermission`, `checkPermission`), `auth_token_lifetime.test.js` (2: PDI mobile app `X-App-Version` gets a 12 h token, others 1 h), `pdi_auth_unavailable.test.js` (3: DB failure must not look like an invalid token)	auth.test is integration (supertest); the others mock the DB
Sales enquiry	`enquiry_controller.test.js` (9), `enquiry_model.test.js` (18): representative role scoping, photo upload ownership	mocked
Manufacturing	`iptKits.test.js` (9): create with auto serial, duplicate-serial field error, reverse search, cursor pagination with no gaps or overlaps, update, delete freeing serials, next-serial preview and set	**integration against a real DB** (inserts temporary admin role-1 and production role-5 users, cleans up afterwards)
PDI reports and lots	`pdiReports`, `pdi_report_batches` (42), `pdi_lot_rules`, `pdi_finalize_pdf`, `pdi_finalized_edit`, `pdi_delete_report`, `pdi_photos_summary`, `pdi_inspection_date`	DB and Drive mocked
PDI templates and PDF	`pdi_template_renderer` (30), `pdi_primitives`, `pdi_generator`, `pdi_generator_combined`, `pdi_general_v1_0_8`, `pdi_autonxt_tolerance`, `pdi_autonxt_controller` (20), `pdi_tolerance`, `pdi_photo_optimizer`, `pdi_pdf_cache_prefix`, `pdi_admin` (16), `pdi_authored_templates`, `pdi_authored_template_adapter`	unit

Not covered by any test: BOM, parts / part-code generation, part drawings, motor recipes, process / work orders, service repair, attendance, employee details, payslip, invoice / invoice records, activities, queries, problems, chatbot, cron jobs, email.

8. Migration scripts (`scripts/migrations/`)

Standalone Node scripts (`node scripts/migrations/<file>.js`). Each loads `.env`, runs idempotent DDL (`IF NOT EXISTS`) through `config/db`, then calls `pool.end()`. There is no migration framework or tracking table, and no other migrations directory exists (`migrations/` at the root is absent). The base schema for most modules is not in the repo.

Script	Effect
`2026-09-04-add-pdi-report-content.js`	`pre_dispatch_inspection_reports`: adds `data JSONB DEFAULT '{}'`, `photos JSONB DEFAULT '[]'`, `template_id TEXT DEFAULT 'general'`, `drive_file_id TEXT`
`2026-09-08-add-pdi-templates.js`	creates `pdi_templates(id, version, name, status in draft/active/archived, definition JSONB, created_by, created_at, PK(id,version))` with an index on (id, version DESC); adds `template_version INT` to reports
`2026-09-23-add-representative-role.js`	`enquiries`: adds `city TEXT` and `photos TEXT[] DEFAULT '{}'`. Ensures a `representative` role exists, first re-syncing `roles_role_id_seq` because explicit ids had collided in production. Grants `permissions(Enquiries: read+write, no delete)`
`2026-09-23-seed-representative-user.js`	seeds a test user `representative@compageauto.com` with password **`password123`** via `User.create`; idempotent on duplicate email
`2026-09-29-add-pdi-report-batches.js`	creates `pdi_report_batches(batch_id, template_id, pdi_no, lot_quantity, status DEFAULT 'In Progress', drive_file_id, created_by, created_at)`; adds `batch_id` FK and `lot_index` to reports
`2026-09-30-add-pdi-batch-shared-fields.js`	batches gain `customer_name`, `product_id`, `product_specifications`, `drawing_no`, `controller_type`. These lot-level fields are entered once and authoritative at render
`2026-09-30-add-pdi-report-revisions.js`	reports gain `revision_no INT DEFAULT 1`; creates `pdi_report_revisions(revision_id, report_id FK ON DELETE CASCADE, revision_no, data JSONB, edited_by, edited_at)`, which snapshots `data` before each edit of a Completed report (photos not snapshotted)
`2026-09-30-fix-autonxt-spec-display.js`	one-off **data** fix, supports `--dry-run`. Recomputes the 19 `spec_<id>_display` strings on every AutoNXT report from stored nominal and tolerance values, mirroring the web form's formula. Overwrites `data` with no version check, so it should run in a maintenance window
`2026-10-05-pdi-batch-indexes.js`	index `idx_pdi_reports_batch_id`; partial unique index `uq_pdi_reports_batch_lot_index (batch_id, lot_index) WHERE batch_id IS NOT NULL`

Related startup behaviour (`server.js` `initializeServer`): any `pdi_report_batches` row left in status `Finalizing` (for example by a crash mid-render) is reset to `In Progress` at boot.

9. Consolidated known gaps / risks (from code)

1. **Parts code generation is not truly transactional**: it uses `pool.query('BEGIN')` instead of a dedicated client, so duplicate part codes are possible under concurrency.
2. **Missing authorisation:**
   - BOM, part drawings (both), payslip, invoice and invoice-records have no `checkPermission`.
   - Chatbot endpoints expose company-wide data to any authenticated user.
   - The activities `getById`, `update` and `delete` paths ignore org and assignment scope.
3. **Socket rooms are never joined**, so query notifications sent to `user:<id>` rooms are never delivered.
4. **Cache defects:**
   - The queries cache is deleted before every read.
   - The process cache key uses `req.user.userId` (undefined).
   - Problems invalidation is per-user only.

- Wide use of blocking `KEYS`.

5. **Status-code inaccuracies:**
   - BOM not-found returns 500.
   - Part-drawing not-found returns 400.
   - Activities pagination cursor splits an ISO timestamp on `:`.
6. **Drive hygiene in service repair:**
   - Uploads become public before the record is validated (orphan on 404).
   - Removing a photo does not trash the Drive file.
   - The `proxyImage` handler is unrouted.
7. **Dead or unrouted code:** `Process.upsertProcessMaterialUsage` / `getProcessMaterialUsage`, `models/operations/query.js`.
8. **Hard-coded operational data:** cron user ids (45, 69-73), manager name "Akshay", the invoice seller GST and bank details, and the payslip company address.
9. **Emails are not HTML-escaped.** The "weekend" skip excludes Sunday only.
10. **No automated tests** for manufacturing (except IPT kits), service, HR, operations, chatbot or jobs.
11. **Seed script contains a default password** (`password123`) for a test representative account.

Pre-Dispatch Inspection (PDI) Subsystem

PDI is the most complex subsystem in the ERP. It covers inspection report capture on the web and the mobile app, a template engine with built-in and admin-authored templates, lots (batches), revision history, tolerance evaluation, and server-side PDF rendering.

Paths are relative to `CRM_BACKEND/` unless they start with `CRM/` (the web frontend). This describes the code as it is at commit `e353328d` (2026-10). Where a design spec says something different from the code, these notes follow the code and say so.

1. Overview

The PDI subsystem records a product's pre-dispatch inspection (electrical and mechanical measurements, checklists, photos, sign-offs) and produces a customer-facing A4 PDF. It serves three clients through one REST API:

- **Web CRM** (React): hand-coded fill-out forms for each built-in template, a generic form for admin-authored templates, lot (batch) pages, a reports table, and an admin template editor.
- **Mobile PDI app** (React Native / Expo, a separate repo `pdi-erp-app`), used by technicians on the shop floor.
- **Background jobs** inside the API itself: Google Drive backup, a PDF disk cache, and Socket.IO change events.

Main building blocks:

Layer	Files
Routes	`routes/operations/pdi.js`, `pdiReports.js`, `pdiReportBatches.js`, `pdiAdmin.js` (mounted in `server.js` lines 196-202)
Access control	`middleware/pdiAccess.js`, `middleware/auth.js`
Controllers	`controllers/operations/pdi.controller.js`, `pdiReports.controller.js`, `pdiReportBatches.controller.js`, `pdi.admin.controller.js`
Models	`models/operations/pdiReports.js`, `pdiReportBatches.js`, `pdi_generator.js`
Template engine	`models/operations/pdi/` (`renderer.js`, `primitives.js`, `tolerance.js`, `photoOptimizer.js`, `pdfCache.js`, `batchOverrides.js`, `authoredTemplate.js`, `authoredTemplates.js`, `templates/{index,general,autonxt,autonxt_controller}.js`)
Migrations	`scripts/migrations/2026-09-*.js`, `2026-10-05-pdi-batch-indexes.js`
Tests	`tests/pdi_*.test.js` (22 files, Jest)

Key libraries (`package.json`): `pdfkit ^0.17.2`, `sharp ^0.35.4`, `pg ^8.13`, `redis ^4.7`, `socket.io ^4.8`, `express ^4.19`, `jest ^29.7`.

Route mount order matters (`server.js` 196-202): `/api/pdi/reports`, `/api/pdi/report-batches` and `/api/pdi/admin/templates` are mounted **before** the catch-all `/api/pdi` router, so Express does not hand them to the template router.

The JSON body limit is raised to **40 MB** (`server.js` line 132) because saves carry base64 photos.

2. Template engine architecture

2.1 The two kinds of template

1. **Built-in (code-registered) templates.** These are JS objects in `models/operations/pdi/templates/`, registered in `templates/index.js` as a map from id to template:
   - `general` ("General", 3 pages: Electrical, Mechanical, Photos)
   - `autonxt` ("AutoNXT Motor PDI", 3 pages)
   - `autonxt_controller` ("AutoNXT Controller PDI", 2 pages, `pageNumberPad: 2`, `endOfReport: true`)

They can contain JS closures: computed cell values, `isOutOfTolerance` predicates, `footerHeight(data)` functions, spec rows, and function defaults. They are **not versioned in the DB**, so `template_version` is `NULL` on their reports. Each has its own hand-coded web form.

2. **Admin-authored templates.** These are stored in the `pdi_templates` table as a declarative JSON `definition`, so they hold no closures. `authoredTemplate.js#hydrateTemplate()` converts a stored definition into the in-memory shape the renderer expects. All of them are filled in through one generic web form (`CRM/src/components/admin/GenericPdiGeneratorForm.jsx`).

Resolution order (`pdi_generator.js`):

- `templates[templateId]` is checked first. If it exists, the built-in template wins.
- Otherwise the code calls `AuthoredTemplates.getByVersion(templateId, templateVersion)` and then `hydrateTemplate(row.definition)`. If that returns nothing it throws `Unknown PDI template: <id>`.

Creating a template whose id collides with a built-in one is rejected with a 409 by `pdi.admin.controller.js#createTemplate`. A stale comment in `pdi_generator.js` still says this check has not been built. It has.

2.2 Section vocabulary (renderer)

`renderer.js` walks `template.pages[].sections[]` and dispatches each section by `type` through the `DRAWERS` map:

Type	Purpose / notable options
`header`	Logo box (`logoAsset`, e.g. `compage_header_left.png`), "Pre Dispatch Inspection (PDI)" title, Format No / Rev No / Eff. Dt, optional `extraFormatLines`, company-name strip, and two-column `infoFields` rows `[leftLabel, fn(data), rightLabel, fn(data)]`. `rLabelW` sets the right-hand label width.
`table`	`mode: 'fixed'` (rows from `fixedRows(data)`, values usually from `sectionData = data[dataKey]`) or `'repeatable'` (one row per item of the `data[dataKey]` array, auto-paginated). Optional `title`, grouped two-level headers (`group`), `specRow` (a yellow "Specification" row; `labelSpan` merges leading columns), `wrap: true` columns that size the row to its text, `filterRow`, and `footerHeight`.
`photo`	`mode: 'freeform'` (list of `{label, images[]}`) or `'fixed-slots'` (`slots[{key,label}]`, each slot value is a URI string, an array of URIs, or null). Photos are drawn two per row with a "PHOTOS:" header on every page. Optional `imgHeight` (default 150 pt) and `footerHeight`.
`image`	A single fixed image (`data[dataKey]`) with a `placeholder` text and annotations as fallback. General uses it for the drawing on page 2.
`signature`	N equal-width role boxes. A role can supply `value(data)`. `reserveBelow` leaves space for END OF REPORT. Moves to a new page if it would cross the bottom margin.
`text`	A fixed 40 pt "Remarks" box. `default` is a string or `fn(data)` and is used when the value is blank. Text wraps within `maxHeight`.

An unknown section type throws `Unknown PDI template section type: <type>`. An invalid table or photo mode throws a descriptive error.

Out-of-tolerance cells are drawn with a light-red fill `#fee2e2`, **bold** dark-red text `#b91c1c` (bold survives black-and-white printing), driven by each column's `isOutOfTolerance(row, data)`.

Page numbering: `renderTemplate(doc, template, data, {numberPages})` draws "Pg X of N" using `drawPageNum`. `numberPageRange(doc, start, count)` numbers a sub-range starting again at 1, which is how the combined lot PDF restarts numbering for each unit. Each template's own `pageNumberPad` and `endOfReport` options are remembered per document in a `WeakMap`.

2.3 Primitives (`primitives.js`)

- **Page geometry:** A4, `PAGE_W 595.28`, `PAGE_H 841.89`, margin `M = 36`, `CW = PAGE_W - 72`, `BOT_M = 36`. The PDFKit document is created with `margins.bottom: 0`, `autoFirstPage: false` and `bufferPages: true`. Every draw call uses an explicit x/y; a non-zero bottom margin would make PDFKit insert blank pages.
- **Fonts:** Roboto Regular and Bold (`assets/fonts/`) are registered per document by `registerFonts(doc)`, with Helvetica / Helvetica-Bold as fallback. The font names are held in **module-level mutable state** (`F`, `FB`), read through `getFonts()`. Callers must not `await` between `registerFonts` and finishing drawing, or concurrent renders can leak font state into each other.
- **`t()`:** draws text at an explicit position, single line with ellipsis by default. `lb: true` wraps; `maxHeight` wraps and puts the ellipsis on the last line that fits. `stripEmoji()` removes emoji because the bundled fonts have no glyphs for them.
- **`decodeImageDataUri()`:** accepts only `...`. PNGs are pre-validated by `isDecodablePng()`, which re-inflates the IDAT chunks synchronously. PDFKit's own async zlib inflate throws an error that cannot be caught, so a corrupt PNG would otherwise crash the process.
- **`drawImageInBox()`:** wraps `doc.image` in a try/catch. A photo that fails to draw is labelled "Image could not be rendered".
- **`drawPaginatedRows()`:** repeatable-table pagination. The header is redrawn on each continuation page, and the last `KEEP_WITH_FOOTER = 3` rows always share a page with the footer (remarks and signatures).
- **`resolveCols()`:** columns without a width share the remaining width, with a 40 pt floor.
- **`fmtDate()`:** formats as DD/MM/YYYY in UTC.

2.4 Authored-template definition format (declarative dialect)

A stored `definition` is `{ pages: [ { sections: [...] } ] }`. Hydration rules (`authoredTemplate.js`):

- **header**: `infoFields` is an array of `{leftLabel, leftKey, leftFormat?, rightLabel, rightKey, rightFormat?}`. A format of `'date'` renders with `fmtDate`. Other keys: `companyName, formatNo, revNo, effDate, extraFormatLines, logoAsset, rLabelW, gap`. A missing `infoFields` throws.
- **table**: `mode`, `dataKey`, `title`, `headerHeight`, `rowHeight`, `gap`, `columns[{key,label,w,align,group,cell}]`. `cell` has exactly three shapes:
  - `{source:'row'}`: the default, `row[key]`
  - `{source:'sectionData', subfield, default}`: `sectionData[row.key][subfield] || default`
  - `{source:'constant', value}`

Fixed mode uses stored `fixedRows: [{key,label,...}]`. Repeatable mode can set `filterKey` (keep a row only if that field is non-empty). Its `footerHeight` is computed automatically by summing the known heights of the sections that follow (text 40, signature 36, fixed table header + rows × rowHeight). If any following section's height cannot be known in advance, it adds a flat **150 pt** buffer instead.

- **image**: `dataKey, width, height, title, gap, placeholder{text, annotations}`.
- **photo, signature, text**: passed through unchanged.
- `buildSampleData(definition)` synthesises preview data (`pdi_no: 'PREVIEW-0000'`, "Sample" strings, 2 sample rows, GO results, "Sample Name" signatures) for the preview endpoint.

The admin editor (`CRM/src/components/admin/PdiTemplateSectionEditors.jsx`) offers section presets: Header, Checklist (fixed table with an Item column `{source:'row'}` and a Result column `{source:'sectionData', subfield:'measured', default:'GO'}`), Fill-in list (repeatable table), Photos, Image, Signatures and Notes (text). Field keys are generated from labels by `CRM/src/utils/pdiTemplateSlug.js` (`slugify`, `dedupeKey` adds `_2`, `_3` and so on). The editor warns when a template has no Header, Checklist or Signatures section (`PdiTemplatesAdminPage.jsx#missingTypicalSections`).

2.5 Versioning, publish and archive (`authoredTemplates.js`)

- `pdi_templates` is **append-only**. The primary key is `(id, version)`.
- `create()` inserts version 1 with status `draft`.

- `saveNewVersion(id, {name, definition, status})` always inserts `latest.version + 1`. Any field left out is copied from the latest row. `PUT` saves, `POST /:id/publish` sets `status='active'`, and `POST /:id/archive` sets `status='archived'`. Each of these creates a new version row. The web editor's "Publish" first PUTs the content and then POSTs publish, so it produces two version rows.
- `getActive(id)` returns the latest version **only if that latest row is active**. An earlier bug returned an older active row underneath a newer archived one, which effectively un-archived the template.
- `listActive()` returns, for each id, the latest version when it is active. It feeds the template picker. `listAll()` returns the latest version of each id regardless of status, for the admin list.
- When a report is created against an authored template, it pins `template_version` to the version that is active at that moment (`PdiReports.resolveTemplateVersion`). Later renders always use that exact version, even after the template is edited or archived.
- **Delete**: `DELETE /api/pdi/admin/templates/:id` hard-deletes every version, but only when no report uses the template (`PdiReports.countByTemplateId`). Otherwise it returns 409 and tells the admin to archive the template instead.
- A concurrent save that hits a duplicate `(id, version)` (Postgres error 23505) returns 409 "This template was just edited elsewhere".

3. Built-in template specifics

3.1 General (`templates/general.js`)

- **Page 1, Electrical.** Header (Format `CASPL/QA/F/14`). A repeatable `rows` table (columns: S.No, Motor Sr. No, Voltage, Current Measured F/R, RPM Measured F/R, Remarks) with a spec row built from `spec_current_display || spec_current_standard` and `spec_rpm_display || spec_rpm_specified`. A fixed `general_electrical` GO/NG checklist (5 rows). Remarks (default "ALL MOTORS OK, PASSED."). Signatures `prepared_by` and `approved_by`.
- **Page 2, Mechanical.** A drawing image placeholder. A repeatable `rows` table (Motor Length, Shaft Length, Shaft Diameter, PCD and MTG under a "Mounting Holes" group, Key Dim., Locating Dia., Remarks) with a spec row. A `general_mechanical` checklist (4 rows; the cable-length labels are interpolated from data). Remarks. Signatures `prepared_by_mechanical` and `approved_by_mechanical`, each falling back with `??` to the page 1 names for reports saved before app 1.0.8.
- **Page 3.** Freeform photos (`photos`).
- Only rows with a non-blank `motor_sr_no` are rendered.
- F/R readings such as "10.2/10.4" are split by `parseForwardReverse`, and **both** values are tolerance-checked.
- Spec text prints exactly as sent in `spec_X_display` (app 1.0.8+), with no bracketed range. Older reports fall back to the plain nominal.

3.2 AutoNXT Motor (`templates/autonxt.js`)

- Header (Format `CASPL/QA/F/23`, extra line "REV DT: 11/10/2024") plus a Motor Sr.No / Controller Type info row.
- **A. Performance Test @ No Load:** a fixed 8-row table (RPM 500 to 3000, 560 V). BEMF and Running Current each have a Specified column (wrapping) and a Measured column (tolerance-flagged). Values come from `data.performance_test[rpm_key].{bemf_measured,current_measured}`.
- **B. General Check:** 7 fixed GO/NG rows from `general_check`.
- **C. Physical Parameters:** 25 fixed rows from `physical_parameters[key].measured` (default "GO").
- Signatures (3 roles): `prepared_by_electrical`, `prepared_by_mechanical`, `approved_by`.
- Page 3: six fixed photo slots (`overall_motor`, `name_plate`, `sr_no_marked`, `power_cable_shielding`, `resolver_cable`, `front_rear_view`).
- `SPEC_DEFAULTS` holds 19 tolerance-eligible fields: 16 BEMF/current fields, plus `motor_total_length`, `shaft_op_length` and `locating_dia` (bilateral, −0.01/−0.05). For each field id:
  - `spec_<id>_display` is what prints.

- `spec_<id>`, `_tol_mode`, `_tol` and `_tol_minus` drive the red flag only.

The default applies when a report has no override. The one-off data fix `scripts/migrations/2026-09-30-fix-autonxt-spec-display.js` (supports `--dry-run`) rebuilt the stored `_display` values and calls `PdiReports.reRenderFinalizedReportForMaintenance`.

3.3 AutoNXT Controller (`templates/autonxt_controller.js`)

- Header (Format `CASPL/QA/F/26`, Rev 01) plus a Controller Sr.No / Controller Type row.
- **A. PARAMETER CHECK:** a repeatable `parameter_rows` table (S.NO, PARAMETER, SPECIFICATION, MEASURED VALUE, REMARKS). Rows are seeded from `CONTROLLER_TYPE_PRESETS` (currently only `CASHV38140`, 35 firmware parameters).
- `computeRemarks(row)`:
  - A stored remark that is not OK/NG (for example "NA") is an override and wins.
  - A blank measured value gives a blank remark.
  - If both values are plain decimals they are compared numerically (`1 = 0001 = 1.0`); otherwise by exact trimmed string.
  - NG flags both the Measured and Remarks cells.
- **B. General Check:** 10 fixed GO/NG rows. "NG" flags the cell.
- Remarks boxes default to "ALL OK, PASSED." only when their own section has no NG; otherwise they are blank.
- Page 2: five fixed photo slots (140 pt high), then signatures (`prepared_by`, `approved_by`) on both pages. Two-digit page numbers and "END OF REPORT" appear on the last page.

4. Tolerance evaluation and rounding (`models/operations/pdi/tolerance.js`)

`checkTolerance(measured, nominal, mode, tol, tol2)` returns `{outOfRange}`:

- `'±'` (symmetric): allowed range is `[nominal − |tol|, nominal + |tol|]`.
- `'%'`: `delta = |nominal| × |tol| / 100`.
- `'bilateral'`: `[nominal + min(tol, tol2), nominal + max(tol, tol2)]`. The min/max wrapping means a sign typed in either field cannot invert the range.
- Any mode other than `'bilateral'` is treated as symmetric or percentage, and `tol2` is ignored.
- A non-numeric measured value, a missing nominal, or a non-numeric tolerance means the check is **skipped** (`outOfRange: false`). It never throws or flags. This is why a "GO" in a tolerance-eligible row is never red.
- **Boundary rounding:** a float slack `EPS = 1e-9` is applied (`measured < low − EPS || measured > high + EPS`). Without it, `0.7 + 0.1 = 0.7999999999999999`, so a reading exactly on the 0.8 limit would be flagged. This was added in commit `0c077787`.
- The same formula is deliberately **duplicated in three places** and must stay identical: the backend, the web forms (`TOLERANCE_EPS` and `checkTolerance` in `CRM/src/components/admin/PDIGeneratorForm.jsx` and `AutoNXTGeneratorForm.jsx`), and the mobile app (`pdi-erp-app/src/services/tolerance.ts`).
- The web forms also rebuild display strings before every save: `formatSpecDisplay` in General gives "50 ±0.5", "3000 ±5%" or "50.0 +0.1/-0.2"; `formatAutoNxtSpecDisplay` does the same for AutoNXT.

5. Report lifecycle

5.1 Statuses (`pre_dispatch_inspection_reports.status`)

Status	How it is set
`Pending`	`createReport`, `duplicateReport`, and every lot member created by `createBatch`
`In Progress`	PATCH with `status: 'In Progress'` (web forms send it on every save of a non-Completed report)
`Failed`	PATCH with `status: 'Failed'`, the mobile app's "Mark as failed" action. It was kept on purpose (commit `4d88e5eb`).
`Completed`	Set **only** by finalize (single report) or by lot finalize

`ALLOWED_PATCH_STATUSES = ['Pending','In Progress','Failed']`. Any other value returns 400 `INVALID_STATUS`. `Failed` does not lock the report: a Failed report can still be edited and finalized.

Lot (`pdi_report_batches.status`) moves `In Progress → Finalizing → Completed`, falling back to `In Progress` if finalize fails or the server restarts.

5.2 Create, save and duplicate

- `POST /api/pdi/reports` validates the template id (`resolveTemplateVersion`; the default is `general`), parses `inspection_date`, and inserts a Pending row. `inspected_by` defaults to `req.user.name`.
- **Signer denormalisation:** `extractSignerNames(data)` scans the keys of `data` for `/prepared/i` and `/approved/i`, joins multiple matches with " / ", and stores the result in the `prepared_by` and `approved_by` columns. This happens on create and on every data PATCH. These columns are used for list search and sort.
- **Date validation:** an unparseable `inspection_date` throws `INVALID_INSPECTION_DATE` (400) before any write. An Indian day-first value such as "22-09-2026" used to produce an opaque 500 (report #1059). On a Completed report a malformed date is silently ignored.
- **PATCH merge:** `data` is shallow-merged in JS as `{...currentData, ...fields.data}`. This is a deliberate fix: Postgres `||` would wrap non-objects into an array. A `data` value that is not an object returns 400 `INVALID_DATA_PAYLOAD`. (Design spec 2026-10-01 proposed `data || $N`; the code does the JS merge instead.)
- **Photos write-skip:** `photos = CASE WHEN photos = $n::jsonb THEN photos ELSE $n::jsonb END` keeps the existing TOAST value when a client re-sends identical photos.
- Every PATCH to a non-Completed report runs `revision_no = revision_no + 1`. If the client sends `expected_revision`, it is added to the `WHERE` clause (optimistic concurrency).
- **Guarded UPDATE:** `WHERE report_id = $id AND status <> 'Completed' [AND revision_no = $r] AND LOT_OPEN_GUARD`. `LOT_OPEN_GUARD` requires either `batch_id IS NULL` or a lot still `In Progress`, using `FOR SHARE` so a concurrent lot finalize and a save are serialised. If no row matches, `#rejectedWriteState` picks the error: `BATCH_MEMBER_LOCKED`, `REPORT_LOCKED`, `REPORT_VERSION_CONFLICT`, or "Report not found".
- **Stale-screen rule** (commit `e353328d`): a PATCH that includes `status` on a report that is already `Completed` is refused with `REPORT_LOCKED` ("This report was finalized while you were editing"). Clients send `status` only while they believe the report is open.
- **Duplicate** (`POST /:id/duplicate`): creates a new Pending report with the same template, customer_id and order_id, empty photos, and data with the per-run keys removed. The removed keys depend on the template:
  - general: `pdi_no, date, rows, prepared_by, approved_by, general_electrical, general_mechanical, electrical_remarks, mechanical_remarks`
  - autonxt: `pdi_no, date, motor_sr_no, performance_test, general_check, physical_parameters, page1_remarks, page2_remarks, prepared_by_electrical, prepared_by_mechanical, approved_by`
  - autonxt_controller: `pdi_no, date, controller_sr_no, parameter_rows, general_check, page1_remarks, page2_remarks, prepared_by, approved_by`
  - authored templates: only `pdi_no` and `date`

Header and spec fields carry over. Duplicating is the recommended way to "correct" a finalized report when the user is not permitted to edit it.

5.3 Finalize flow (single report): `PdiReports.finalizeReport`

1. Load the report including photos.
2. If it is a lot member, fail with `BATCH_MEMBER_USE_LOT` (409).
3. If `data.pdi_no` is missing, fail with `PDI_NO_REQUIRED` (400).
4. If it is already Completed, fail with `REPORT_LOCKED` (409).
5. Render: `PDIGenerator.generate(template_id, template_version, {...data, photos})`. The photos are optimised first (see §6).
6. Run a guarded `UPDATE ... SET status='Completed' WHERE status <> 'Completed' AND batch_id IS NULL`. If it loses a race it returns `REPORT_LOCKED`, and nothing is uploaded.
7. Respond immediately with the PDF bytes (`Content-Disposition: attachment; filename="PDI_<pdi_no>.pdf"`) and emit the Socket.IO event `pdiReportUpdate`.
8. In the background, without waiting:
   - Upload to Google Drive (`uploadBufferToDrivePrivate`, `PDI_<no>.pdf`) and store `drive_file_id`. If the report was deleted during the upload, the Drive file is deleted. This step never fails the finalize.
   - Write the disk cache with `pdfCache.write`.
9. The controller logs one timing line: photo count, loadMs, renderMs (with downscale ms/bytes), updateMs, pdfBytes, totalMs, and X-Request-ID.

The web forms always **save first, then finalize**, so the PDF matches the screen (`PDIGeneratorForm.jsx#handleFinalize`). They also show a confirm dialog with warnings before finalizing.

5.4 PDF download and cache

`GET /api/pdi/reports/:id/pdf` (`getPdfForDownload`):

- It reads only `status` and `pdi_no` first. A missing `pdi_no` returns 400 `PDI_NO_REQUIRED`.
- A Completed report is served from `pdfCache` when it is there (`source: 'cache'`).
- Otherwise the PDF is rendered. For a member of a Completed lot, the lot's authoritative fields (`getActiveBatchOverride`) are applied. A Completed report's render is then cached. Drafts are rendered every time and never cached.

`pdfCache.js`:

- Files live in `PDI_PDF_CACHE_DIR` or `os.tmpdir()/pdi-pdf-cache`, named `report-<id>.pdf` or `batch-<id>.pdf`.
- Writes go to a temp file that is then renamed. Reads check for the `%PDF-` header, and a corrupt file is deleted.
- LRU eviction by mtime keeps `MAX_FILES = 100` across both prefixes.
- Every operation is best-effort and never throws. The cache is emptied by a redeploy (it lives on the container's disk).

5.5 Editing a finalized report and the revisions table

Rule (`patchReport`, Completed branch; spec `docs/superpowers/specs/2026-09-30-pdi-finalized-report-editing-design.md`):

- The caller's role must have `can_write` on module `PreDispatchInspectionReports` in the `permissions` table (`hasPermission(role_id, ...)` in `middleware/auth.js`). Otherwise the request fails with **403 `FINALIZED_REPORT_FORBIDDEN`**.
- `expected_revision` is required and must equal the current `revision_no`. Otherwise the request fails with **409 `REPORT_VERSION_CONFLICT`**.
- Only `data` (and therefore `prepared_by`/`approved_by`) and `photos` can change. `status`, `inspected_by`, `customer_id`, `order_id` and `inspection_date` are never added to the SET clause. An earlier version stripped them after the fact, which left an unbound parameter (Postgres error 42P18).
- Inside one transaction: a guarded `UPDATE ... WHERE status='Completed' AND revision_no=$cur AND LOT_OPEN_GUARD` (which bumps `revision_no`), then `INSERT INTO pdi_report_revisions (report_id, revision_no, data, edited_by)` with the **pre-edit** data. If the UPDATE matches no row, the transaction is rolled

back, no audit row is written, and the request fails with `REPORT_VERSION_CONFLICT` (or `BATCH_MEMBER_LOCKED`).

- After the commit, `#reRenderFinalizedReport` runs in the background. It renders again, overwrites the cache, uploads a new Drive file, swaps `drive_file_id` under `SELECT ... FOR UPDATE`, and deletes the superseded Drive file. The background promise is attached as a non-enumerable `background` property so it never leaks into the JSON response.
- Because `LOT_OPEN_GUARD` applies here too, members of a finalized lot cannot be edited at all.
- Photos are not snapshotted in revisions.
- `GET /api/pdi/reports/:id/revisions` returns `[{revision_no, edited_by, edited_at, data}]`, newest first. It has no extra permission gate beyond PDI access.

5.6 Delete

`DELETE /api/pdi/reports/:id` is a single guarded statement: `DELETE ... WHERE report_id=$1 AND batch_id IS NULL AND (status <> 'Completed' OR $canWrite)`.

- Deleting any lot member fails with `BATCH_MEMBER_LOCKED`; the whole lot must be deleted from the lot page.
- Deleting a Completed report without `can_write` fails with `FINALIZED_REPORT_FORBIDDEN`.
- If the row changed in the meantime, it fails with `REPORT_VERSION_CONFLICT`.
- On success it removes the cache entry, makes a best-effort Drive delete, emits `pdiReportUpdate {status:'Deleted'}`, and returns `{report_id}`. Revision rows cascade.

5.7 Listing

`GET /api/pdi/reports?limit&cursor&offset&status&template_id&search&sortBy&sortDir`:

- `limit` is clamped to the range 1 to 100 (default 10).
- **Default ordering** is `report_id DESC` with keyset/cursor pagination (`cursor` = the last report_id).
- **Explicit sort** uses an allow-list of 7 keys: `sr_no, pdi_no, customer_name, status, prepared_by, approved_by, inspection_date`. It uses offset pagination, `NULLS LAST`, and `report_id` as a tie-breaker.
- **Search** is an ILIKE match over pdi_no, customer name (form value or linked customer), status, prepared_by, approved_by, motor_sr_no, controller_sr_no and the authored template name, plus built-in template names (matched in JS).
- **Response:** `{data:[{report_id, sr_no, status, template_id, template_name, customer_id, order_id, pdi_no, customer_name, inspected_by, prepared_by, approved_by, inspection_date, report_link, motor_sr_no, controller_sr_no, batch_id, lot_index, lot_quantity, batch_status, batch_pdi_no}], total, cursor, offset}`.

6. Photos: limits and optimisation

Client side (web):

- Crop to 4:3 and compress to a long edge of 1600 px, JPEG quality 0.85 (`CRM/src/utils/pdiImageUpload.js` and copies in each form).
- Raw file cap is 20 MB.
- Ceilings:

Form	Photo ceiling
General (`PDIGeneratorForm.jsx`)	`MAX_PHOTOS = 12` slots, `MAX_IMAGES_PER_SLOT = 25`, `MAX_TOTAL_PHOTOS = 60`, `MAX_ROWS = 100`
AutoNXT and Controller forms	`MAX_IMAGES_PER_SLOT = 10`
Generic form (`GenericPdiSections.jsx`)	`MAX_PHOTOS = 12`, `MAX_ROWS = 100`

- Saves are blocked client-side above `MAX_SAVE_PAYLOAD_BYTES = 35 MB`; the server limit is 40 MB.
- Forms send `photos` only when the photo signature has changed.

Storage shapes: freeform is `[{id, label, images:[dataURI]}]`. Fixed-slots is `{slotKey: dataURI | [dataURI] | null}`; AutoNXT originally stored a single string, and the renderer normalises both shapes.

`?photos=summary` on GET and PATCH: the server returns `photos` as `[{id, label, image_count}]`, computed inside Postgres (`photosSummarySql`), so the 20 to 40 MB photo value is never sent to Node or the phone. This is opt-in; app 1.0.8 and older still receive full photos. A PATCH still stores the photos in full.

Server-side optimisation (`photoOptimizer.js`, used for every render):

- `sharp` applies EXIF `.rotate()` (PDFKit ignores EXIF), resizes to fit inside **1000 px** without enlarging, flattens onto white, and encodes **JPEG quality 70**.
- **Concurrency is 4** (each decoded image is about 5.8 MB).
- If the result would be larger than the original, the original is kept.
- A photo that cannot be decoded is kept unchanged and only logs a warning.
- If `sharp` fails to load (its native binary is missing), photos pass through unchanged. The PDF is larger but not broken.
- Measured: a 60-photo PDF shrank from 21.6 MB to 8.1 MB, at about 22 ms per photo.
- Photos are found through every `photo` section's `dataKey` in the template (`photoDataKeys`).

7. Lots / batches (AutoNXT)

Spec: `docs/superpowers/specs/2026-09-29-autonxt-batch-reports-design.md`. Model: `models/operations/pdiReportBatches.js`.

- **Batchable templates:** `autonxt` and `autonxt_controller` only. Any other template fails with `TEMPLATE_NOT_BATCHABLE`.
- **Quantity:** an integer from 1 to 50 (`MAX_LOT_QUANTITY` is a technical safety cap). A digit string is accepted after trimming; values like "2.5", "1e1", "" or a boolean fail with `INVALID_LOT_QUANTITY`.
- **Create** (`POST /api/pdi/report-batches`): runs as one transaction. It inserts the lot row (status `In Progress`) and N member reports (`Pending`, `lot_index` 1 to N, `template_version` pinned, `photos '{}'`). Each member's data is `{pdi_no, ...shared}`. If any insert fails, the whole transaction rolls back. It emits `Pending` for each member.
- **Shared fields** (`batchOverrides.js#SHARED_FIELDS`): `customer_name, product_id, product_specifications, drawing_no, controller_type`. They are optionally seeded at creation (trimmed strings only). On a Controller lot, `controller_type` must be a key of `CONTROLLER_TYPE_PRESETS`, otherwise `INVALID_CONTROLLER_TYPE`; a Motor lot accepts free text.
- **Update** (`PATCH /:batchId`): only while the lot is `In Progress` (otherwise `BATCH_ALREADY_FINALIZED`). It takes the lot row with `FOR UPDATE`. A cleared field becomes `NULL` on the lot and `''` in each member's data. An empty `pdi_no` fails with `PDI_NO_REQUIRED`. The change is copied into every member's `data` in the same transaction, and each member's `revision_no` is bumped so open forms get a version conflict instead of writing old values back. The web forms treat these "lot fields" as read-only on member reports (`LOT_FIELDS` in `AutoNXTGeneratorForm.jsx`).
- **Delete** (`DELETE /:batchId`): only while `In Progress`. Members and the lot row are deleted in one transaction. Cache entries and Drive files are then cleaned up, and `Deleted` is emitted for each member.
- **Finalize** (`POST /:batchId/finalize`):
  1. A guarded flip `In Progress → Finalizing`. If the lot is already Finalizing this fails with `BATCH_FINALIZING`; if it is Completed, with `BATCH_ALREADY_FINALIZED`. While the lot is `Finalizing`, every member save is refused by `LOT_OPEN_GUARD`.
  2. Validate: the member count must equal `lot_quantity` (otherwise `BATCH_INCOMPLETE`). Every member must have a `motor_sr_no` (or `controller_sr_no` for a Controller lot). Missing values give `MOTOR_SR_NO_REQUIRED` or `CONTROLLER_SR_NO_REQUIRED`, with `lots: [indexes]` in the response body. Per-member `pdi_no` is no longer required.
  3. Render **one combined PDF** with `PDIGenerator.generateCombined` in lot_index order. The lot's `pdi_no` and any shared field that is set override each member's data. Photos are loaded **one unit at a time** through

`loadPhotos()` to limit memory. Each unit's pages are numbered from "Pg 1 of N" again, matching the real Compage QA format. There is no PDF-merge library; PDFKit renders every unit into one document.

4. In one transaction, mark every member that is not yet Completed as `Completed` (members already Completed, from older data, are skipped), and move the lot `Finalizing → Completed`.
5. Remove each member's single-report cache entry, emit `Completed` for each member, and respond with the PDF (`PDI_BATCH_<pdi_no>.pdf`). In the background, back up to Drive (`PDI_BATCH_<no>.pdf`, stored in `pdi_report_batches.drive_file_id`) and write the cache entry `batch-<id>.pdf`.
6. A `finally` block puts the lot back to `In Progress` on any failure before the commit, such as a bad photo or running out of memory.

- **Startup recovery** (`server.js` lines 103-109): on boot, `UPDATE pdi_report_batches SET status='In Progress' WHERE status='Finalizing'` resets lots left stuck by a crashed process. This is safe because only one container runs at a time.
- **Combined PDF download** (`GET /:batchId/pdf`): fails with `BATCH_NOT_READY` (409) unless the lot is Completed. It serves from cache, or re-renders with the same overrides and caches the result.
- **Member PDFs after the lot is finalized:** `getActiveBatchOverride(reportId)` applies the lot's `pdi_no` and shared fields when the lot is Completed. This lives in a separate leaf module to avoid a circular `require`.
- **List** (`GET /`): newest first, with `completed_count`, an optional `template_id` filter, and a limit defaulting to 20 (maximum 100).
- **Get** (`GET /:batchId`): returns the lot columns plus `reports[{report_id, lot_index, status, motor_sr_no, controller_sr_no, pdi_no}]`. Any id that is not 1 to 9 plain digits returns `Batch not found` without querying the database.
- **Web:** `CRM/src/components/admin/AutoNXTBatchForm.jsx` and `AutoNXTControllerBatchForm.jsx` (routes `/pdi-generator/autonxt-batch?batch=` and `/autonxt-controller-batch?batch=`) poll every 5 s while the lot is `Finalizing`. The reports table shows a "Lot i/N" chip linking to the lot page and hides Delete for lot members.

8. Access control

- **Authentication** (`middleware/auth.js#authenticateToken`): a JWT bearer token. The user and `role_name` are re-read from the database on every request.
  - No token: 401 `AUTH_NO_TOKEN`.
  - Bad token: 403 `AUTH_INVALID_TOKEN`.
  - Unknown user: 403 `AUTH_INVALID_USER`.
  - Database error during the user lookup: **503 `AUTH_UNAVAILABLE`**, so clients do not log the user out mid-inspection (commit `0c077787`).
- **PDI access** (`middleware/pdiAccess.js#requirePdiAccess`): only **admin and production** roles. Access is granted if the lowercased `roles.role_name` is in `{admin, production}` **or** the `role_id` maps to one of them (`{1:'admin', 5:'production'}`, mirroring `ROLE_MAP` in `auth.controller.js`). Anyone else gets **403 `PDI_FORBIDDEN` "You don't have access to PDI reports." This deliberately does** not** use the `permissions` table, which has known gaps in its PDI rows. It is applied to every report, lot and template-picker route (commit `50eca80f`). Mobile technicians share the Production login and type their own names.
- **Template administration** (`routes/operations/pdiAdmin.js`): `req.user.role_id !== 1` gets 403 `{error:'Admin only'}`, with no error code.
- **Finalized edit and delete of a Completed report:** additionally gated by the `permissions` table (`PreDispatchInspectionReports.can_write`). See §5.5 and §5.6.
- **Web routes** (`CRM/src/routeConfig.jsx`):
  - `/pdi-generator` and all form and lot routes: `["admin","production"]`
  - `/pdi-templates`: `["admin"]`
  - `/pdi` (`PdiPage`): `["admin"]`
  - `/production-pdi` (`ProductionPDIPage`): `["production"]`

Both list pages wrap `CRM/src/components/shared/PdiReportsTable.jsx`. `RESUME_ROLES = ['admin','production']` controls the Resume, Duplicate and Delete buttons.

9. Database tables and key columns

`pre_dispatch_inspection_reports`

This is a legacy table that was extended. Its original columns include `report_id` (PK), `sr_no`, `customer_id`, `order_id`, `status`, `inspected_by`, `inspection_date`, the legacy `report_link`, and `created_at`. Columns added later:

Column	Source
`data JSONB NOT NULL DEFAULT '{}'`, `photos JSONB NOT NULL DEFAULT '[]'`, `template_id TEXT NOT NULL DEFAULT 'general'`, `drive_file_id TEXT`	`2026-09-04-add-pdi-report-content.js`
`template_version INT` (NULL for built-in templates)	`2026-09-08-add-pdi-templates.js`
`batch_id INT REFERENCES pdi_report_batches`, `lot_index INT`	`2026-09-29-add-pdi-report-batches.js`
`revision_no INT NOT NULL DEFAULT 1`	`2026-09-30-add-pdi-report-revisions.js`
`prepared_by TEXT`, `approved_by TEXT` (denormalised signers)	Added by an ad-hoc `ALTER` in `docs/superpowers/plans/2026-09-16-pdi-dataset-wide-search-sort-filter.md`. **There is no migration file** for these in `scripts/migrations/`.
Indexes `idx_pdi_reports_batch_id`, and unique partial `uq_pdi_reports_batch_lot_index (batch_id, lot_index) WHERE batch_id IS NOT NULL`	`2026-10-05-pdi-batch-indexes.js`

The API payload (`#toPayload`) also adds `report_link: /api/pdi/reports/:id/pdf`, `lot_quantity`, `batch_status` and `batch_pdi_no`, taken from a scalar subquery on the lot.

`pdi_templates`

`id TEXT`, `version INT`, `name TEXT`, `status TEXT CHECK IN ('draft','active','archived') DEFAULT 'draft'`, `definition JSONB`, `created_by INT REFERENCES users`, `created_at`. The primary key is `(id, version)`, with index `idx_pdi_templates_id_version (id, version DESC)`.

`pdi_report_batches`

`batch_id SERIAL PK`, `template_id TEXT`, `pdi_no TEXT`, `lot_quantity INT`, `status TEXT DEFAULT 'In Progress'` (In Progress, Finalizing or Completed; there is no CHECK constraint), `drive_file_id`, `created_by INT REFERENCES users`, `created_at TIMESTAMPTZ`. Shared fields added by `2026-09-30-add-pdi-batch-shared-fields.js`: `customer_name`, `product_id`, `product_specifications`, `drawing_no`, `controller_type`.

`pdi_report_revisions`

`revision_id SERIAL PK`, `report_id INT REFERENCES pre_dispatch_inspection_reports ON DELETE CASCADE`, `revision_no INT` (the revision that the snapshot **was**), `data JSONB` (the pre-edit data), `edited_by INT REFERENCES users`, `edited_at TIMESTAMPTZ`.

The `permissions` table row for module `PreDispatchInspectionReports` (`can_write`) is used for finalized edits and deletes.

10. API endpoint reference

All endpoints require `Authorization: Bearer <JWT>`. Errors from the reports and lots controllers are `{error, code}`, so a client can show `error` as-is and branch on `code`.

Templates (`routes/operations/pdi.js`, PDI access)

Method	Path	Notes
GET	`/api/pdi/templates`	`[{id,name,version}]`: built-in templates followed by active authored ones
GET	`/api/pdi/templates/:id/definition[?version=N]`	Authored templates only; a built-in id returns 404. Returns `{id,name,version,definition}`. `?version` serves that exact version even if archived (used to edit an existing report against its pinned version). Without it, only the active version is served.

Reports (`routes/operations/pdiReports.js`, PDI access)

Method	Path	Body / response
POST	`/api/pdi/reports`	`{customer_id?, order_id?, inspected_by?, inspection_date?, data?, photos?, template_id?}`. Returns 201 with the report payload (status Pending).
GET	`/api/pdi/reports`	List (§5.7)
GET	`/api/pdi/reports/:id[?photos=summary]`	Report payload: `report_id, sr_no, status, template_id, template_version, customer_id, order_id, inspected_by, inspection_date, report_link, drive_file_id, revision_no, batch_id, lot_index, lot_quantity, batch_status, batch_pdi_no, data, photos`
PATCH	`/api/pdi/reports/:id[?photos=summary]`	Any of `{status, inspected_by, inspection_date, customer_id, order_id, data, photos, expected_revision}`. Returns the updated payload.
GET	`/api/pdi/reports/:id/revisions`	`[{revision_no, edited_by, edited_at, data}]`
POST	`/api/pdi/reports/:id/duplicate`	Returns 201 with the new report payload
POST	`/api/pdi/reports/:id/finalize`	Returns `application/pdf` as an attachment
GET	`/api/pdi/reports/:id/pdf`	Returns `application/pdf` (cached or freshly rendered)
DELETE	`/api/pdi/reports/:id`	`{report_id}`

Lots (`routes/operations/pdiReportBatches.js`, PDI access)

Method	Path	Body / response
GET	`/api/pdi/report-batches[?template_id&limit]`	`[{batch_id, template_id, pdi_no, lot_quantity, status, customer_name, created_at, completed_count}]`
POST	`/api/pdi/report-batches`	`{template_id ('autonxt' default), pdi_no, quantity, customer_name?, product_id?, product_specifications?, drawing_no?, controller_type?}`. Returns 201 with the lot plus `reports[{report_id, lot_index, lot_quantity}]`.
GET	`/api/pdi/report-batches/:batchId`	The lot plus `reports[]`
PATCH	`/api/pdi/report-batches/:batchId`	`{pdi_no?, ...shared}`. Returns the refreshed lot.
DELETE	`/api/pdi/report-batches/:batchId`	`{deleted:true, batch_id, report_ids}`
POST	`/api/pdi/report-batches/:batchId/finalize`	Returns the combined `application/pdf`
GET	`/api/pdi/report-batches/:batchId/pdf`	Returns the combined PDF (409 `BATCH_NOT_READY` until the lot is Completed)

Template administration (`routes/operations/pdiAdmin.js`, admin only, role_id 1)

Method	Path	Notes
GET	`/api/pdi/admin/templates`	Latest version of each id, any status
GET	`/api/pdi/admin/templates/:id`	Full latest row
POST	`/api/pdi/admin/templates`	`{id, name, definition}`. Returns 201 (v1, draft). Returns 400 if a field is missing, 409 if the id is reserved by a built-in template or already exists.
PUT	`/api/pdi/admin/templates/:id`	`{name?, definition?}` creates a new version
POST	`/api/pdi/admin/templates/:id/publish` and `/archive`	Creates a new version with status active or archived
POST	`/api/pdi/admin/templates/:id/preview`	`{definition}`. Returns an inline PDF rendered with sample data. A malformed definition returns 400 with the renderer's message.
DELETE	`/api/pdi/admin/templates/:id`	204. Returns 409 if any report uses the template.

Side effects:

- Every mutating report or lot call invalidates Redis keys `pdi_*` (best-effort; skipped if Redis is not ready) and emits the Socket.IO event `pdiReportUpdate {report_id, status}`. The web table listens for it.
- Saves slower than 5 s log a warning with the byte count.

11. Error codes

Code	HTTP	Raised by
`INVALID_STATUS`	400	PATCH status not in Pending, In Progress or Failed
`INVALID_INSPECTION_DATE`	400	Unparseable date (create, or PATCH of a non-Completed report)
`INVALID_DATA_PAYLOAD`	400	`data` is not a JSON object
`PDI_NO_REQUIRED`	400	Finalize or PDF without `pdi_no`; lot create or update with an empty `pdi_no`
`UNKNOWN_TEMPLATE`	400	`Unknown PDI template: <id>` on create, or lot create
`FINALIZED_REPORT_FORBIDDEN`	403	Editing or deleting a Completed report without `can_write`
`PDI_FORBIDDEN`	403	Role is not admin or production
`REPORT_NOT_FOUND`	404	Report missing or non-numeric id
`REPORT_LOCKED`	409	Finalize of an already Completed report; save after the report was finalized; status sent for a Completed report
`REPORT_VERSION_CONFLICT`	409	Stale or missing `expected_revision`, or a concurrent edit
`BATCH_MEMBER_LOCKED`	409	Edit of a member of a non-open lot; delete of any lot member
`BATCH_MEMBER_USE_LOT`	409	Single-report finalize on a lot member
`INVALID_LOT_QUANTITY`, `TEMPLATE_NOT_BATCHABLE`, `INVALID_CONTROLLER_TYPE`, `MOTOR_SR_NO_REQUIRED`, `CONTROLLER_SR_NO_REQUIRED`	400	Lot validation. The serial-number errors include `lots: []`.
`BATCH_ALREADY_FINALIZED`, `BATCH_FINALIZING`, `BATCH_INCOMPLETE`, `BATCH_NOT_READY`	409	Lot state
`BATCH_NOT_FOUND`	404	Missing lot or malformed id
`INTERNAL_ERROR`	500	Anything else, returned as "Internal Server Error"
`AUTH_NO_TOKEN` 401, `AUTH_INVALID_TOKEN` 403, `AUTH_INVALID_USER` 403, `AUTH_UNAVAILABLE` 503		Authentication

Admin template endpoints return plain `{error}` without a `code`.

12. Mobile-app compatibility notes

- **`X-App-Version` header: **the mobile app sends it on every request and the web app does not. On login, a request carrying it receives a** 12 h** JWT instead of 1 h (`controllers/auth.controller.js`, `MOBILE_APP_TOKEN_TTL`), so a long on-site inspection does not expire mid-way. The header is self-declared, so it only widens the expiry window for someone who already has valid credentials.
- **`status: 'Failed'`** is still accepted on PATCH for the app's "Mark as failed" action (commit `4d88e5eb`, after a brief removal).
- **`?photos=summary`** is opt-in, so app 1.0.8 and older still receive full photos.
- **App 1.0.8 General fields** (`spec_*_display`, `prepared_by_mechanical`, `approved_by_mechanical`, a GO/NG PCD) render, and pre-1.0.8 reports keep their old output (`tests/pdi_general_v1_0_8.test.js`).
- **AutoNXT fixed-slot photos** accept both a single URI string (older app versions) and an array.
- **`AUTH_UNAVAILABLE` (503)** instead of 403 on a database outage stops the app from logging the user out.
- The app relies on the same `REPORT_LOCKED`, `REPORT_VERSION_CONFLICT` and `BATCH_*` codes and `{error, code}` shape. Saves re-send the full photo set, which the photo write-skip in §5.2 makes cheap.
- The 40 MB body limit and the free-text date handling (`INVALID_INSPECTION_DATE`) were both motivated by app behaviour.

13. Test coverage (`tests/pdi_*.test.js`, Jest)

Note: `*.test.js` files are gitignored but tracked, so a new test must be added with `git add -f`. Most tests mock `pool.query`; some assert on SQL text.

File	Covers
`pdi_admin.test.js`	Admin-only gate on every endpoint; create v1 draft; built-in and duplicate id 409; append-only save; publish and archive with picker visibility; preview PDF and 400 for a malformed definition; list latest; version pinning of reports; delete with and without report history
`pdi_authored_templates.test.js`	Version model: getLatest, getActive after archive, listActive and listAll, getByVersion(null), deleteAll
`pdi_authored_template_adapter.test.js`	Hydration of the three cell sources, automatic footerHeight and the 150 pt fallback, unknown section type, buildSampleData preview, image placeholder
`pdi_template_renderer.test.js`	Every section type; pagination; invalid modes; spec-row `labelSpan` and auto-height; `wrap` columns; table titles; signature `value()`; per-range page numbering, pad and END OF REPORT; text defaults and wrapping; photo height, footer and undecodable labels; signature page break
`pdi_primitives.test.js`	Data-URI and corrupt-PNG checks, `resolveCols`, `fmtDate`, Roboto and Helvetica fallback, `stripEmoji`, `t()` ellipsis and wrapping, `drawPaginatedRows` keep-with-footer rule
`pdi_generator.test.js`, `pdi_generator_combined.test.js`	Template dispatch (code vs DB, wrong version), `pdi_no` required, combined render with per-unit numbering and lazy `loadPhotos`
`pdi_tolerance.test.js`, `pdi_autonxt_tolerance.test.js`	±, %, bilateral, skip rules, EPS boundary; the 19 AutoNXT spec defaults, verbatim display text, red flagging
`pdi_autonxt_controller.test.js`	35-row preset, `computeRemarks` rules, NG flagging, remarks defaults, 2-page layout, page padding and END OF REPORT
`pdi_general_v1_0_8.test.js`	1.0.8 vs older General rendering
`pdi_finalize_pdf.test.js`	Finalize responds before the Drive upload; Drive failure tolerated; orphan cleanup; cache write; locks; race; no photos read back; timings; download cache behaviour; pdfCache round-trip, eviction and unwritable directory
`pdi_pdf_cache_prefix.test.js`	`report` and `batch` prefixes, shared eviction budget

File	Covers
pdi_photo_optimizer.test.js	Downscale, aspect ratio, never-bigger, PNG to JPEG, undecodable input, freeform and slot shapes, no mutation, sharp missing, PDF size reduction
pdi_photos_summary.test.js	?photos=summary on GET and PATCH, both photo shapes
pdi_finalized_edit.test.js	Permitted edit with revision bump, snapshot and background re-render; FORBIDDEN; missing or stale revision; race with no phantom audit row; transaction order; stale-status refusal; excluded fields; JS merge; getRevisions
pdi_lot_rules.test.js	Lot fields in payload and list; search, sort and cursor; PATCH lot guard and revision rules; 0-row explanations; BATCH_MEMBER_USE_LOT; controller error mapping; requirePdiAccess
pdi_report_batches.test.js	createBatch validation and atomicity; shared fields; controller type; getBatch id parsing; finalizeBatch (Finalizing flip, lazy photos, all error states, revert on failure, legacy Completed member, pdi_no and shared overrides); list, update, delete; combined PDF cache and re-render
pdi_delete_report.test.js	Guarded DELETE rules
pdi_inspection_date.test.js	Date parsing, and Completed reports ignoring a bad date
pdi_auth_unavailable.test.js	503 on database failure, 403 on a bad token, pass-through

14. Web frontend map (CRM)

Component	Purpose
components/shared/PdiTemplatePicker.jsx (/pdi-generator)	Fetches /api/pdi/templates; goes straight to the form if there is only one template. Form URLs come from utils/pdiRoutes.js (pdiFormPath, pdiBatchPath), because built-in slugs differ from ids (autonxt_controller lives at /pdi-generator/autonxt-controller; the old path redirects).
components/admin/PDIGeneratorForm.jsx (General), AutoNXTGeneratorForm.jsx, AutoNXTControllerGeneratorForm.jsx	Hand-coded forms. Start a draft (POST), save (PATCH ?photos=summary with expected_revision and status In Progress), save then finalize, download PDF, "Reload latest" after a conflict, resume via ?report=<id>. Drafts that were never saved are deleted when closed.
components/admin/GenericPdiGeneratorForm.jsx (/pdi-generator/:templateId) plus GenericPdiSections.jsx and GenericPdiSidebar.jsx	Schema-driven form for authored templates. Fetches /definition?version= for existing reports. Data and photos are autosaved on separate debounced channels (about 1.5 s). Known limitation: on a Completed report the two channels can trigger a spurious REPORT_VERSION_CONFLICT. Keyed by templateId so the form remounts when the id changes.
components/admin/AutoNXTBatchForm.jsx, AutoNXTControllerBatchForm.jsx	Lot create, edit, finalize, download and delete; polls every 5 s while Finalizing
components/admin/PdiTemplatesAdminPage.jsx, PdiTemplateSectionEditors.jsx, PdiTemplatePreviewPane.jsx (/pdi-templates)	Admin authoring with a debounced live PDF preview through /preview. Id collisions are retried with numeric suffixes.
components/shared/PdiReportsTable.jsx (used by admin/PdiPage.jsx and production/ProductionPDIPage.jsx)	Reports table with search, status, template and sort filters, mobile cards, Resume, View PDF (disabled without a PDI No.), Duplicate, Delete (not shown for lot members), lot chip, and a live pdiReportUpdate socket listener
utils/pdiImageUpload.js, utils/pdiTemplateSlug.js, utils/pdiRoutes.js	Image crop and compress, key slugs, URL helpers

15. Notable gotchas and caveats

- The font names in `primitives.js` are module-global. `renderOptimizedPdfDoc` awaits photo optimisation **before** `registerFonts` and then draws synchronously, so it is safe. `generateCombined` registers fonts once and then **awaits** (template lookup, `loadPhotos`, optimisation) between units, which breaks the "no await" rule its own comment states. In practice this is harmless, because every document registers the same names (`Roboto` and `Roboto-Bold`, or Helvetica when the asset files are missing).
- The comment in `authoredTemplate.js` says that `TEXT_H` and `SIG_H` duplicate the renderer's constants. The values match (40 and 36), but the comment's own wording swaps them ("36/40").
- `GenericPdiSections.jsx` still has a stale comment citing a 25 MB server limit; the actual limit is 40 MB.
- `prepared_by` and `approved_by` columns have no migration script in the repo.
- `pdi_report_batches.status` has no CHECK constraint.
- Several design specs (for example `2026-09-29-autonxt-batch-reports-design.md`, written before `requirePdiAccess` existed) describe the PDI routes as "no permission gate". That is no longer true.

Client Applications: Web CRM and PDI Mobile App

Source roots:

- Web CRM: `C:\Users\Rahul\OneDrive\Desktop\Projects\ERP-CRM\CRM` (paths below are relative to it unless stated)
- PDI mobile app: `C:\Users\Rahul\OneDrive\Desktop\Projects\ERP-CRM\pdi-erp-app` (git repo). Note: the `main` checkout is at `10e518e` (app v1.0.1, docs-only commits on top). The current app code lives on branch `origin/feature/pdi-testing-feedback-1.0.7` (head `faa0d61`, 2026-10-09, app v1.1.1 build 14), checked out as the detached worktree `ERP-CRM\pdi-erp-app-review`. Section 2 documents that branch; differences from `main` are called out.

1. Web CRM (React SPA)

1.1 Tech stack (`package.json`)

Area	Library
UI runtime	React 19 (`react`, `react-dom` ^19.0.0)
Build tool	Vite 6 (`vite`, `@vitejs/plugin-react`)
Routing	`react-router-dom` ^7.2 (BrowserRouter)
State	Redux Toolkit ^2.11 + `react-redux` ^9
HTTP	`axios` ^1.8 (23 files) and native `fetch` (61 files) — no shared axios instance
Realtime	`socket.io-client` ^4.8
Styling	Tailwind CSS 3.4 (+ PostCSS/autoprefixer); `bootstrap` 5.3 is a dependency but is not imported anywhere in `src` (Tailwind is the effective styling layer)
Icons	`lucide-react`, `react-icons`
Other	`@hello-pangea/dnd` (drag/drop, PDI template editor), `react-easy-crop` (photo crop), `react-select`, `react-modal`, `xlsx` (SheetJS, Excel export), `qrcode`, `number-to-words` (invoice amounts in words), `date-fns`, `lodash.debounce`, `serve` (static server for `npm start`)

Scripts: `dev` (vite), `build` (vite build -> `dist/`), `start` (`serve dist -s -p $PORT`), `lint` (eslint), `preview`.

1.2 Entry point and app bootstrap

- `src/main.jsx` — creates the React root, wraps `<App/>` in the Redux `<Provider store>`, and calls `installApiStatusTracking(store)` before render (global 401/403 handling, see 1.5).
- `src/app/store.js` — `configureStore` with two slices only:
  - `auth` (`src/features/auth/authSlice.js`): `userRole`, `userName`, `token`, `showLogin`, `socketStatus` (`'idle' | 'connected' | 'error'`). Initial state is hydrated synchronously from `localStorage` (`token`, `role`, `name`) so a page refresh renders the current route as the signed-in user. Actions: `setAuth`, `logout`, `toggleLogin`, `setSocketStatus`.
  - `notifications` (`src/features/notifications/notificationSlice.js`): array of `{id, type, message}` toasts; `addNotification` de-duplicates identical messages; `removeNotification`, `clearNotifications`.
  - All business data (orders, inventory, PDI, etc.) is held in component-local state, not Redux.
- `src/App.jsx` (`AppWrapper` = `<Router><ErrorBoundary><App/></ErrorBoundary></Router>`):
  - Re-reads `token`/`role`/`name` from localStorage on mount; shows `LoginModal` if absent.
  - Login success (`handleLoginSubmit`) stores `role`, `name`, `token`, and a JSON `user` object `{user_id, role_name, name}` (used by Design/Representative pages to know "is this enquiry assigned to me").
  - Opens the shared Socket.IO connection when `userRole` + `token` exist; maps `connect` -> `socketStatus='connected'`, `connect_error` -> `'error'`.
  - Role-based redirect: if the current path is `/`, empty, or not in `allowedPathsByRole[role]` (`:param` segments matched generically by regex), navigates to `DASHBOARD_ROUTES[role]`.

  - Chooses chrome: roles with an entry in `config/roleNav.js` get the sidebar `AppShell`; others (hr, employee, ia_hr, ia_employee) get the classic top `Navbar` (`components/pages/Navbar.jsx`).
  - Always mounts `NotificationCenter`; mounts `ConnectionBanner` when `socketStatus==='error'`; mounts `ChatbotWidget` only for `admin`.
  - Logout: `POST {VITE_BACKEND_URL}/api/auth/logout` with Bearer token, then `localStorage.clear()`, Redux `logout`, socket disconnect.
- `src/routeConfig.jsx` — array of `{path, allowedRoles, component}`; `renderRoute` wraps every route in its own `ErrorBoundary` and passes `socket` + `userRole` props; unauthorised role -> `<Navigate to="/">`. Several shared paths pick a role-specific component (e.g. `/inventory` -> `StoreInventoryPage` / `ProductionInventoryPage` / `InventoryPage`; `/stock`, `/bom`, `/orders`, `/proforma` similarly). Generic PDI template forms (`/pdi-generator/:templateId`) are keyed by `templateId` to force a remount; a legacy `/pdi-generator/autonxt_controller` redirects to `/pdi-generator/autonxt-controller`. A typo redirect maps `/account-dashboard` -> `/accounts-dashboard`.

1.3 Roles, dashboards and allowed paths (`src/constants.js`)

`ROLES` defines 14 roles. `DASHBOARD_ROUTES` (landing page per role):

Role	Dashboard path	Dashboard component
admin	`/admin-dashboard`	`AdminDashboard`
sales	`/sales-dashboard`	`RoleDashboard`
design	`/design-dashboard`	`RoleDashboard`
representative	`/representative-dashboard`	`RoleDashboard`
production	`/production-dashboard`	`RoleDashboard`
store	`/store-dashboard`	`RoleDashboard`
dispatch	`/dispatch-dashboard`	`RoleDashboard`
accounts	`/accounts-dashboard`	`RoleDashboard`
customer	`/customer-dashboard`	`RoleDashboard`
service_repair	`/service-repair-dashboard`	`RoleDashboard`
hr (Compage)	`/hr-dashboard`	`PeopleDashboard`
employee (Compage)	`/employee-dashboard`	`PeopleDashboard`
ia_hr (Intute AI)	`/ia-hr-dashboard`	`PeopleDashboard`
ia_employee (Intute AI)	`/ia-employee-dashboard`	`PeopleDashboard`

`allowedPathsByRole` is a second, client-side allow-list used by the redirect effect; it must stay in sync with `routeConfig.jsx` (comments warn that each new PDI template slug must be added per role). Observed inconsistency: `/edit-profile` is in `allowedPathsByRole.representative` but `representative` is not in the route's `allowedRoles`, so a representative opening Edit Profile is bounced to `/`.

1.4 Navigation configuration

- `src/config/adminNav.js` — `adminNavSections`, single source of truth for the admin Sidebar accordion and the dashboard Module Showcase. Sections: Sales & Customer Management (Orders, Customer Invoices, Customers, Enquiries, Queries, Quotations, Proforma, Delivery Challan); Inventory & Materials (Finished Goods, Raw Materials, Price List, BOM, Part Drawings, Part Creation); Production Management (Work Orders, Motor Recipes, IPT Kit Assembly); Quality & Logistics (PDI Records, PDI Generator, PDI Templates, Dispatch Tracking, Problems); Procurement (Purchase Orders, Purchase Invoices); Service & Repair (Repair Records). Each section has an `accent` colour and `shortLabel`.
- `src/config/roleNav.js` — sidebar sections for sales, production, design, accounts, service_repair, store, dispatch, representative, customer (+ admin via `adminNavSections`). `getRoleNav(role)` returns `{dashboardPath, sections, socketToasts?}`. Customer config carries `socketToasts` for `orderUpdate` and `queryUpdate`.
- `src/config/peopleNav.js` — card menus + `socketToasts` for hr, employee, ia_employee, ia_hr (`getPeopleDashboard(role)`). Toast events: `attendanceMarked`, `leaveRequestCreated`, `payrollProcessed`,

`activities:created`, `ia_orders:created`.

1.5 Layout components (`src/components/layout`)

- `AppShell.jsx` — sidebar + header shell. Uses a `useMediaQuery('(min-width: 768px)')` hook: below `md` the sidebar becomes an off-canvas drawer opened from the header menu button (auto-closes on navigation); from `md` up it is fixed and collapsible, with the collapsed state persisted in `localStorage['sidebar:collapsed']`.
- `Sidebar.jsx` — fixed left `w-64` (collapsed `md:w-16`) navy gradient (`from-navy-800 to-navy-600`) accordion; auto-opens the section containing the current path.
- `TopHeader.jsx` — dark navy header with search, bell (animated `anim-ring`), clock (`useClock`), user and logout buttons, mobile menu toggle.
- `components/pages/Navbar.jsx` — classic top bar for people roles; also calls `/api/auth/logout`.
- `components/ErrorBoundary.jsx` — app-level and per-route error boundary.

1.6 Dashboards (`src/components/dashboards`)

- **AdminDashboard.jsx** — greeting (`shared/DashboardGreeting`), 4 KPI `StatCard`s (Open Orders, Pending Queries, Low Stock Items, Dispatches Today) loaded via `hooks/useAdminStats.js` -> `GET /api/dashboard/admin-stats` (fetch + Bearer token), a live activity feed/rail (`admin/ActivityFeed.jsx`, `ActivityRail.jsx`, last 15 events), and `ModuleShowcase` cards generated from `adminNavSections`. Socket listeners: `newQuery`, `queryUpdate`, `stockUpdate`, `customerUpdate`, `problem:created`, `solution:created`, `processUpdate` (each produces a toast + feed entry; comment notes the backend never emits `orderUpdate`, so no listener).
- **RoleDashboard.jsx** — generic module launcher for any role with a `roleNav` entry: greeting + `ModuleShowcase` of that role's sections. Calls no API; subscribes to the role's optional `socketToasts` (currently customer only).
- **PeopleDashboard.jsx** — HR/Employee/IA dashboards: warm `PeopleBackdrop` (from `shared/PeoplePage.jsx`), greeting with company subtitle (e.g. "HR · Compage", "Employee · Intute AI"), and accent-coloured cards from `peopleNav.js`. Calls no API; subscribes to the role's `socketToasts`.

1.7 Hooks (`src/hooks`)

Hook	Purpose
`useAdminStats.js`	`GET /api/dashboard/admin-stats`; returns `{stats, isLoading, error, fetchStats}`
`useProblems.js`	`GET /api/problems` plus socket `problem:created` / `solution:created` live merge
`useQueries.js`	`GET /api/queries?limit=10&offset=0`
`useNotify.js`	Dispatches `addNotification` with `success`/`error`/`info`/`warning`
`useClock.js`	Ticking clock for the header
`useCountUp.js`	Animated number count-up for KPI cards

1.8 How API calls are made

- **Base URL**: `import.meta.env.VITE_BACKEND_URL` (referenced ~143 times), usually with fallback `http://localhost:5000`. There is no axios instance or `baseURL`; each page builds `${BASE_URL}/api/...` itself.
- **Auth header**: `Authorization: Bearer ${localStorage.getItem('token')}` is added manually per request. Login (`components/pages/LoginModal.jsx`) is `POST /api/auth/login` with `{email, password}` and `credentials: 'include'`; response `{token, role, name, user_id}`; a 429 is shown as "Too many login attempts".
- **Global 401/403 handling** — `src/services/apiStatus.js` (`installApiStatusTracking`):
  - Monkey-patches `window.fetch` and installs an axios response interceptor.
  - On 401/403 to a `/api/` URL it reads the error `code`. Codes `AUTH_INVALID_TOKEN`, `AUTH_INVALID_USER`, `AUTH_NO_TOKEN` end the session (clear localStorage, disconnect socket, Redux logout, show login, warning toast "Your session has expired"). Safeguards: ignores late replies carrying an older token; for `AUTH_INVALID_TOKEN` only signs out if the JWT's `exp` really has passed (the backend can return that code on DB errors).

  - Any other 403 records a timestamp; `wasForbiddenRecently()` (10 s window) lets `components/pages/ConnectionError.jsx` say "Access denied" instead of "Connection interrupted".
- **ConnectionBanner** (`components/pages/ConnectionBanner.jsx`) — gold-accent floating banner "Connection interrupted", shown when the socket reports `connect_error`; actions: reconnect (implemented as logout -> sign in again) or dismiss (resets `socketStatus` to idle).
- **ConnectionError** (`components/pages/ConnectionError.jsx`) — full-page error state used by data pages; buttons: Retry, Back to dashboard, Sign out.
- **Vite dev proxy** (`vite.config.js`): `/api` -> `https://crmbackend-production-c426.up.railway.app` (legacy Railway backend); `allowedHosts: ['intuteai.up.railway.app']`.

Backend endpoint families used by the web client (counted from template literals in `src/components`): `/api/pdi/reports` (most used), `/api/pdi/report-batches`, `/api/pdi/templates`, `/api/pdi/admin/templates`, `/api/inventory` (+ `/available`, `/check-part-number`, `/hold/`), `/api/stock`, `/api/queries`, `/api/orders`, `/api/parts` (+ `/types`, `/next-code`), `/api/bom`, `/api/activities`, `/api/attendance` (+ `/summary`), `/api/service-repair`, `/api/invoice-records` (+ `/upload`), `/api/ia-orders` (+ `/check-serials`), `/api/quotation/generate`, `/api/proforma/generate`, `/api/purchase-order/generate`, `/api/delivery-challan/generate`, `/api/payslip/generate`, `/api/purchase-invoices`, `/api/customer-invoices`, `/api/part-drawings`, `/api/part-drawings-raw`, `/api/ipt-kits` (+ `/next-serial`), `/api/process/orders`, `/api/problems`, `/api/price-list`, `/api/dispatch-tracking`, `/api/employee-details`, `/api/users/employees-hr`, `/api/users/customers`, `/api/customers` (+ `/search`), `/api/enquiry`, `/api/motor-recipes` (+ `/dropdown-data`), `/api/auth/login|logout|update-password`, `/api/dashboard/admin-stats`, and chatbot endpoints `/api/{module}/chatbot`.

1.9 Realtime (Socket.IO)

- Shared connection: `src/services/socket.js` — singleton `io(VITE_BACKEND_URL, {path:'/socket.io', transports:['websocket','polling'], withCredentials:true, reconnection:true, reconnectionAttempts:5, reconnectionDelay:2000, auth:{token}})`; `connectSocket`, `getSocket`, `disconnectSocket`. Passed to every routed page as the `socket` prop.
- Additionally, ~27 page components import `socket.io-client` and open their own connection (e.g. `components/pages/OrdersPage.jsx` opens `io(backendUrl, {withCredentials:true, transports:['websocket']})` for `orderUpdate`; `components/admin/QueriesPage.jsx` similar).
- Events listened to across the client: `newQuery`, `queryUpdate`, `orderUpdate`, `stockUpdate`, `inventoryUpdate`, `customerUpdate`, `priceListUpdate`, `invoiceUpdate`, `dispatchUpdate`, `processUpdate`, `workOrderUpdate`, `instanceGroupUpdate`, `partDrawingsUpdate`, `partDrawingsRawUpdate`, `pdiReportUpdate`, `enquiryUpdate`, `enquiryActivity`, `problem:created`, `solution:created`, `bom:created|updated|deleted`, `ipt_kits:created|updated|deleted`, `ia_orders:created|updated|deleted`, `invoice_records:created|deleted`, `activities:created|updated|deleted`, `attendanceMarked`, `leaveRequestCreated`, `payrollProcessed`, plus `connect`/`disconnect`/`connect_error`/`reconnect`.

1.10 NotificationCenter

`components/pages/NotificationCenter.jsx` renders the Redux `notifications` array as a bottom-right stack of gradient toasts (green success, red error, yellow warning, blue info); each auto-dismisses after 4 s or on close. All pages raise toasts through `useNotify()`.

1.11 Chatbot widget (`src/chatbot`)

- Mounted only for `admin` (App.jsx).
- `chatbotConfig.js` — `CHATBOT_MODULES`: Orders (`orders/chatbot`), Customer Invoices (`customer-invoices/chatbot`), Customers (`customers/chatbot`), Inventory (`inventory/chatbot`), Stock (`stock/chatbot`), BOM (`bom/chatbot`), Work Orders (endpoint `null`, "not connected yet"). Each module has preset question chips (e.g. "Show pending orders", "Count orders by status", "Total invoice value", "Show monthly invoice trend").
- `chatbotApi.js` — `askChatbot(endpoint, message)` -> `POST {VITE_BACKEND_URL}/api/{endpoint}` with `{message}` and Bearer token; throws on non-OK using `data.response`/`data.error`.

- `ChatbotWidget.jsx` (~480 lines) — floating launcher, module selector, message list and loading state.

1.12 PDI support utilities

- `utils/pdiRoutes.js` — single source of truth for PDI form URLs: `pdiFormPath(templateId, reportId)` (slugs `general`, `autonxt`, `autonxt-controller`) and `pdiBatchPath(templateId, batchId)` (`autonxt-batch`, `autonxt-controller-batch`); `BATCHABLE_TEMPLATE_IDS = ['autonxt','autonxt_controller']`.
- `utils/pdiImageUpload.js` — 20 MB raw cap; crop to 4:3 (matches PDF photo box), downscale long edge to 1600 px, re-encode JPEG at 0.85 via canvas.
- `utils/pdiTemplateSlug.js`, `components/shared/PdiImageUpload.jsx`, `PdiReportsTable.jsx` (shared list used by admin `/pdi` and production `/production-pdi`), `PdiTemplatePicker.jsx` (`GET /api/pdi/templates`).

1.13 Theming (navy/gold) and responsive approach

- `tailwind.config.js` extends colours: `navy` 50 `#eef1f6` … 800 `#0b1a33`, 900 `#081328`; `gold` 300 `#f6d989`, 400 `#f2c14e`, 500 `#e0a92e`, 600 `#b8841c`. Display font `Plus Jakarta Sans` (Google Fonts, linked in `index.html`). Favicon `/IntuteAIYellow.png`.
- Conventions: navy sidebar/header, gold primary/create buttons and focus rings (`focus:ring-gold-400`), page background `bg-navy-50`, per-module accent colours (indigo/amber/emerald/violet/orange) on module cards; People dashboards use a warm cream backdrop with border `#f0e6c8`.
- Motion: CSS classes `anim-in` (staggered via `--i` custom property), `anim-bar-x`, `anim-ring` (bell), card hover lift.
- Responsive: mobile-first Tailwind breakpoints (`sm`/`md`/`lg` grids such as `grid-cols-1 sm:grid-cols-2 lg:grid-cols-4`), off-canvas sidebar below 768 px, toasts capped to `max-w-[calc(100vw-2.5rem)]`.

1.14 Build, deploy and environment

- **Build**: `npm run build` -> static bundle in `dist/` (`vite.config.js` `build.outDir`). A built `dist/` folder is present in the working tree.
- **Production deploy** — `.github/workflows/frontend-s3.yml` ("Deploy CRM Frontend to S3"): triggers on every push to `main` (paths `**`) and manually (`workflow_dispatch`). Steps: checkout, Node 18, cache `node_modules`, `npm install`, `npm run build`, sync `dist/` to the S3 bucket (`jakejarvis/s3-sync-action`, `--delete`, region `ap-south-1`), then `aws cloudfront create-invalidation --distribution-id E2JXDLEFAXYK5P --paths "/*"`. Secrets: `AWS_S3_BUCKET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`. So the web CRM is served as a static SPA from S3 + CloudFront and auto-deploys on push.
- **Legacy Railway config** — `railway.json` (NIXPACKS builder, `startCommand: "vite"`, 60 s health-check timeout) and `npm start` (`serve dist -s -p $PORT`) remain from the earlier Railway hosting; the Vite proxy/allowedHosts also reference Railway hosts.
- **Environment variables** (`.env`, `.env.local`; values not reproduced here):
  - `VITE_BACKEND_URL` — backend origin for REST and Socket.IO (required).
  - `VITE_DEFAULT_SALES_USER_ID`, `VITE_DEFAULT_ADMIN_USER_ID` — optional, used by `components/design/DesignEnquiryPage.jsx` when handing enquiries back to sales/admin (backend default sales user 7).
  - Vite inlines `VITE_*` values at build time, so the CI build environment must supply them.

1.15 Per-role page / feature catalogue

Common to all roles except representative: **Edit Profile** (`/edit-profile`, `pages/EditProfile.jsx`) — change own password (`/api/auth/update-password`).

admin (AppShell, `/admin-dashboard`)

- Admin Dashboard — KPI cards, live activity feed and module launcher.
- Orders (`/orders`, `pages/OrdersPage.jsx` + `forms/CreateOrderForm`/`EditOrderForm`) — create, edit and track sales orders with live `orderUpdate`.
- Customer Invoices (`/customer-invoices`) — list and manage customer invoices.
- Customers (`/customer-list`) — customer master with detail view.

- Enquiries (`/enquiries`, `admin/EnquiryPage.jsx`) — lead/enquiry pipeline across sales, design and representatives.
- Queries (`/queries`) — customer and internal query tickets with live updates.
- Quotations (`/quotation`), Proforma (`/proforma`), Delivery Challan (`/delivery-challan`), Purchase Orders (`/purchase-order`) — form-driven PDF document generators (`/api/{doc}/generate`).
- Documents hub (`/documents`, `admin/DocumentsHub.jsx`) — landing tiles for Quotations, Proforma Invoice and Delivery Challan.
- Finished Goods inventory (`/inventory`) and Raw Materials stock (`/stock`) — stock levels, part-number checks, holds.
- Price List (`/price-list`) — product pricing.
- Bill of Materials (`/bom`) — priced BOM management.
- Part Drawings (`/part-drawings` -> `/part-drawings/finished`, `/part-drawings/raw`) — drawing library for finished and raw parts.
- Part Creation (`/part-creation`) — create parts with auto-generated codes (`/api/parts/next-code`, `/types`).
- Work Orders (`/work-orders`, `/select-component/:orderId/:action`, `/processes/:orderId`, `/processes/non-motor/:orderId`) — production process tracking for motor and non-motor orders.
- Motor Recipes (`/motor-recipes`) — winding specification per customer motor.
- IPT Kit Assembly (`/ipt-kits`) — record component serials per IPT kit with auto serials.
- PDI Records (`/pdi`) — searchable table of all PDI reports.
- PDI Generator (`/pdi-generator` -> General, AutoNXT, AutoNXT Batch, AutoNXT Controller, Controller Batch, authored `/:templateId`) — fill in, save drafts and finalize PDI report PDFs, including lot/batch reports.
- PDI Templates (`/pdi-templates`, `admin/PdiTemplatesAdminPage.jsx`) — drag-and-drop authoring of new PDI report formats with live preview.
- Dispatch Tracking (`/dispatch-tracking`) — shipment status.
- Problems (`/problems`) — reported problems and solutions with live updates.
- Purchase Invoices (`/purchase-invoices`) — supplier invoices.
- Service & Repair (`/service-repair`) — repair job records (shared with service_repair).
- Chatbot widget — natural-language queries over orders, invoices, customers, inventory, stock and BOM.

sales (AppShell, `/sales-dashboard`)

- Orders, Customer Invoices, Dispatch Tracking, Price List, Raw-material Stock — shared pages as above.
- Quotations (`/sales/quotations`, `SalesQuotationForm`) — generate customer quotations.
- Proforma (`/proforma` -> `SalesProformaForm`) and Purchase Orders (`/purchase-order`); Documents hub (`/documents`).
- Enquiries (`/sales/enquiries`, `SalesEnquiryPage`) — manage and assign leads.
- Queries (`/sales-queries`) — handle customer queries.
- Finished Goods (`/sales-inventory`) — sales view of finished goods stock.

design (AppShell, `/design-dashboard`)

- Enquiries (`/design/enquiries`, `DesignEnquiryPage`) — design-side work on assigned enquiries, "Mark Done", hand back to sales/admin.
- Queries (`/queries`) — technical queries.
- Part Creation (`/design/part-creation`) — create part definitions.
- Motor Recipes (`/motor-recipes`).
- Raw and Finished Part Drawings (`/part-drawings/raw`, `/part-drawings/finished`) — upload/manage drawings.
- Unpriced BOM (`/bom`) — BOM view without prices.

representative (AppShell, `/representative-dashboard`)

- Enquiries (`/representative/enquiries`, `RepresentativeEnquiryPage`) — field sales lead capture and follow-up (`/api/enquiry`).

production (AppShell, `/production-dashboard`)

- Orders (`/production-orders`; `/orders` also routes to `ProductionOrdersPage`) — production view of orders.
- Work Orders and process pages (`/work-orders`, `/processes/...`) — update production process steps.
- Motor Recipes, IPT Kit Assembly — as admin.
- Queries (`/production-queries`) — production queries.
- Raw Materials (`/production-stock`), Finished Goods (`/production-inventory`), Unpriced BOM (`/production-bom-unpriced`).
- Raw/Finished Drawings (`/production-part-drawings-raw`, `/production-part-drawings`) — read drawings.
- PDI Records (`/production-pdi`) and PDI Generator (`/pdi-generator/...`) — same PDI tooling as admin except template authoring.

store (AppShell, `/store-dashboard`)

- Inventory (`/inventory` -> `StoreInventoryPage`) — manage finished goods.
- Raw Materials (`/stock` -> `StoreStockPage`) — raw material levels.
- BOM (`/bom` -> `StoreBOMPage`) — unpriced BOM.

dispatch (AppShell, `/dispatch-dashboard`)

- Queries (`/queries`), Stock (`/stock`), Dispatch Tracking (`/dispatch-tracking`) — dispatch updates and logistics.

accounts (AppShell, `/accounts-dashboard`)

- Customer Invoices, Purchase Invoices — billing and expenses.
- Dispatch Tracking, Orders — order and shipment visibility.

customer (AppShell, `/customer-dashboard`)

- Orders (`/customer-orders`) — track own orders with live `orderUpdate` toasts.
- Queries (`/customer-queries`) — raise and follow own queries with `queryUpdate` toasts.

service_repair (AppShell, `/service-repair-dashboard`)

- Repair Records (`/service-repair`, `service/ServiceRepairPage.jsx`) — log repair jobs with customer typeahead (`/api/customers/search`), status badges, photo thumbnails/lightbox.

hr — Compage (Navbar, PeopleDashboard)

- Attendance Summary (`/attendance-summary`) — daily/monthly attendance for all employees.
- Activities (`/hr-activities`) — create and assign tasks to Compage employees.
- Employee Details (`/employee-details`) — employee contact and onboarding records (`/api/employee-details`).

employee — Compage

- Attendance History (`/attendance-history`) — own attendance.
- My Activities (`/my-activities`) — update status of assigned tasks.

ia_hr — Intute AI

- Attendance Summary (`/ia-attendance-summary`).
- Activities (`/activities`) — manage team tasks.
- Dispatch Orders (`/ia-orders`) — VCU/HMI dispatch with serial numbers (format `{prefix}{code}--{running}`, e.g. `VCL001--058`; `/api/ia-orders/check-serials`).
- Payslip Generator (`/hr-payslips`) — manual payslip PDF (`/api/payslip/generate`).
- Invoice Generator (`/ia-invoices`) — customer invoices with auto GST, PDF and history (`/api/invoice-records`).

ia_employee — Intute AI

- Attendance History (`/ia-attendance-history`), Activities (`/activities`), Dispatch Orders (`/ia-orders`), Invoice Generator (`/ia-invoices`).

2. PDI Mobile App (`pdi-erp-app`)

2.1 Tech stack

It is a **bare React Native CLI app, not Expo** (no `eas.json`; `app.json` only holds `{"name":"PDIApp","displayName":"PDIApp"}`). Written in TypeScript.

Area	Library (feature branch `package.json`)
Runtime	React Native 0.87.1, React 19.2.3
Navigation	`@react-navigation/native` 7 + `native-stack` 7, `react-native-screens`, `react-native-safe-area-context`
Storage	`@react-native-async-storage/async-storage` ^3.1 (drafts)
Secure storage	`react-native-keychain` ^10 (login session) — not on `main`
Photos	`react-native-image-crop-picker` ^0.51 (camera/gallery + crop + compression)
Files/PDF	`react-native-blob-util` ^0.24 (PDF download to cache, publish to Downloads)
Date input	`@react-native-community/datetimepicker` ^9.2 — not on `main`
Tooling	Jest 29 (`@react-native/jest-preset`), ESLint, Prettier, TypeScript; Node >= 22.11

Package name / Android `applicationId`: `com.compageautomation.pdi`. Android `versionCode`/`versionName`: `main` = 2 / "1.0.1"; feature branch at commit `41fbc40` = 13 / "1.1.1"; `src/config/appVersion.ts` at branch head = `1.1.1 (14)`.

2.2 App structure (feature branch)

- `App.tsx` — `SafeAreaProvider` > `AppDialogProvider` (custom modal dialogs) > `AuthProvider` > `AppNavigator`; status bar hidden.
- `src/navigation/AppNavigator.tsx` + `types.ts` — native stack: `Login`, `Reports`, `TemplatePicker`, `BatchCreate {templateId, templateName, batchId?}`, `BatchOverview {batchId, fromCreate?}`, `ReportEditor {reportId?, createdHere?}`, `GenericReportEditor {reportId, templateId, templateName, batchId?, lotIndex?, lotQuantity?}`. Theme colours from `src/theme/colors.ts` (navy `#111B2E`, orange/amber `#F5A019`, cream background `#FFFCF2`).
- `src/screens/`
  - `LoginScreen.tsx` — email/password login; explains expired sessions.
  - `ReportsScreen.tsx` — merged list of server reports and device drafts, status filters (All/Pending/In Progress/Completed/Failed), pending-sync markers, duplicate/delete, PDF actions.
  - `TemplatePickerScreen.tsx` — loads `GET /api/pdi/templates`; General opens `ReportEditor`; other templates open `GenericReportEditor`; AutoNXT and AutoNXT Controller also show a "create lot" card.
  - `ReportEditorScreen.tsx` (~1600 lines on main) — the hand-built General template editor (header, electrical/mechanical checks with tolerance evaluation, photos).
  - `GenericReportEditorScreen.tsx` — data-driven editor that renders any template definition (`/api/pdi/templates/:id/definition`), used for AutoNXT Motor, AutoNXT Controller and admin-authored templates; lot-aware.
  - `BatchCreateScreen.tsx` / `BatchOverviewScreen.tsx` — lot creation and lot dashboard (see 2.6).
- `src/components/pdi/` — `ElectricalCheckForm.tsx`, `MechanicalCheckForm.tsx`, `PhotosForm.tsx`.
- `src/api/` — `client.ts` (fetch wrapper), `reports.ts`, `reportBatches.ts`, `templates.ts`.
- `src/auth/` — `AuthContext.tsx`, `sessionStorage.ts` (Keychain), `sessionEvents.ts` (expiry warnings), `expiredSession.ts` (backup draft on expiry).
- `src/services/` — `tolerance.ts` (Symmetric/Percentage/Bilateral checks mirroring backend), `autonxtParity.ts` (AutoNXT spec presets and display-text lock), `genericTemplate.ts` (initialise/split template data, validate definitions and dates), `finalizeWarnings.ts` (pre-finalize checks per template), `motorSerials.ts`, `inspectionDate.ts`, `reportList.ts` (merge local+server lists), `reportPhotos.ts`, `reportPdf.ts`, `pdiCompatibility.ts` (migrations of older draft/photo shapes).

- `src/data/` — `autonxtTemplate.ts`, `autonxtControllerTemplate.ts` (built-in definitions, incl. the 35-row CASHV38140 controller preset).
- `src/storage/` — `pdiDrafts.ts`, `genericPdiDrafts.ts`, `indexedDraftStorage.ts`.
- `__tests__/` — Jest suites for client, auth, screens, templates, tolerance, drafts, duplicate, etc.; `__mocks__/` for AsyncStorage, blob-util, image-crop-picker.

2.3 Authentication and token persistence

- `POST /api/auth/login` with `{email, password}` (email trimmed and lower-cased). Response must contain `token`, `name`, `role`.
- Role gate (feature branch): only `admin` or `production` may use the app (`PDI_FORBIDDEN` "This account doesn't have access to PDI reports"). On `main` the gate was a hard-coded email check (`admin@compageauto.com`); this is now deprecated, matching the backend's admin+production PDI permission.
- **Persistence**: on `main` the session lived only in React state (lost on app restart). On the feature branch `sessionStorage.ts` stores `{name, role, token, email, expiresAt}` in the platform Keychain/Keystore (`react-native-keychain`, service `com.compageautomation.pdi.session`, 5 s read timeout). At launch `restoreStoredSession()` drops a stored session whose JWT `exp` has passed or which lacks an email, and tells Login it expired.
- **X-App-Version header**: `src/api/client.ts` sends `X-App-Version: "1.1.1 (14)"` (from `config/appVersion.ts`) on every request. The backend (`CRM_BACKEND/controllers/auth.controller.js`) issues a **12 h** JWT when this header is present on login, versus **1 h** for the web CRM, because inspections are long and the app restores its login from secure storage.
- **Expiry handling**: a timer fires 10 min before `expiresAt`, emits `sessionExpiring` (open editors write a device draft) and shows a "Session expires soon" dialog. Any 401, or 403 with `AUTH_INVALID_TOKEN`/`AUTH_INVALID_USER`, triggers `notifySessionExpired()`; the editor backs up the open report as a `pendingSync` draft and Login explains "Your session expired".

2.4 API client and `/api/pdi` usage

`apiRequest<T>(path, options, token, timeoutMs)` in `src/api/client.ts`:

- Base URL hard-coded `https://api.intute.biz`.
- Headers: `Accept`/`Content-Type: application/json`, `X-App-Version`, `Authorization: Bearer <token>`.
- Timeout via `AbortController`: default 15 s, extended for large bodies (`30 s + 1 s per 64 KB`); report writes 60 s; PDF 60 s; lot PDF 180 s.
- Error mapping to `ApiError {status, code, lots?}` with friendly messages for 413, 429, 502, 503, 504 and any 5xx/`INTERNAL_ERROR` (raw server text never shown); `TIMEOUT`, `NETWORK_ERROR`, `BAD_RESPONSE` for client-side failures; 204 handled.

Endpoints used:

Purpose	Call
Template list / definition	`GET /api/pdi/templates`, `GET /api/pdi/templates/:id/definition`
List reports (cursor paging, status filter)	`GET /api/pdi/reports?limit=&cursor=&status=` -> `{data, total, cursor}`
Open report / light open	`GET /api/pdi/reports/:id`, `GET /api/pdi/reports/:id?photos=summary`
Create / duplicate	`POST /api/pdi/reports`, `POST /api/pdi/reports/:id/duplicate`
Save	`PATCH /api/pdi/reports/:id` (with `expected_revision` when editing a Completed report; retried once on 500)
Delete	`DELETE /api/pdi/reports/:id` (retried once on 500)
Revisions	`GET /api/pdi/reports/:id/revisions`
Finalize + PDF	`POST /api/pdi/reports/:id/finalize`, `GET /api/pdi/reports/:id/pdf`

Purpose	Call
Lots	`POST/GET /api/pdi/report-batches`, `GET/PATCH/DELETE /api/pdi/report-batches/:id`, `POST /api/pdi/report-batches/:id/finalize`, `GET /api/pdi/report-batches/:id/pdf`

Report statuses: `Pending`, `In Progress`, `Completed`, `Failed`. Report payload: `{template_id, inspected_by, inspection_date, status, data, photos}` where `data` is the shared JSON blob also edited by the web CRM (so tolerance logic must match the backend exactly). Write size is checked client-side before sending: max 60 photos per report (`MAX_GENERAL_REPORT_PHOTOS`) and 35 M characters per request (backend JSON limit 40 MiB).

PDF handling (`src/services/reportPdf.ts`): downloads with `react-native-blob-util` to `CacheDir/PDI_<pdi_no>.pdf`, sends `X-Request-ID` so phone and server logs can be matched, requests Android storage permission where needed, and publishes a copy to Downloads / opens it.

2.5 Offline / draft storage

- AsyncStorage namespace `pdi-app` (`createAsyncStorage('pdi-app')`). On `main`: one JSON array per type under keys `pdi:drafts` (General) and `pdi:generic-drafts` (template-driven), newest first.
- Feature branch (`indexedDraftStorage.ts`): drafts are stored one item per key with an index (`<root>:index:v2`, `<root>:item:v2:<id>`), large items split into 256 KB chunks with a generation-numbered manifest, migrated from the legacy single-blob key, and **scoped per account email** so one user's drafts are never read or deleted by another. Damaged entries are quarantined to `<key>:corrupt:<timestamp>` rather than overwritten, and the UI offers retry/delete for "device backup issues".
- Drafts carry `pendingSync: true` when a save could not reach the server (offline, expired session); the Reports list merges local and server entries and shows the device copy until it is uploaded. Drafts of a deleted lot can be purged from the device.

2.6 Lots / batches

Per `docs/pdi-batch-lot-reports.md` and `src/api/reportBatches.ts`:

- A batch is `{batch_id, template_id, pdi_no, lot_quantity, status}`; creating one atomically creates N ordinary PDI reports stamped with the shared `pdi_no` and `lot_index` 1..N.
- Supported for `autonxt` and `autonxt_controller`. Shared fields set once for the lot: `customer_name`, `product_id`, `product_specifications`, `drawing_no`, `controller_type`.
- Lot status: `In Progress` | `Finalizing` | `Completed` (a failed finalize returns to In Progress). Server errors may include `lots: number[]` (e.g. missing serial numbers) which the app turns into messages (`reportBatchErrorMessage`, `missingSerialMessage`).
- `BatchCreateScreen` creates the lot; `BatchOverviewScreen` lists units (serial `motor_sr_no`/`controller_sr_no`, status), opens each unit in `GenericReportEditor`, polls status while finalizing, then downloads/opens the combined lot PDF; handles a lot deleted elsewhere (404).

2.7 Photo capture and compression

`src/components/pdi/PhotosForm.tsx` (feature branch):

- Camera (`ImagePicker.openCamera`) or gallery (`openPicker`, multi-select up to remaining slots), each image then cropped.
- Crop/compress options: `cropping: true`, output 1600x1200, `compressImageMaxWidth 1600`, `compressImageMaxHeight 1200`, `compressImageQuality 0.85`, `includeBase64: true` (images stored as base64 data URIs inside the report JSON).
- Limits: 12 labelled photo entries, 25 images per entry, 60 images per report, ~33 M characters of photo data; the UI shows a running "n / 60 photos · ≈ size" counter. These numbers are kept in parity with the web CRM (`docs/pdi-web-mobile-parity-audit.md`).

2.8 Build and release

- No Expo/EAS. Builds use the React Native CLI and Gradle: `npm run android` / `npm run ios` for development, `npm start` for Metro, `npm test` for Jest.
- Release signing (`android/app/build.gradle`): reads `android/keystore.properties` (template `keystore.properties.example`) for the release keystore when present; otherwise the debug keystore is used. Version is bumped manually in `build.gradle` (`versionCode`/`versionName`) and `src/config/appVersion.ts` (which also drives the `X-App-Version` header).
- The release artifact is an Android APK/AAB distributed outside CI; there is no `.github` workflow in this repo. The API URL is hard-coded, so there are no environment files.

2.9 `docs/` folder summary (handoff documents from the backend/web side to the app developer)

File	Content
`pdi-general-template-changes.md` (1191 lines)	Adds Bilateral tolerance mode and Shaft Diameter to General; tolerance parity with backend
`pdi-shaft-diameter-length-split.md`	Reverts the combined "diameter/length" field into two separate measured fields
`pdi-web-mobile-parity-audit.md`	Side-by-side photo/upload settings, web vs mobile; one gap found (photos per entry)
`pdi-batch-lot-reports.md`	API contract for AutoNXT lot/batch reports
`pdi-finalized-report-editing.md`	Controlled editing of Completed reports (permission, `expected_revision` conflict check, revisions)
`pdi-autonxt-spec-display-lock.md`	Fix for AutoNXT "PDF display text" drifting from nominal/tolerance values
`pdi-report-open-timeout.md`	"Report could not be opened" timeout on photo-heavy reports; client fix (photo summary, longer timeouts)
`2026-10-01-qa-followups.md`	Forward/reverse measurement parser mismatch and lot shared-field lock fallback
`2026-10-05-autonxt-controller-template.md`	Handoff for the third template, `autonxt_controller` (fields, CASHV38140 preset, batch support)
`2026-10-05-autonxt-controller-followups.md`	Review of commit `b75bb07`; fixes required before release
`2026-10-06-pdi-app-full-hardening.md` (2380 lines)	Line-by-line hardening audit: no lost work, PDF matches screen, sync with backend/web
`2026-10-09-pdi-app-hardening-final.md`	Round-5 handover with a reference patch for every open item

Several of these (`2026-10-05-autonxt-controller-followups.md`, `2026-10-06-...`, `2026-10-09-...`, `pdi-finalized-report-editing.md`, `pdi-report-open-timeout.md`) are untracked or modified in the `main` working tree.

3. Observations useful for the documentation

1. The web client has no central HTTP layer: base URL and Bearer header are repeated per page; session expiry and 403 handling are centralised only through the fetch/axios interception in `services/apiStatus.js`.
2. Two parallel allow-lists (`constants.js` `allowedPathsByRole` and `routeConfig.jsx` `allowedRoles`) must be kept in sync; the representative/`/edit-profile` mismatch shows the risk.
3. Many pages open their own Socket.IO connection besides the shared one in `services/socket.js`, which multiplies connections per user.
4. `railway.json`, the Vite Railway proxy and `npm start` are legacy; the live web deploy is GitHub Actions -> S3 -> CloudFront (`ap-south-1`).
5. The PDI app's `main` branch is behind the active feature branch (`feature/pdi-testing-feedback-1.0.7`); Keychain session persistence, `X-App-Version`, the 12 h token, lots and per-account chunked drafts exist only on that branch.

Deployment and Operations

Environments and configuration

Variable	Purpose
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	PostgreSQL (AWS RDS, TLS)
JWT_SECRET	JWT signing key
REDIS_URL	Redis connection (default `redis://localhost:6379`)
EMAIL_USER, EMAIL_PASS	Gmail SMTP sender (app password)
FRONTEND_URL	CORS and Socket.IO origin
PORT	HTTP port (default 8000)
GOOGLE_SERVICE_ACCOUNT_PATH, GOOGLE_DRIVE_FOLDER_ID	Google Drive uploads
MANAGER_DAILY_REPORT_EMAIL	Recipient of daily task and attendance digests
VITE_BACKEND_URL (web)	API and Socket.IO base URL for the React build

Backend build and release

- **Container:** `Dockerfile` starts from `node:22.17.0-alpine`, runs `npm ci --omit=dev`, copies the source and `.env`, exposes 8000, and starts with `node server.js`.
- **CI/CD:** `.github/workflows/backend-erp.yml` runs **only on manual trigger** (`workflow_dispatch`). It builds and pushes `intuteai/erp-backend:latest` to Docker Hub, then connects to the EC2 host over SSH and runs `docker-compose down`, removes the old image, runs `docker-compose pull erp-backend`, and runs `docker-compose up -d`.
- Only one container runs at a time; the old one stops before the new one starts. This is why the PDI startup recovery can safely reset lots stuck in `Finalizing`.
- **Web CRM:** every push to `main` triggers a GitHub Actions workflow in the CRM repository. It runs the Vite build to `dist/`, syncs the output to S3 and serves it through CloudFront. `railway.json` and the Railway dev proxy in `vite.config.js` are left over from the old hosting.
- **PDI mobile app:** a plain React Native (non-Expo) Android build, signed with Gradle using `keystore.properties` and distributed as an APK.

Database migrations

Schema changes are plain Node scripts in `scripts/migrations/`, named by date and run by hand against the database. Recent examples are PDI report content, PDI templates, the representative role and its seed user, PDI report batches and their shared fields, PDI report revisions, the AutoNXT spec-display fix, and PDI batch indexes.

Logging and monitoring

Winston writes JSON lines with timestamps to `logs/error.log` (errors only), `logs/combined.log` (all levels) and the console. Rate-limit hits, logins, cron runs, PDI recovery and pool errors are all logged.

Development workflow

```
npm install        # install dependencies
npm run dev        # nodemon auto-reload
npm start          # production start
npm test           # Jest + Supertest
npm run lint       # ESLint
npm run format     # Prettier
```

Feature work is designed and planned in `docs/superpowers/specs` and `docs/superpowers/plans`; that folder holds about 22 design specs and plans, most of them for the PDI subsystem. Test files (`*.test.js`) match a `.gitignore` rule even though they are tracked, so a new test file must be added with `git add -f`.

Appendix A - REST API Endpoint Catalogue

This list is generated automatically from `server.js` mount points and the router files. **Access** shows the guard on each route. *Module:action* is a check against the permissions table (create maps to write). "PDI roles" means only admin and production. "Any authenticated" means a valid JWT is enough. The only public route is login. Total endpoints: **218**.

A.1 Authentication (5)

Method	Path	Access	Handler
POST	`/api/auth/login`	—	login
GET	`/api/auth/verify-token`	Any authenticated	verifyToken
GET	`/api/auth/user`	Any authenticated	getUser
POST	`/api/auth/logout`	Any authenticated	logout
PUT	`/api/auth/update-password`	Any authenticated	updatePassword

A.2 Core (39)

Method	Path	Access	Handler
GET	`/api/customers/search`	Any authenticated	search
GET	`/api/customers`	Customers:read	getAll
POST	`/api/customers`	Customers:create	create
GET	`/api/orders`	Orders:read	getAll
GET	`/api/orders/export`	Orders:read	exportOrders
POST	`/api/orders`	Orders:write	create
PUT	`/api/orders/:id/update`	Orders:write	update
POST	`/api/orders/:id/cancel`	Orders:write	cancel
POST	`/api/inventory`	Inventory:write	create
GET	`/api/inventory/available`	Any authenticated	getAvailable
GET	`/api/inventory/check-part-number`	Any authenticated	checkPartNumber
GET	`/api/inventory`	Inventory:read	getAll
PUT	`/api/inventory/:id`	Inventory:write	update
DELETE	`/api/inventory/:id`	Inventory:write	delete
POST	`/api/inventory/:id/hold`	Inventory:write	createHold
POST	`/api/inventory/hold/:hold_id/release`	Inventory:write	releaseHold
GET	`/api/inventory/:id/holds`	Inventory:read	getHolds
POST	`/api/inventory/:id/accept-return`	Inventory:write	acceptReturn
GET	`/api/stock/check-part-number`	Any authenticated	checkPartNumber
GET	`/api/stock`	Stock:read	getAll
POST	`/api/stock`	Stock:write	create
PUT	`/api/stock/:productId`	Stock:write	update
POST	`/api/stock/:productId/photo`	Stock:write, multipart upload	uploadPhoto
DELETE	`/api/stock/:productId`	Stock:delete	delete
POST	`/api/stock/:productId/adjust`	Stock:write	adjust
GET	`/api/users/customers`	Customers:read	getCustomers
GET	`/api/users/employees-hr`	Any authenticated	getEmployeesHr
GET	`/api/reports/order-summary`	Reports:read	orderSummary
GET	`/api/reports/inventory-status`	Reports:read	inventoryStatus
GET	`/api/dashboard/sales-dashboard`	dashboard:read	salesDashboard

Method	Path	Access	Handler
GET	/api/dashboard/customer-dashboard	dashboard:read	customerDashboard
GET	/api/dashboard/admin-dashboard	dashboard:read	adminDashboard
GET	/api/dashboard/design-dashboard	dashboard:read	designDashboard
GET	/api/dashboard/production-dashboard	dashboard:read	productionDashboard
GET	/api/dashboard/stores-dashboard	dashboard:read	storesDashboard
GET	/api/dashboard/dispatch-dashboard	dashboard:read	dispatchDashboard
GET	/api/dashboard/accounts-dashboard	dashboard:read	accountsDashboard
GET	/api/dashboard/admin-stats	dashboard:read	adminStats
GET	/api/dashboard	Any authenticated	main

A.3 Sales (33)

Method	Path	Access	Handler
GET	/api/enquiry/templates	Enquiries:read	getTemplates
GET	/api/enquiry/representatives	Enquiries:read	getRepresentatives
POST	/api/enquiry/refresh	Enquiries:read	refreshCache
POST	/api/enquiry	Enquiries:write	create
GET	/api/enquiry	Enquiries:read	getAll
GET	/api/enquiry/:id	Enquiries:read	getOne
PUT	/api/enquiry/:id	Enquiries:write	update
DELETE	/api/enquiry/:id	Enquiries:delete	delete
POST	/api/enquiry/:id/assign	Enquiries:write	assign
POST	/api/enquiry/:id/mark-done	Enquiries:write	markDone
POST	/api/enquiry/:id/comment	Enquiries:write	addComment
PATCH	/api/enquiry/:id/stage	Enquiries:write	changeStage
POST	/api/enquiry/:id/follow	Enquiries:read	follow
DELETE	/api/enquiry/:id/follow	Enquiries:read	unfollow
POST	/api/enquiry/:enquiryId/activity/:activityId/read	Enquiries:read	markActivityRead
POST	/api/enquiry/:id/photo	Enquiries:write, multipart upload	uploadPhoto
DELETE	/api/enquiry/:id/photo	Enquiries:write	deletePhoto
GET	/api/enquiry-requirements	EnquiryRequirements:read	getAll
GET	/api/enquiry-requirements/:id	EnquiryRequirements:read	getOne
POST	/api/enquiry-requirements	EnquiryRequirements:write	create
PUT	/api/enquiry-requirements/:id	EnquiryRequirements:write	update
DELETE	/api/enquiry-requirements/:id	EnquiryRequirements:delete	delete
GET	/api/enquiry-requirements/:requirementId/motors	EnquiryRequirements:read	listMotors
POST	/api/enquiry-requirements/:requirementId/motors	EnquiryRequirements:write	addMotor
PUT	/api/enquiry-requirements/motors/:motorId	EnquiryRequirements:write	updateMotor
DELETE	/api/enquiry-requirements/motors/:motorId	EnquiryRequirements:delete	deleteMotor
POST	/api/quotation/generate	Any authenticated	generate
POST	/api/proforma/generate	Any authenticated	generate
POST	/api/purchase-order/generate	Any authenticated	generate
GET	/api/price-list	PriceList:read	getAll
POST	/api/price-list	PriceList:write	create
PUT	/api/price-list/:priceId	PriceList:write	update

Method	Path	Access	Handler
DELETE	/api/price-list/:priceId	PriceList:delete	delete

A.4 Manufacturing (51)

Method	Path	Access	Handler
GET	/api/bom	Any authenticated	getAll
GET	/api/bom/:id	Any authenticated	getOne
POST	/api/bom	Any authenticated	create
PUT	/api/bom/:id	Any authenticated	update
DELETE	/api/bom/:id	Any authenticated	delete
GET	/api/ipt-kits	ipt_kits:read	getAll
GET	/api/ipt-kits/next-serial	ipt_kits:read	nextSerial
GET	/api/ipt-kits/:id	ipt_kits:read	getOne
POST	/api/ipt-kits	ipt_kits:write	create
PUT	/api/ipt-kits/next-serial	ipt_kits:write	setNextSerial
PUT	/api/ipt-kits/:id	ipt_kits:write	update
DELETE	/api/ipt-kits/:id	ipt_kits:delete	delete
GET	/api/parts	Parts:read	getAll
GET	/api/parts/types	Parts:read	getPartTypes
GET	/api/parts/next-code	Parts:write	previewNextCode
POST	/api/parts	Parts:write	create
PUT	/api/parts/:id	Parts:write	update
DELETE	/api/parts/:id	Parts:delete	delete
GET	/api/part-drawings	Any authenticated	getAll
POST	/api/part-drawings	Any authenticated	create
PUT	/api/part-drawings/:srNo	Any authenticated	update
DELETE	/api/part-drawings/:srNo	Any authenticated	delete
GET	/api/part-drawings-raw	Any authenticated	getAll
POST	/api/part-drawings-raw	Any authenticated	create
PUT	/api/part-drawings-raw/:srNo	Any authenticated	update
DELETE	/api/part-drawings-raw/:srNo	Any authenticated	delete
GET	/api/motor-recipes	MotorRecipes:read	getAll
GET	/api/motor-recipes/dropdown-data	MotorRecipes:read	getDropdownData
GET	/api/motor-recipes/customer/:customerId	MotorRecipes:read	getByCustomer
GET	/api/motor-recipes/:customerId/:productId	MotorRecipes:read	getOne
POST	/api/motor-recipes	MotorRecipes:write	upsert
DELETE	/api/motor-recipes/:customerId/:productId	MotorRecipes:delete	delete
GET	/api/process/orders	Processes:read	getOrders
GET	/api/process/components	Processes:read	getComponents
POST	/api/process/components	Processes:write	createComponent
POST	/api/process/components/:componentId/processes	Processes:write	createComponentProcess
POST	/api/process/work-orders/:workOrderId/components	Processes:write	createWorkOrderComponent
GET	/api/process/components/:workOrderComponentId/processes	Processes:read	getComponentProcesses
PUT	/api/process/components/:workOrderComponentId/process-status	Processes:write	updateProcessStatus

Method	Path	Access	Handler
GET	/api/process/components/:workOrderComponentId/materials	Processes:read	getComponentMaterials
POST	/api/process/components/:workOrderComponentId/materials	Processes:write	createWorkOrderMaterial
PUT	/api/process/components/:workOrderComponentId/materials/:materialId	Processes:write	updateMaterial
DELETE	/api/process/components/:workOrderComponentId/materials/:materialId	Processes:delete	deleteMaterial
GET	/api/process/work-orders/:workOrderId/stages	Processes:read	getWorkOrderStages
PUT	/api/process/work-orders/:workOrderId/stages	Processes:write	updateWorkOrderStage
GET	/api/process/work-orders/:workOrderId/testing	Processes:read	getTestingResults
PUT	/api/process/work-orders/:workOrderId/testing	Processes:write	upsertTestingResult
POST	/api/process/:orderId/work-orders	Processes:write	createWorkOrder
GET	/api/process/:orderId/instance-groups	Processes:read	getInstanceGroups
POST	/api/process/:orderId/instance-groups	Processes:write	createInstanceGroup
GET	/api/process/:orderId	Processes:read	getAll

A.5 Invoicing (9)

Method	Path	Access	Handler
POST	/api/customer-invoices	CustomerInvoices:write	create
GET	/api/customer-invoices	CustomerInvoices:read	getAll
GET	/api/customer-invoices/:id	CustomerInvoices:read	getOne
PUT	/api/customer-invoices/:id	CustomerInvoices:write	update
DELETE	/api/customer-invoices/:id	CustomerInvoices:delete	delete
GET	/api/purchase-invoices	Any authenticated	getAll
POST	/api/purchase-invoices	Any authenticated	create
PUT	/api/purchase-invoices/:invoiceId	Any authenticated	update
DELETE	/api/purchase-invoices/:invoiceId	Any authenticated	delete

A.6 Dispatch (10)

Method	Path	Access	Handler
GET	/api/dispatch-tracking	DispatchTracking:read	getAll
PUT	/api/dispatch-tracking/:trackingId	DispatchTracking:write	update
DELETE	/api/dispatch-tracking/:trackingId	DispatchTracking:delete	delete
POST	/api/delivery-challan/generate	Any authenticated	generate
GET	/api/ia-orders	ia_orders:read	getAll
GET	/api/ia-orders/check-serials	ia_orders:read	checkSerials
GET	/api/ia-orders/:id	ia_orders:read	getOne
POST	/api/ia-orders	ia_orders:write	create
PUT	/api/ia-orders/:id	ia_orders:write	update
DELETE	/api/ia-orders/:id	ia_orders:delete	delete

A.7 HR (14)

Method	Path	Access	Handler
GET	/api/attendance	attendance_history:read	getPersonal

Method	Path	Access	Handler
POST	/api/attendance	mark_attendance:write	markAttendance
GET	/api/attendance/summary	attendance_summary:read	getHRSummary
GET	/api/employee-details	employee_details:read	getAll
GET	/api/employee-details/:employee_id	employee_details:read	getOne
PATCH	/api/employee-details/:employee_id	employee_details:write	update
POST	/api/payslip/generate	Any authenticated	generate
POST	/api/invoice/generate	Any authenticated	generate
GET	/api/invoice-records	Any authenticated	getAll
POST	/api/invoice-records	Any authenticated	create
POST	/api/invoice-records/upload	multipart upload	uploadOld
GET	/api/invoice-records/:id/pdf	Any authenticated	downloadPdf
GET	/api/invoice-records/:id	Any authenticated	getOne
DELETE	/api/invoice-records/:id	Any authenticated	delete

A.8 Operations (incl. PDI) (39)

Method	Path	Access	Handler
GET	/api/queries	Any authenticated	getAll
POST	/api/queries	Any authenticated	create
PUT	/api/queries/:id/respond	Any authenticated	respond
PUT	/api/queries/:id/in-progress	Any authenticated	setInProgress
PUT	/api/queries/:id/close	Any authenticated	close
POST	/api/activities	role_id in [9, 10, 11, 12]	create
GET	/api/activities	role_id in [9, 10, 11, 12]	getAll
GET	/api/activities/:id	role_id in [9, 10, 11, 12]	getOne
PUT	/api/activities/:id	role_id in [9, 10, 11, 12]	update
DELETE	/api/activities/:id	role_id in [9, 10, 11, 12]	delete
GET	/api/problems	Problems:read	getAll
POST	/api/problems	Problems:write	create
POST	/api/problems/:id/solutions	Problems:write	addSolution
POST	/api/pdi/reports	PDI roles (admin, production)	createReport
GET	/api/pdi/reports	PDI roles (admin, production)	listReports
GET	/api/pdi/reports/:id	PDI roles (admin, production)	getReport
PATCH	/api/pdi/reports/:id	PDI roles (admin, production)	patchReport
GET	/api/pdi/reports/:id/revisions	PDI roles (admin, production)	getRevisions
POST	/api/pdi/reports/:id/duplicate	PDI roles (admin, production)	duplicateReport
POST	/api/pdi/reports/:id/finalize	PDI roles (admin, production)	finalizeReport
GET	/api/pdi/reports/:id/pdf	PDI roles (admin, production)	downloadPdf
DELETE	/api/pdi/reports/:id	PDI roles (admin, production)	deleteReport
GET	/api/pdi/report-batches	PDI roles (admin, production)	listBatches
POST	/api/pdi/report-batches	PDI roles (admin, production)	createBatch
GET	/api/pdi/report-batches/:batchId	PDI roles (admin, production)	getBatch
PATCH	/api/pdi/report-batches/:batchId	PDI roles (admin, production)	updateBatch
DELETE	/api/pdi/report-batches/:batchId	PDI roles (admin, production)	deleteBatch
POST	/api/pdi/report-batches/:batchId/finalize	PDI roles (admin, production)	finalizeBatch
GET	/api/pdi/report-batches/:batchId/pdf	PDI roles (admin, production)	downloadBatchPdf
GET	/api/pdi/admin/templates	Admin only	listTemplates

Method	Path	Access	Handler
GET	/api/pdi/admin/templates/:id	Admin only	getTemplate
POST	/api/pdi/admin/templates	Admin only	createTemplate
PUT	/api/pdi/admin/templates/:id	Admin only	saveTemplate
POST	/api/pdi/admin/templates/:id/publish	Admin only	publishTemplate
POST	/api/pdi/admin/templates/:id/archive	Admin only	archiveTemplate
POST	/api/pdi/admin/templates/:id/preview	Admin only	previewTemplate
DELETE	/api/pdi/admin/templates/:id	Admin only	deleteTemplate
GET	/api/pdi/templates	PDI roles	getTemplates
GET	/api/pdi/templates/:id/definition	PDI roles	getTemplateDefinition

A.9 Service & Repair (12)

Method	Path	Access	Handler
GET	/api/service-repair	ServiceRepair:read	getAll
GET	/api/service-repair/:id	ServiceRepair:read	getById
POST	/api/service-repair	ServiceRepair:write	create
PUT	/api/service-repair/:id	ServiceRepair:write	update
DELETE	/api/service-repair/:id	ServiceRepair:delete	remove
DELETE	/api/service-repair/:id/photo	ServiceRepair:write	deletePhoto
POST	/api/service-repair/:id/photo	ServiceRepair:write, multipart upload	uploadPhoto
POST	/api/service-repair/:id/chalan-photo	ServiceRepair:write, multipart upload	uploadChalanPhoto
POST	/api/service-repair/:id/delivery-challan-photo	ServiceRepair:write, multipart upload	uploadDeliveryChallanPhoto
POST	/api/service-repair/:id/fault-photo	ServiceRepair:write, multipart upload	uploadFaultPhoto
POST	/api/service-repair/:id/actual-issue-photo	ServiceRepair:write, multipart upload	uploadActualIssuePhoto
POST	/api/service-repair/:id/repaired-photo	ServiceRepair:write, multipart upload	uploadRepairedPhoto

A.10 Chatbot (6)

Method	Path	Access	Handler
POST	/api/orders/chatbot	Any authenticated	inline handler
POST	/api/customer-invoices/chatbot	Any authenticated	inline handler
POST	/api/customers/chatbot	Any authenticated	inline handler
POST	/api/inventory/chatbot	Any authenticated	inline handler
POST	/api/stock/chatbot	Any authenticated	inline handler
POST	/api/bom/chatbot	Any authenticated	inline handler

Appendix B - Role Permission Matrix (live data)

This matrix comes from the `permissions` table in the production database. Codes: R = can_read, W = can_write (also used for create), D = can_delete. A blank cell means the role has no row for that module, so `checkPermission` denies it (fail closed). PDI report routes do not use this table. They use the admin + production allow-list in `middleware/pdiAccess.js`.

Module	Adm	Cust	Sal	Des	Prod	Store	Disp	Acc	Emp	HR	IA-E	IA-HR	S&R	Rep
activity									RWD	RWD	RWD	RWD		
attendance										RWD		RWD		
attendance_history									R		R			
attendance_summary										R		R		
BillOfMaterials	RWD				RW	RW								
CustomerInvoices	RWD		R					RW						
Customers	RWD		R		R			R						
dashboard	R	R	R	R	R	R	R	R	R	R	R	R		
DispatchTracking	RWD		R				RW	R						
employee_details	RWD									RW		RW		
Enquiries	RWD		RW	RW										RW
EnquiryRequirements	RWD													
ia_orders											RWD	RWD		
Inventory	RWD		R	R	RW	RW								
ipt_kits	RWD				RWD									
mark_attendance									RW		RW			
MotorRecipes	RWD			RW	RW									
Orders	RWD	RW	RW		RW			R						
PartDrawings	RWD				R									
PartDrawingsRaw	RWD				R									
Parts	RWD			RW										
payslip										RW		RW		
PreDispatchInspectionReports	RWD				RW									
PriceList	RWD		R											
Problems	RWD													
Processes	RWD				RW									
PurchaseInvoices	RWD													
Queries	RWD	RW	RW		RW									
RawMaterials	RWD													
Reports	RWD													
ServiceRepair	RWD												RW	
Stock	RWD		R	R	RW	RW	R							
users										RWD		RWD		

Abbrev.	Role (role_id)
Adm	Admin (1)
Cust	Customer (2)
Sal	Sales (3)
Des	Design (4)
Prod	Production (5)
Store	store (6)
Disp	Dispatch (7)
Acc	Accounts (8)

Abbrev.	Role (role_id)
Emp	employee (9)
HR	hr (10)
IA-E	IA_employee (11)
IA-HR	IA_HR (12)
S&R	service_repair (13)
Rep	representative (15)

Appendix C - Database Schema Reference (live)

This reference was generated from PostgreSQL `information_schema` on the production database. It lists 61 tables and 474 columns. Key: PK = primary key, FK = foreign key (with the column it references), UQ = unique. Row counts are approximate (pg_stat live tuples). Defaults are shortened.

C.1 Table summary

Table	Cols	~Rows	Primary key
activities	10	50	id
activity_assignees	3	91	activity_id, user_id
activity_logs	5	658	log_id
attendance	8	2670	attendance_id
bill_of_materials	4	0	bom_id
bom_materials	6	0	bom_material_id
comment_templates	6	5	id
component_processes	6	33	process_id
components	5	14	component_id
customer_invoices	10	1	invoice_id
customers	9	35	customer_id
dispatch_tracking_details	9	2	tracking_id
employee_details	7	39	employee_id
enquiries	24	17	sr_no
enquiry_activities	9	77	activity_id
enquiry_counters	2	2	year
enquiry_read_receipts	3	238	activity_id, user_id
enquiry_requirement_motors	16	0	id
enquiry_requirements	6	0	id
enquiry_watchers	3	3	enquiry_id, user_id
ia_order_items	8	210	item_id
ia_orders	8	16	order_id
instance_groups	5	3	instance_group_id
inventory	9	2	product_id
inventory_holds	10	2	hold_id
invoices	22	1	invoice_id
ipt_kits	13	0	kit_id
motor_recipes	8	1	recipe_id
order_items	5	2	order_item_id
order_material_summary	5		
order_stage_summary	4		
order_stages	5	0	stage_id
orders	7	2	order_id
part_drawings	6	2	sr_no
part_drawings_raw	6	80	sr_no
part_serials	2	2	part_type_id
part_types	3	13	id
parts	10	0	id
pdi_report_batches	13	2	batch_id
pdi_report_revisions	6	0	revision_id

Table	Cols	~Rows	Primary key
pdi_templates	7	5	id, version
permissions	6	95	permission_id
pre_dispatch_inspection_reports	19	25	report_id
price_list	7	2	price_id
problem_solutions	5	0	solution_id
problems	4	0	id
process_material_usage	6	0	process_material_usage_id
process_status	10	78	status_id
purchase_invoices	13	0	invoice_id
queries	6	2	query_id
query_responses	5	5	response_id
raw_materials	12	80	product_id
roles	2	14	role_id
service_repair_records	24	1	record_id
users	6	100	user_id
v_activity_assignees	8		
work_order_components	4	18	work_order_component_id
work_order_materials	6	2	work_order_material_id
work_order_stages	5	40	work_order_stage_id
work_order_testing_results	7	0	testing_result_id
work_orders	6	8	work_order_id

C.2 Table definitions

activities - 10 columns, ~50 rows, indexes: idx_activities_status, activities_pkey, idx_activities_priority

Column	Type	Null	Key	Default
id	integer	N	PK	nextval('activities_id_seq')
summary	varchar	N		
status	varchar	Y		'todo'
due_date	date	Y		
priority	varchar	Y		'medium'
created_by	integer	N	FK > users.user_id	
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()
comments	text	Y		''
org	enum/user type	N		

activity_assignees - 3 columns, ~91 rows, indexes: idx_activity_assignees_user, activity_assignees_pkey

Column	Type	Null	Key	Default
activity_id	integer	N	PK, FK > activities.id	
user_id	integer	N	PK, FK > users.user_id	
assigned_at	timestamp	Y		now()

activity_logs - 5 columns, ~658 rows, indexes: activity_logs_pkey

Column	Type	Null	Key	Default
log_id	integer	N	PK	nextval('activity_logs_log_id_seq')
user_id	integer	Y		
timestamp	timestamp	Y		now()
action	varchar	N		
details	text	Y		

attendance - 8 columns, ~2670 rows, indexes: idx_attendance_date, attendance_user_id_date_key, attendance_pkey, idx_attendance_created_at_desc, idx_attendance_user_date

Column	Type	Null	Key	Default
attendance_id	integer	N	PK	nextval('attendance_attendance_id_seq'
user_id	integer	N	FK > users.user_id, UQ	
date	date	N	UQ	
check_in_time	timestamp	Y		
check_out_time	timestamp	Y		
present_absent	enum/user type	N		
created_at	timestamp	Y		CURRENT_TIMESTAMP
mode	enum/user type	Y		'office'_mode

bill_of_materials - 4 columns, ~0 rows, indexes: bill_of_materials_product_id_idx, new_bill_of_materials_pkey

Column	Type	Null	Key	Default
bom_id	integer	N	PK	nextval('new_bill_of_materials_bom_id_
product_id	integer	N	FK > inventory.product_id	
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP

bom_materials - 6 columns, ~0 rows, indexes: bom_materials_pkey, bom_materials_bom_id_idx, bom_materials_material_id_idx

Column	Type	Null	Key	Default
bom_material_id	integer	N	PK	nextval('bom_materials_bom_material_id
bom_id	integer	N	FK > bill_of_materials.bom_id	
material_id	integer	N	FK > raw_materials.product_id	
quantity_per_unit	numeric	N		
unit_price	numeric	Y		
total_value	numeric	Y		

comment_templates - 6 columns, ~5 rows, indexes: comment_templates_pkey

Column	Type	Null	Key	Default
id	integer	N	PK	nextval('comment_templates_id_seq')
title	varchar	N		
content	text	N		
is_global	boolean	Y		true
created_by	integer	Y	FK > users.user_id	
created_at	timestamp	Y		now()

component_processes - 6 columns, ~33 rows, indexes: component_processes_pkey, unique_process_per_component, idx_component_processes_component_id

Column	Type	Null	Key	Default
process_id	integer	N	PK	nextval('component_processes_process_i
component_id	integer	N	FK > components.component_id, UQ	
process_name	varchar	N		
sequence	integer	N	UQ	
responsible_person	varchar	Y		
description	text	Y		

components - 5 columns, ~14 rows, indexes: components_pkey, idx_components_is_fixed, idx_components_product_type, unique_component_name

Column	Type	Null	Key	Default
component_id	integer	N	PK	nextval('components_component_id_seq')
component_name	varchar	N	UQ	
product_type	varchar	N		'Motor'
is_fixed	boolean	N		false
created_at	timestamp	Y		CURRENT_TIMESTAMP

customer_invoices - 10 columns, ~1 rows, indexes: customer_invoices_pkey

Column	Type	Null	Key	Default
invoice_id	integer	N	PK	nextval('customer_invoices_invoice_id_
sr_no	integer	N		nextval('customer_invoices_sr_no_seq')
invoice_number	varchar	N		
issue_date	timestamp	N		CURRENT_TIMESTAMP
total_value	numeric	N		
link_pdf	varchar	Y		
order_id	integer	Y	FK > orders.order_id	
customer_id	integer	Y	FK > customers.customer_id	
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP

customers - 9 columns, ~35 rows, indexes: customers_user_id_key, customers_pkey

Column	Type	Null	Key	Default
customer_id	integer	N	PK	nextval('customers_customer_id_seq')
user_id	integer	N	FK > users.user_id, UQ	
contact_person	text	N		
city	text	N		
phone	text	N		
gst	text	Y		
shipping_address	text	N		
billing_address	text	N		
created_at	timestamp	Y		CURRENT_TIMESTAMP

dispatch_tracking_details - 9 columns, ~2 rows, indexes: dispatch_tracking_details_pkey, idx_dispatch_tracking_status

Column	Type	Null	Key	Default
tracking_id	varchar	N	PK	
sr_no	integer	N		nextval('dispatch_tracking_details_sr_
order_id	integer	Y	FK > orders.order_id	
docket_number	varchar	Y		
dispatch_date	timestamp	Y		
delivery_date	timestamp	Y		
status	enum/user type	Y		'Pending'_status
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP

employee_details - 7 columns, ~39 rows, indexes: employee_details_pkey

Column	Type	Null	Key	Default
employee_id	varchar	N	PK	
user_id	integer	N	FK > users.user_id	
phone_number	varchar	N		
date_of_joining	date	Y		
address	text	Y		
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()

enquiries - 24 columns, ~17 rows, indexes: idx_enquiries_company_name, idx_enquiries_status, idx_enquiries_assigned_to, idx_enquiries_priority, idx_enquiries_tags, idx_enquiries_created_by, enquiries_pke

Column	Type	Null	Key	Default
sr_no	integer	N	PK	nextval('enquiries_sr_no_seq')
enquiry_id	varchar	N	UQ	
company_name	varchar	N		
contact_person	varchar	Y		
mail_id	varchar	Y		
phone_no	varchar	Y		
items_required	text	Y		
Column	**Type**	**Null**	**Key**	**Default**
status	varchar	Y		'Pending'
last_discussion	timestamp	Y		
next_interaction	timestamp	Y		
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP
stage	varchar	Y		'new'
assigned_to	integer	Y	FK > users.user_id	
assigned_by	integer	Y	FK > users.user_id	
assigned_at	timestamp	Y		
due_date	timestamp	Y		
lead	varchar	Y		'lead'
source	varchar	Y		
tags	ARRAY	Y		'{}'[]
application	varchar	Y		
created_by	integer	Y	FK > users.user_id	
city	text	Y		

Column	Type	Null	Key	Default
photos	ARRAY	N		'{}'[]

enquiry_activities - 9 columns, ~77 rows, indexes: idx_activities_type, idx_activities_enquiry, enquiry_activities_pkey, idx_activities_mentions, idx_activities_created

Column	Type	Null	Key	Default
activity_id	integer	N	PK	nextval('enquiry_activities_activity_i
enquiry_id	varchar	Y	FK > enquiries.enquiry_id	
user_id	integer	N	FK > users.user_id	
activity_type	varchar	N		
message	text	N		
mentions	ARRAY	Y		'{}'[]
expected_by	timestamp	Y		
is_internal	boolean	Y		false
created_at	timestamp	Y		now()

enquiry_counters - 2 columns, ~2 rows, indexes: enquiry_counters_pkey

Column	Type	Null	Key	Default
year	integer	N	PK	
last_seq	integer	N		

enquiry_read_receipts - 3 columns, ~238 rows, indexes: enquiry_read_receipts_pkey

Column	Type	Null	Key	Default
activity_id	integer	N	PK, FK > enquiry_activities.activity_id	
user_id	integer	N	PK, FK > users.user_id	
read_at	timestamp	Y		now()

enquiry_requirement_motors - 16 columns, ~0 rows, indexes: idx_erm_controller, idx_erm_requirement_id, enquiry_requirement_motors_pkey, idx_erm_motor_type

Column	Type	Null	Key	Default
id	bigint	N	PK	nextval('enquiry_requirement_motors_id
requirement_id	bigint	N	FK > enquiry_requirements.id	
application	text	Y		
power_rating	jsonb	Y		
motor_voltage	varchar	Y		
motor_rpm	integer	Y		
motor_max_rpm	integer	Y		
Column	**Type**	**Null**	**Key**	**Default**
motor_type	varchar	Y		
em_brake	boolean	Y		false
em_brake_voltage	varchar	Y		
shaft_type	text	Y		
gvw	text	Y		
controller	varchar	Y		
controller_text	text	Y		
created_at	timestamptz	Y		now()
updated_at	timestamptz	Y		now()

enquiry_requirements - 6 columns, ~0 rows, indexes: enquiry_requirements_pkey

Column	Type	Null	Key	Default
id	bigint	N	PK	nextval('enquiry_requirements_id_seq')
enquiry_id	bigint	N		
title	text	N		
description	text	Y		
created_at	timestamptz	Y		now()
updated_at	timestamptz	Y		now()

enquiry_watchers - 3 columns, ~3 rows, indexes: enquiry_watchers_pkey

Column	Type	Null	Key	Default
enquiry_id	varchar	N	PK, FK > enquiries.enquiry_id	
user_id	integer	N	PK, FK > users.user_id	
watched_at	timestamp	Y		now()

ia_order_items - 8 columns, ~210 rows, indexes: ia_order_items_pkey, ia_order_items_hmi_serial_key, ia_order_items_vcu_serial_key

Column	Type	Null	Key	Default
item_id	integer	N	PK	nextval('ia_order_items_item_id_seq')
order_id	integer	N	FK > ia_orders.order_id	
vcu_serial	varchar	N	UQ	
hmi_imei	varchar	Y	UQ	
vcu_make	varchar	N		
vcu_model	varchar	N		
hmi_make	varchar	Y		
hmi_model	varchar	Y		

ia_orders - 8 columns, ~16 rows, indexes: ia_orders_pkey, ia_orders_invoice_number_key

Column	Type	Null	Key	Default
order_id	integer	N	PK	nextval('ia_orders_order_id_seq')
customer_name	varchar	N		
invoice_number	varchar	N	UQ	
dispatch_date	date	N		
notes	text	Y		
created_by	integer	Y	FK > users.user_id	
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()

instance_groups - 5 columns, ~3 rows, indexes: unique_instance_name_per_order, instance_groups_pkey, idx_instance_groups_order_id, idx_instance_groups_instance_type

Column	Type	Null	Key	Default
instance_group_id	integer	N	PK	nextval('instance_groups_instance_grou
order_id	integer	N	FK > orders.order_id, UQ	
instance_name	varchar	N	UQ	
instance_type	varchar	N		
created_at	timestamp	Y		CURRENT_TIMESTAMP

inventory - 9 columns, ~2 rows, indexes: inventory_pkey, inventory_part_number_unique

Column	Type	Null	Key	Default
product_id	integer	N	PK	nextval('inventory_product_id_seq')
stock_quantity	integer	Y		
price	numeric	Y		
created_at	timestamp	Y		now()
product_name	varchar	N		
description	text	Y		
product_code	varchar	N		
returnable_qty	integer	N		0
part_number	varchar	Y	UQ	

inventory_holds - 10 columns, ~2 rows, indexes: inventory_holds_pkey, idx_inventory_holds_product_active

Column	Type	Null	Key	Default
hold_id	integer	N	PK	nextval('inventory_holds_hold_id_seq')
product_id	integer	N	FK > inventory.product_id	
quantity	integer	N		
reason	text	N		
reference_type	varchar	Y		
reference_value	text	Y		
status	varchar	N		'ACTIVE'
created_by	integer	Y		
created_at	timestamptz	N		now()
released_at	timestamptz	Y		

invoices - 22 columns, ~1 rows, indexes: invoices_pkey, invoices_invoice_number_key

Column	Type	Null	Key	Default
invoice_id	integer	N	PK	nextval('invoices_invoice_id_seq')
invoice_number	varchar	N	UQ	
invoice_date	date	N		
order_no	varchar	Y		
order_date	date	Y		
billing_name	varchar	N		
billing_address	text	Y		
Column	**Type**	**Null**	**Key**	**Default**
billing_phone	varchar	Y		
billing_email	varchar	Y		
billing_gst	varchar	Y		
hsn	varchar	Y		
vendor_code	varchar	Y		
gst_percent	numeric	N		18
items	jsonb	N		
total_amount	numeric	N		0
gst_amount	numeric	N		0
grand_total	numeric	N		0
created_by	integer	Y	FK > users.user_id	
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()
source	varchar	N		'generated'

Column	Type	Null	Key	Default
drive_file_id	varchar	Y		

ipt_kits - 13 columns, ~0 rows, indexes: ipt_kits_dcdc_serial_key, ipt_kits_vcu_serial_key, ipt_kits_cluster_serial_key, ipt_kits_harness_serial_key, ipt_kits_gearbox_serial_key, ipt_kits_controller_se

Column	Type	Null	Key	Default
kit_id	integer	N	PK	nextval('ipt_kits_kit_id_seq')
kit_serial	varchar	N	UQ	
motor_serial	varchar	N	UQ	
controller_serial	varchar	N	UQ	
gearbox_serial	varchar	N	UQ	
harness_serial	varchar	N	UQ	
cluster_serial	varchar	N	UQ	
vcu_serial	varchar	N	UQ	
dcdc_serial	varchar	N	UQ	
created_by	integer	Y	FK > users.user_id	
updated_by	integer	Y	FK > users.user_id	
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()

motor_recipes - 8 columns, ~1 rows, indexes: motor_recipes_pkey, uq_motor_recipes_customer_product

Column	Type	Null	Key	Default
recipe_id	integer	N	PK	nextval('motor_recipes_recipe_id_seq')
customer_id	integer	N	FK > customers.customer_id, UQ	
product_id	integer	N	FK > inventory.product_id, UQ	
num_turns	integer	N		
num_coils	integer	N		
notes	text	Y		
created_at	timestamptz	N		now()
updated_at	timestamptz	N		now()

order_items - 5 columns, ~2 rows, indexes: order_items_pkey

Column	Type	Null	Key	Default
order_item_id	integer	N	PK	nextval('order_items_order_item_id_seq
order_id	integer	Y	FK > orders.order_id	
product_id	integer	Y	FK > inventory.product_id	
quantity	integer	Y		
price	numeric	Y		

order_material_summary - 5 columns

Column	Type	Null	Key	Default
work_order_id	integer	Y		
component_id	integer	Y		
raw_material_id	integer	Y		
product_name	varchar	Y		
quantity	integer	Y		

order_stage_summary - 4 columns

Column	Type	Null	Key	Default
order_id	integer	Y		
stage_name	varchar	Y		
status	text	Y		
last_completed_date	date	Y		

order_stages - 5 columns, ~0 rows, indexes: unique_stage_per_order, order_stages_pkey

Column	Type	Null	Key	Default
stage_id	integer	N	PK	nextval('order_stages_stage_id_seq')
order_id	integer	N	FK > orders.order_id, UQ	
stage_name	varchar	N	UQ	
stage_date	date	Y		
created_at	timestamp	Y		CURRENT_TIMESTAMP

orders - 7 columns, ~2 rows, indexes: orders_pkey

Column	Type	Null	Key	Default
order_id	integer	N	PK	nextval('orders_order_id_seq')
user_id	integer	Y		
target_delivery_date	date	Y		
created_at	timestamptz	Y		now()
status	enum/user type	Y		
payment_status	varchar	Y		
status_reason	text	Y		

part_drawings - 6 columns, ~2 rows, indexes: part_drawings_pkey, part_drawings_drawing_id_key

Column	Type	Null	Key	Default
sr_no	integer	N	PK	nextval('part_drawings_sr_no_seq')
drawing_id	integer	N	UQ	nextval('part_drawings_drawing_id_seq'
item_name	varchar	Y		
drawing_link	varchar	Y		
product_id	integer	N	FK > inventory.product_id	
updated_at	timestamp	Y		

part_drawings_raw - 6 columns, ~80 rows, indexes: part_drawings_raw_pkey, part_drawings_raw_product_id_key, part_drawings_raw_drawing_id_key

Column	Type	Null	Key	Default
sr_no	integer	N	PK	nextval('part_drawings_raw_sr_no_seq')
drawing_id	integer	N	UQ	nextval('part_drawings_raw_drawing_id_
item_name	varchar	Y		
drawing_link	varchar	Y		
product_id	integer	N	FK > raw_materials.product_id, UQ	
updated_at	timestamp	Y		

part_serials - 2 columns, ~2 rows, indexes: part_serials_pkey

Column	Type	Null	Key	Default
part_type_id	integer	N	PK, FK > part_types.id	
last_serial	bigint	N		0

part_types - 3 columns, ~13 rows, indexes: part_types_pkey, part_types_type_name_key, part_types_prefix_key

Column	Type	Null	Key	Default
id	integer	N	PK	nextval('part_types_id_seq')
type_name	text	N	UQ	
prefix	text	N	UQ	

parts - 10 columns, ~0 rows, indexes: parts_pkey, idx_parts_part_type_id, parts_part_code_key

Column	Type	Null	Key	Default
id	integer	N	PK	nextval('parts_id_seq')
part_code	text	N	UQ	
part_type_id	integer	N	FK > part_types.id	
name	text	N		
description	text	N		
drawing_no	text	N		
customer_part_no	text	Y		
supplier_part_no	text	Y		
created_at	timestamp	Y		now()
updated_at	timestamp	Y		now()

pdi_report_batches - 13 columns, ~2 rows, indexes: pdi_report_batches_pkey

Column	Type	Null	Key	Default
batch_id	integer	N	PK	nextval('pdi_report_batches_batch_id_s
template_id	text	N		
pdi_no	text	N		
lot_quantity	integer	N		
status	text	N		'In Progress'
drive_file_id	text	Y		
created_by	integer	Y	FK > users.user_id	
created_at	timestamptz	N		now()
customer_name	text	Y		
product_id	text	Y		
product_specifications	text	Y		
drawing_no	text	Y		
controller_type	text	Y		

pdi_report_revisions - 6 columns, ~0 rows, indexes: pdi_report_revisions_pkey

Column	Type	Null	Key	Default
revision_id	integer	N	PK	nextval('pdi_report_revisions_revision
report_id	integer	N	FK > pre_dispatch_inspection_reports.report_id	
revision_no	integer	N		
data	jsonb	N		
edited_by	integer	Y	FK > users.user_id	
edited_at	timestamptz	N		now()

pdi_templates - 7 columns, ~5 rows, indexes: pdi_templates_pkey, idx_pdi_templates_id_version

Column	Type	Null	Key	Default
id	text	N	PK	
version	integer	N	PK	
name	text	N		
status	text	N		'draft'
definition	jsonb	N		
created_by	integer	Y	FK > users.user_id	
created_at	timestamp	N		now()

permissions - 6 columns, ~95 rows, indexes: permissions_pkey

Column	Type	Null	Key	Default
permission_id	integer	N	PK	nextval('permissions_permission_id_seq
role_id	integer	Y		
can_read	boolean	Y		false
can_write	boolean	Y		false
can_delete	boolean	Y		false
module	varchar	N		

pre_dispatch_inspection_reports - 19 columns, ~25 rows, indexes: pre_dispatch_inspection_reports_pkey, idx_pdi_reports_batch_id, uq_pdi_reports_batch_lot_index, unique_order_id

Column	Type	Null	Key	Default
sr_no	integer	N		nextval('pre_dispatch_inspection_repor
customer_id	integer	Y	FK > customers.customer_id	
order_id	integer	Y	FK > orders.order_id, UQ	
report_id	integer	N	PK	nextval('pre_dispatch_inspection_repor
status	varchar	N		'Pending'
inspected_by	varchar	Y		
inspection_date	timestamp	Y		
Column	**Type**	**Null**	**Key**	**Default**
created_at	timestamp	Y		CURRENT_TIMESTAMP
report_link	varchar	Y		
data	jsonb	N		'{}'
photos	jsonb	N		'[]'
template_id	text	N		'general'
drive_file_id	text	Y		
template_version	integer	Y		
prepared_by	text	Y		
approved_by	text	Y		
batch_id	integer	Y	FK > pdi_report_batches.batch_id	
lot_index	integer	Y		
revision_no	integer	N		1

price_list - 7 columns, ~2 rows, indexes: price_list_pkey, price_list_product_id_key

Column	Type	Null	Key	Default
price_id	integer	N	PK	nextval('price_list_price_id_seq')
sr_no	integer	N		nextval('price_list_sr_no_seq')
item_description	varchar	N		
price	numeric	N		
product_id	integer	N	FK > inventory.product_id, UQ	
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP

problem_solutions - 5 columns, ~0 rows, indexes: idx_problem_solutions_problem_id, problem_solutions_pkey

Column	Type	Null	Key	Default
solution_id	integer	N	PK	nextval('problem_solutions_solution_id
problem_id	integer	N	FK > problems.id	
solution_description	text	N		
is_successful	boolean	N		false
attempted_at	timestamptz	Y		CURRENT_TIMESTAMP

problems - 4 columns, ~0 rows, indexes: idx_problems_product_id, problems_pkey

Column	Type	Null	Key	Default
id	integer	N	PK	nextval('problems_id_seq')
product_id	integer	N	FK > inventory.product_id	
problem_description	text	N		
created_at	timestamptz	Y		CURRENT_TIMESTAMP

process_material_usage - 6 columns, ~0 rows, indexes: process_material_usage_pkey, unique_usage_per_component_process

Column	Type	Null	Key	Default
process_material_usage_id	integer	N	PK	nextval('process_material_usage_proces
process_id	integer	N	FK > component_processes.process_id, UQ	
raw_material_id	integer	N	FK > raw_materials.product_id, UQ	
used_quantity	integer	N		
created_at	timestamp	Y		CURRENT_TIMESTAMP
work_order_component_id	integer	N	FK > work_order_components.work_order_component_id, UQ	

process_status - 10 columns, ~78 rows, indexes: unique_process_status_per_component, process_status_pkey, idx_process_status_process_id

Column	Type	Null	Key	Default
status_id	integer	N	PK	nextval('process_status_status_id_seq'
process_id	integer	N	FK > component_processes.process_id, UQ	
status	varchar	N		'Pending'
completed_quantity	integer	N		0
completion_date	date	Y		
raw_quantity_used	integer	Y		
responsible_person	text	Y		
in_use_quantity	integer	N		0
work_order_component_id	integer	N	FK > work_order_components.work_order_component_id, UQ	
allowed_quantity	integer	N		0

purchase_invoices - 13 columns, ~0 rows, indexes: purchase_invoices_pkey, purchase_invoices_sr_no_idx, idx_purchase_invoices_product_id

Column	Type	Null	Key	Default
invoice_id	integer	N	PK	nextval('purchase_invoices_invoice_id_
supplier_code	varchar	Y		
supplier_name	varchar	Y		
invoice_number	varchar	N		
issue_date	timestamp	N		CURRENT_TIMESTAMP
description	varchar	Y		
unit_price	numeric	N		
quantity	numeric	N		
link_pdf	varchar	Y		
created_at	timestamp	Y		CURRENT_TIMESTAMP
updated_at	timestamp	Y		CURRENT_TIMESTAMP
product_id	integer	Y	FK > raw_materials.product_id	
sr_no	integer	N		nextval('purchase_invoices_sr_no_seq')

queries - 6 columns, ~2 rows, indexes: queries_pkey

Column	Type	Null	Key	Default
query_id	integer	N	PK	nextval('queries_query_id_seq')
user_id	integer	Y		
date_of_query_raised	timestamp	Y		now()
last_updated	timestamp	Y		now()
query_text	text	N		
query_status	varchar	Y		

query_responses - 5 columns, ~5 rows, indexes: query_responses_pkey

Column	Type	Null	Key	Default
response_id	integer	N	PK	nextval('query_responses_response_id_s
query_id	integer	Y		
responded_by	integer	Y		
response_date	timestamp	Y		now()
response	text	N		

raw_materials - 12 columns, ~80 rows, indexes: raw_materials_pkey, raw_materials_product_code_key, raw_materials_part_number_unique

Column	Type	Null	Key	Default
product_id	integer	N	PK	nextval('raw_materials_product_id_seq'
stock_quantity	numeric	N		
price	numeric	Y		
created_at	timestamp	N		CURRENT_TIMESTAMP
product_name	varchar	N		
description	text	Y		
product_code	varchar	N	UQ	
qty_required	integer	Y		0
location	varchar	Y		
image_url	text	Y		
returnable_qty	numeric	N		0.00
part_number	varchar	Y	UQ	

roles - 2 columns, ~14 rows, indexes: roles_pkey

Column	Type	Null	Key	Default
role_id	integer	N	PK	nextval('roles_role_id_seq')
role_name	varchar	N		

service_repair_records - 24 columns, ~1 rows, indexes: service_repair_records_pkey

Column	Type	Null	Key	Default
record_id	integer	N	PK	nextval('service_repair_records_record
customer_name	varchar	N		
dispatch_material_no	varchar	Y		
part_details	text	Y		
chalan_photo_url	text	Y		
receiving_quantity	numeric	Y		
dispatch_quantity	numeric	Y		
Column	**Type**	**Null**	**Key**	**Default**
fault_query	text	Y		
actual_issue	text	Y		
delivery_challan_photo_url	text	Y		
repaired_photos_urls	ARRAY	Y		
repair_status	varchar	N		'pending'
responsibility_person	varchar	Y		
testing_responsibility	varchar	Y		
remarks	text	Y		
sent_date	date	Y		

Column	Type	Null	Key	Default
created_by	integer	Y	FK > users.user_id	
created_at	timestamp	N		now()
updated_at	timestamp	N		now()
service_type	varchar	N		'repair'
actual_issue_photos_urls	ARRAY	Y		
chalan_photos_urls	ARRAY	Y		
delivery_challan_photos_urls	ARRAY	Y		
fault_photos_urls	ARRAY	Y		

users - 6 columns, ~100 rows, indexes: unique_email, users_pkey

Column	Type	Null	Key	Default
user_id	integer	N	PK	nextval('users_user_id_seq')
role_id	integer	Y		
created_at	timestamp	Y		now()
name	varchar	N		
email	varchar	N	UQ	
password_hash	text	N		

v_activity_assignees - 8 columns

Column	Type	Null	Key	Default
activity_id	integer	Y		
summary	varchar	Y		
status	varchar	Y		
due_date	date	Y		
user_id	integer	Y		
assignee_name	varchar	Y		
assignee_email	varchar	Y		
assigned_at	timestamp	Y		

work_order_components - 4 columns, ~18 rows, indexes: unique_component_per_work_order, work_order_components_pkey

Column	Type	Null	Key	Default
work_order_component_id	integer	N	PK	nextval('work_order_components_work_or
work_order_id	integer	N	FK > work_orders.work_order_id, UQ	
component_id	integer	N	FK > components.component_id, UQ	
quantity	integer	N		

work_order_materials - 6 columns, ~2 rows, indexes: unique_material_per_component_instance, idx_work_order_materials_work_order_id, work_order_materials_pkey

Column	Type	Null	Key	Default
work_order_material_id	integer	N	PK	nextval('work_order_materials_work_ord
work_order_id	integer	N		
quantity	integer	N		
raw_material_id	integer	N	FK > raw_materials.product_id, UQ	
component_id	integer	N		
work_order_component_id	integer	N	FK > work_order_components.work_order_component_id, UQ	

work_order_stages - 5 columns, ~40 rows, indexes: work_order_stages_pkey, unique_stage_per_work_order

Column	Type	Null	Key	Default
work_order_stage_id	integer	N	PK	nextval('work_order_stages_work_order_
work_order_id	integer	N	FK > work_orders.work_order_id, UQ	
stage_name	varchar	N	UQ	
stage_date	date	Y		
created_at	timestamp	Y		CURRENT_TIMESTAMP

work_order_testing_results - 7 columns, ~0 rows, indexes: work_order_testing_results_work_order_id_testing_type_key, work_order_testing_results_pkey

Column	Type	Null	Key	Default
testing_result_id	integer	N	PK	nextval('work_order_testing_results_te
work_order_id	integer	N	FK > work_orders.work_order_id, UQ	
testing_type	varchar	N	UQ	
qty	integer	Y		
test_date	date	Y		
controller_type	varchar	Y		
updated_at	timestamp	Y		CURRENT_TIMESTAMP

work_orders - 6 columns, ~8 rows, indexes: idx_work_orders_instance_group_id, work_orders_pkey, idx_work_orders_order_id

Column	Type	Null	Key	Default
work_order_id	integer	N	PK	nextval('work_orders_work_order_id_ seq
order_id	integer	N	FK > orders.order_id	
instance_group_id	integer	Y	FK > instance_groups.instance_group_id	
target_date	date	Y		
status	varchar	Y		'Pending'
created_at	timestamp	Y		CURRENT_TIMESTAMP

Appendix D - Glossary

Term	Meaning
PDI	Pre-Dispatch Inspection: the quality inspection report filled in before a product ships
Lot / batch	A group of PDI reports for several units that share header fields and are finalized together into one combined PDF
AutoNXT	A customer product family with dedicated PDI templates (motor and controller), tolerance rules and batch forms
IPT kit	An assembled kit tracked by serial number through the ipt_kits module and its serial sequence
BOM	Bill of Materials: the raw materials and quantities needed to build a product
Work order	A production instruction raised against a sales order; it tracks components, processes, stages and testing
Proforma	A proforma invoice: a pre-sale commercial document generated as a PDF
Delivery challan	The document that goes with goods being dispatched
IA	Intute AI: the second company sharing the platform, with its own roles and modules
Compage	The manufacturing company: the 9 ERP roles plus hr and employee
RBAC	Role-Based Access Control, enforced by the permissions table and role allow-lists
Hold	A reservation of inventory quantity against a purpose, which reduces available stock until it is released